XIMOD Translator

User Manual

Cross-platform mod translation tool for Bethesda games

Skyrim · Fallout · Starfield · Oblivion · Morrowind

Version 2.0 — XIMOD Translator 0.3.0 (end of stage 2)

Cross-platform: Windows · Linux · macOS

Contents

1. Introduction 5
1.1 Who this tool is for 5
1.2 Supported games 5
1.3 Supported formats 5
2. Installation and startup 5
2.1 Contents of the application folder 5
2.2 Starting the application 6
2.3 First launch 6
2.4 Detecting games 6
2.5 Where your settings are stored: portable or per user 7
2.6 Interface languages 7
3. Key concepts 8
4. Interface overview 8
4.1 The menu bar 8
4.2 The toolbar 9
4.3 The filter bar 9
4.4 Tabs 9
4.5 The status bar 9
4.6 Window behavior 10
4.7 The session log 10
5. Quick start 11
6. The toolbar: Game, Source, Target, Output 12
6.1 Output modes 12
6.2 Changing the Game 13
6.3 The Source: the languages the file provides 13
6.4 The Target: by country 14
6.5 The Output: requested at export 14
6.6 Undo and redo 14
7. Opening files 14
7.1 Open, drag and drop, recent files 14
7.2 Localized plugins and STRINGS 15
7.3 Auto-translation: the dictionary stack 15
7.4 Several documents 16
7.5 Recovery after an abnormal shutdown 18
7.6 The mod dictionary: the working memory 18
7.7 Interface files and text formats 19
7.8 Import with a format 20
7.9 Analyse a folder 21
8. The translation grid 23
8.1 The columns 23
8.2 Entering a translation 24

8.3 Book preview 26
8.4 “Unique lines” view 26
8.5 The edit window 26
8.6 Find and replace 30
8.7 Quality checks 32
8.8 Dialogues 32
8.9 Voices 34
8.10 Translating online 34
8.11 Reviewing a translation with someone else 38
8.12 History and going back 39
8.13 Scripts 40
8.14 Exclusion rules 45
9. Statuses and their colors 47
9.1 The provenance of translations 48
10. Filters and counters 49
10.1 Column filters 50
10.2 Advanced search 51
10.3 Concordance 52
11. Saving and exporting 53
11.1 Save as… 53
11.2 Export translation… 54
11.3 Apply the dictionaries… 55
11.4 Ambiguous sources 56
11.5 Apply the assistants: glossary, patterns, derivations 56
12. The Dictionaries menu 59
12.1 Before you start 59
12.2 Menu overview 60
12.3 Converting between XIMOD and external tools 61
12.4 Building a dictionary 62
12.5 Exporting a game dictionary to STRINGS 63
12.6 Managing games 63
12.7 Worked examples 63
12.8 Managing dictionaries: the tree 63
12.9 Text-only databases 64
12.10 My personal dictionary and the dictionary editor 64
12.11 Mod update 65
12.12 Compare two dictionaries 66
13. The dictionary viewer 66
14. The spell checker 67
14.1 Installing a dictionary 67
14.2 Opening and using the spell checker 67
14.3 The personal dictionary 67
15. The explorers (Tools menu) 67

15.1 Tools ▸ Data ▸ Translatable fields per game… 67
15.2 Tools ▸ Data ▸ Text formats… 68
15.3 Tools ▸ Translate a FOMOD… 69
15.4 Tools ▸ Package a translation… 70
15.5 Tools ▸ Translate the interface… 71
15.6 Tools ▸ Data ▸ Replacement matrices… 72
15.7 Tools ▸ Data ▸ Glossary, patterns and derivations… 73
16. Settings 78
16.1 General tab 79
16.2 Recent files tab 79
16.3 Advanced tab 80
16.4 Online tab 80
16.5 Keyboard navigation 80
16.6 Options ▸ Properties…: the country / language database 82
17. The configuration file 83
18. The command line 84
18.1 Common options and return codes 85
18.2 The commands 86
18.3 The batch file 87
18.4 The manifest 87
18.5 Scripts, continuous integration and keys 88
19. Localization of Bethesda games 88
19.1 Where the application finds the STRINGS 89
19.2 Knowing the language of a plugin 89
19.3 Adding a language that Bethesda does not offer to a game 89
19.4 Localizing a mod 90
19.5 Encodings and code pages 91
20. Frequently asked questions 93
21. Glossary 93
Appendix — Summary of formats 94

1. Introduction

XIMOD Translator is a desktop tool for translating mods (extensions) of Bethesda games. It reads the plugins and the translation databases produced by the community's reference tools, presents each text string in an editable grid, then writes back a translated version ready to be installed in the game.

Designed to be cross-platform (Windows, Linux, macOS) and free of proprietary dependencies, it brings the mod translator's complete workflow together in a single application: opening, dictionary-assisted translation, spell checking, status management and export in the formats the game engines expect. It is the companion of XIMOD Architect (FOMOD installer creation). The interface is available in French and English.

1.1 Who this tool is for

Anyone who wants to translate a mod, whether a beginner or experienced. No programming knowledge is required: you open a file, type the translations in a grid, and the tool takes care of the rest. Experienced translators will find the concepts of xTranslator and ESP-ESM Translator (statuses, databases, STRINGS, FormID).

1.2 Supported games

XIMOD Translator knows the following Bethesda games:

- Morrowind, Oblivion and **Oblivion Remastered**;
- Fallout 3, Fallout New Vegas, Fallout 4, Fallout 76;
- Skyrim (Legendary Edition) and Skyrim Special Edition / Anniversary Edition;
- Starfield.

The *Game* you select determines the character set (encoding) and the dictionaries loaded. The chapter “Localization of Bethesda games” explains how each generation stores its texts.

1.3 Supported formats

As input, XIMOD Translator reads:

- translation databases `.sst` (xTranslator), `.eet` (ESP-ESM Translator), `.dsd` (Dynamic String Distributor), `.json` (pivot or DSD) and `.xml` (EET, xTranslator and native dialects);
- plugins `.esp` / `.esm` / `.esl` (extraction of the translatable strings);
- string tables `.strings` / `.dlstrings` / `.ilstrings`;
- archives `.bsa` / `.ba2` (through the archive explorer).

As output: `.sst`, `.eet`, `.dsd`, `.json`, `.csv`, XML, as well as translated plugins and STRINGS files. The complete table is in the appendix.

2. Installation and startup

2.1 Contents of the application folder

The application comes as a self-contained folder containing:

- the executable `ximod-translator` (or `ximod-translator.exe` on Windows) and, next to it, the command line `ximod-cli` (`ximod-cli.exe`, chapter 18), with its scripts and examples in `docs/cli/`;
- the `assets/` folder with the shipped data: interface languages (`assets/locales`), game sheets and translatable field tables (`assets/data/games`, `assets/data/fields`), fonts, images and the splash screen (`assets/images/splash.png`); the dictionaries (`assets/bases/jeux` and `assets/bases/mods`) are stored there in portable mode;
- the `Manuals/` folder: this manual in French (`XIMOD_Translator_Manuel_FRA.docx`) and in English (`XIMOD_Translator_Manual_ENG.docx`), which *F1* opens in the interface language (§ 2.6);
- optionally the `Config.ini` file: its presence next to the executable makes the application run in **portable mode** (§ 2.5).

> **Note —** No system installation is needed: copy the folder wherever you want and run the executable. On Windows, an installer is also offered (installation in Program Files, shortcuts, optional association of `.ximdb` files); it installs no game dictionary, which you build from the Dictionaries menu or install separately. The application degrades gracefully if an optional file is missing (built-in fonts by default, no splash, default icon).

2.2 Starting the application

- **Windows**: double-click `ximod-translator.exe`.
- **Linux**: make the file executable, then run it (`./ximod-translator`). The usual egui/OpenGL graphics libraries (X11/Wayland) must be present.
- **macOS**: a single universal `.dmg` image (Apple Silicon and Intel); open it and drag “XIMOD Translator” to Applications — the same bundle is also offered alone, as an `.app.zip`. Since the application is not signed, you launch it the first time by right-clicking ▸ *Open*, then confirming.
- **Open with**: one or more files passed as arguments (or dropped on the executable) open at startup.

2.3 First launch

1. A splash screen is displayed while the application detects the installed games and prepares the dictionaries. Its duration can be set in the Settings (it can be set to zero). On macOS, the image is not shown: the application simply waits for this preparation to finish before opening its window.
2. Open *Options* ▸ *Settings…* and, if needed, fill in the folders of the external tools and of the databases.
3. Choose the *Game* in the toolbar (at launch, the four lists are empty), then the country of the target language with the *Target* button and the *Output* mode: with no document open, these choices apply to the next file.

4. Open a file to translate (*File* ▸ *Open…* or drag and drop) — these commands become available only once a game is chosen. The *Source* language fills itself in from the file.

2.4 Detecting games

The *Dictionaries* ▸ *Detect installed games…* menu automatically searches for your games on all platforms: Steam (Windows, Linux — native, Flatpak and Snap installations and secondary libraries, Proton games included — and macOS), GOG (Windows registry, GOG Galaxy database on Windows and macOS), Epic Games (Windows, macOS), Heroic Games Launcher (GOG, Epic and Amazon, on Linux, macOS and Windows), Lutris (Linux), Xbox Game Pass (Windows), then a scan of folders. Each location is kept only if the game's master file is there (`Skyrim.esm`, `Fallout4.esm`…); a Windows game installed through Wine or Proton is validated on its real folder. *Add a game…* lets you designate one manually by pointing to its installation folder; a game added this way is locked and never overwritten by detection. The registered games feed the *Game* selector.

2.5 Where your settings are stored: portable or per user

The application writes its configuration, your personal dictionaries, its backup files, the data files you modify and its log to a **configuration folder** that depends on the mode:

Mode	Configuration folder
Portable	The executable's folder, as soon as a Config.ini is in it. This is the case for existing installations: nothing changes for them.
Per user	Windows: %APPDATA%\XIMOD Translator; Linux: ~/.config/ximod-translator; macOS: ~/Library/Application Support/XIMOD Translator. This is the mode of an installation in Program Files or in Applications.

The folder contains `Config.ini` and the subfolders `data/` (modified data files, for example the field tables, and the default dictionaries in per-user mode), `autosave/` (backup files), `dict/` (personal dictionaries) and `logs/` (session log `ximod.log`, rewritten at each start, and startup log `ximod_startup.log` when something went wrong). You switch from one mode to the other in *Settings* ▸ *Advanced* (§ 16.3): the configuration is copied and the change takes effect at the next startup. If the folder is not writable, the application starts anyway, with the configuration read-only, and says so.

2.6 Interface languages

The interface exists in thirty-two languages, those of XIMOD Architect except Arabic: Bulgarian, Chinese, Croatian, Czech, Danish, Dutch, English, Estonian, Finnish, French, German, Greek, Hungarian, Irish, Italian, Japanese, Korean, Latvian, Lithuanian, Maltese, Norwegian (Bokmål, and “nor”), Polish, Portuguese, Romanian, Romansh, Russian, Slovak, Slovenian, Spanish, Swedish and Turkish. Each language is a file `assets/locales/<code>/main.ftl` (ISO 639-3 code) headed by a block giving the code, the name of the language, the font and the author. It is chosen in *Settings* ▸ *General* ▸ *Interface language* (§ 16.1); the change applies at once, without restarting, including to the generated comments of the grid. Arabic is already translated and its file is shipped, but it is offered neither in this list nor as displayed language of the translation editor until the interface can be drawn right to left; a Config.ini that still names it opens the interface in English.

The terms shared by both applications (menus, buttons, settings, countries and languages) reuse the translation of Architect, reviewed by its community. The others were machine-translated: the header says so (`# @author = … (traduction automatique, à relire)`) and the interface translation editor (§ 15.5) lets anyone correct them. A message missing from a language is shown in English, the reference language; French is a complete translation of it.

Each language is displayed in its font (`Languages.json`, `font` field): Noto Sans for the Latin, Cyrillic and Greek scripts, Noto Sans JP, KR and SC for Japanese, Korean and Chinese, all loaded at start-up from `assets/fonts/`. Every text of the application is in these files, none is written in the program: menus and buttons, but also the error messages of the file readers and archives, size units (B, KB, MB), the date format, the names of the modifier keys (Ctrl, Shift, Alt), the punctuation that joins two messages (":" with no space in English, with a space in French). Only technical identifiers (record signatures, file names, addresses) and the diagnostic log, written in English, stay outside the language files. The manual exists in French and English: *Help ▸ Manual* (*F1*) opens the one in the interface language, English for the other languages.

To know before translating whether your language is in the database, under which ISO 639-3 code it is recorded, with which font and in which countries it is offered, open *Options ▸ Properties…* (§ 16.6).

3. Key concepts

A few notions are enough to understand how the whole tool works.

Term	What to remember
Plugin	The mod's file (.esp/.esm/.esl). It contains the game objects and, sometimes, their texts.
Translation database	A file that associates original texts with their translation (.sst, .eet, .dsd, .xml…). It serves as a reusable memory.
Dictionary (.ximdb)	XIMOD's own compact database, stored per game. It automatically pre-fills the translations already known.
STRINGS	The external text tables (.STRINGS/.DLSTRINGS/.ILSTRINGS) of recent games; one per language.
Localized / non-localized	A "localized" plugin stores its text in the STRINGS and keeps only identifiers; a non-localized plugin contains the text in plain form.
Status	The state of a line (to translate, validated, pending…), shown by a color and a glyph.
Output mode	The format produced on export (full plugin, patch, STRINGS, DSD…).
FormID / EDID	Technical identifiers of a record, used to match a translation precisely to its string.

4. Interface overview

From top to bottom, the main window is organized into several areas.

4.1 The menu bar

Five menus: *File* (open, save, load a review, export, recent files, exit), *Options* (settings, properties — the country / language database —, colors, special characters, optional columns, spell checker, databases and applying dictionaries), *Dictionaries* (conversions, creation, viewing, tree and editing of dictionaries), *Tools* (find and replace, checks, dialogues, online translation, explorers and, under *Data*, the data file editors) and *Help* (manual, session log, about).

Menu shortcuts. The same as in XIMOD Architect, shown to the right of the entries (⌘ on macOS): *Ctrl+O* Open…, *Ctrl+S* Save as…, *Ctrl+Shift+E* Export translation…, *Ctrl+W* Close document, *Ctrl+Q* Exit, *Ctrl+,* Settings…, *Ctrl+H* Find and replace…, *F12* Quality checks…, *F1* User manual (the file in the `Manuals` folder in the interface language), *Ctrl+Shift+J* Session log… (§ 4.7). They do nothing while a dialog is open. Closing a document or exiting with unsaved changes first asks: Save as… (Enter), Close without saving, Cancel (Esc).

4.2 The toolbar

Four lists: *Game*, *Source* (original language), *Target* (translation language) and *Output* (export format), followed by the ↺ *Undo* and ↻ *Redo* buttons. They show and change the settings of the **active document**; with no document, the settings of the next file. The label on the right says which ("Settings of MonMod.esp" / "Settings of the next file"). When the application starts, and as soon as the last tab is closed, the four lists are empty. *Source* offers only the languages the open file provides; *Target* is a button that opens the flags window, and the list to its right contains only the official languages of the chosen country; *Output* can stay empty until export, which asks for it. Only the *Game* is required to open files. Details in chapter 6.

4.3 The filter bar

The *Filter* field searches for a text in the target chosen in the *In* list: *All* (source, translation, EDID and record signature), *Source*, *Translation*, *EDID or FormID* — a text of six to eight hexadecimal digits, with or without `0x`, is compared with the FormID, with or without the master index ("0001A3F2" and "01A3F2" find the same line); any other text is searched for in the EDID —, *Signature* ("WEAP:FULL", "WEAP" or ":FULL"), or *Comment* — the text of the Comment column (§ 8.1), author's comment and generated text combined, which allows "voice ✗", "female", "stage 20" or "master: Skyrim.esm"; this target triggers the context pass (§ 8.8) if it has not run yet, and works whether or not the column is shown. The `.*` box reads the field as a regular expression (Rust syntax, case-insensitive; `(?-i)` at the start to match case): "(dragon|draugr)" finds both. The filter applies a quarter of a second after the last keystroke, or immediately on Enter; an invalid expression does not empty the grid: the view keeps the last valid filter and the field turns red, with the error in a tooltip. *Ctrl+F* gives focus to the field and selects its content, *Esc* in the field clears it, as does the red cross; *Reset* sets the target back to All and turns off the expression. The target and the option last for the session. After the statuses, the *[!] Anomalies* button keeps only the

lines flagged by the last quality check (§ 8.7, F12); Reset releases it. On a 50,000-line file, a filter applies in a few milliseconds, and the view is recomputed only when the filter or the document changes.

Located below the toolbar, it appears as soon as a file is open and restricts the lines displayed (chapter 10).

4.4 Tabs

Each open file gets its own tab; click it to switch, or click the cross to close it. Next to the name, a discreet badge recalls its settings (“Skyrim SE · ENG→FRA · STRINGS”), and a ● dot precedes the name as long as the content differs from what was saved. Right-clicking the tab gives access to the *Use my personal dictionary* box (§ 7.3). Known DLCs show their plain name (the real file name remains in a tooltip).

4.5 The status bar

To the right of the counters, “● Modified” signals an unsaved document (the “●” also appears in the window title and on the tab). Important outcomes — save, export, errors — are also shown for a few seconds at the top right (transient notifications: 4 s for a success, 7 s for a warning; an error stays until you close it); the *Transient notifications* box in the Settings turns them off. Between the message and the counters, the *Log* button opens the session log (§ 4.7): it appears only after the first operation of the session and shows, in the colour of the most serious one, the number of warnings or errors since the log was last opened — “Log (2 ⚠)”; in a narrow window it shrinks to an icon.

At the bottom: on the left, the last message (import, export, auto-translation…), cut off with an ellipsis if it is too long for the window (the full text appears in a tooltip); on the right, always visible, the counters: one **pill per status** — glyph and number of lines in the status colour, in the order of the statuses (§ 9); a dimmed pill for a status without a line —, then “Total N · M shown (U unique, R to translate)”. The pills count the whole document, whatever the filter (lines never shown in game only when the filter bar's box shows them), and are recomputed in the same frame as the operation or undo that changes them. A pill's tooltip reads “Validated: 812 lines, 7 of them shown”. A **click** on a pill shows only that status (a second click releases the filter), *Ctrl+click* adds it to the statuses already filtered: it is the same filter as the status buttons of the filter bar, which follow. When the window is narrow, the pills lose their glyph, then fold into a single one, *Statuses* ▾, whose tooltip gives the detail.

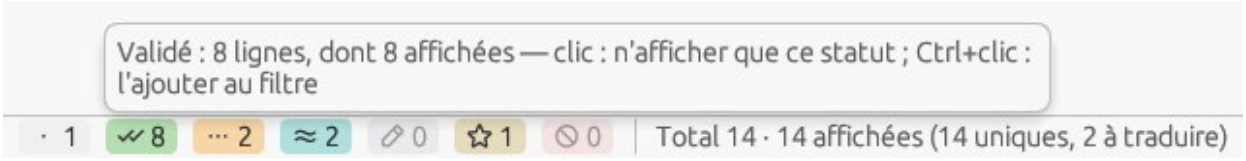


Figure 4.5 — The per-status counters of the status bar and the tooltip of a pill (French interface).

4.6 Window behavior

Secondary windows (Settings, dictionary viewer, explorers, spell checker…) are **independent**: they open next to the main window, can be moved and resized freely, and their size and position are remembered in `Config.ini`. You can operate the dialogs from the keyboard (Tab, Esc, Enter, arrow keys).

4.7 The session log

Help › Session log… (*Ctrl+Shift+J*, or the *Log* button of the status bar) opens a window that keeps, for the whole session, what the status line shows only for a moment: the start (version, configuration mode, language, folder), each opening (game, languages, lines, encoding, duration), the dictionary stack applied — **one entry per dictionary** with the number of lines it filled, then the total, the lines left and locked —, a change of target or output, the quality checks, each file written by an export, and every message, warning, error or alert. The log lives in memory (10,000 entries at most, the oldest make room) and is gone when you quit; each entry is also written to `logs/ximod.log`, which keeps its trace from one session to the next. No API key and no text sent to an online service ever goes into it.

At the top, three filters: the level (*All*, *Warnings and errors*, *Errors*), the document, and a text looked for ignoring case and accents. Each entry gives the time, the level (ⓘ, ⚠ orange, ✕ red), the document and the message, in full as a tooltip; the list follows the last entry. An entry about lines — the anomalies of a check, the lines an export refuses because the code page cannot write them — carries the *Show lines* button: it activates the document's tab and puts in the filter bar a “Log: …” chip that shows only those lines; the chip's cross or *Reset* removes it. Click, Ctrl+click and Shift+click select entries; *Copy selection* (or Ctrl+C), *Copy all* and *Save…* give the same tab-separated text, one entry per line “time ⇥ level ⇥ document ⇥ message”, saving offering `ximod-journal-YYYYMMDD-HHMM.txt` in `logs/`; *Clear* empties the list after confirmation. The command line writes the same log (§ 18.1).

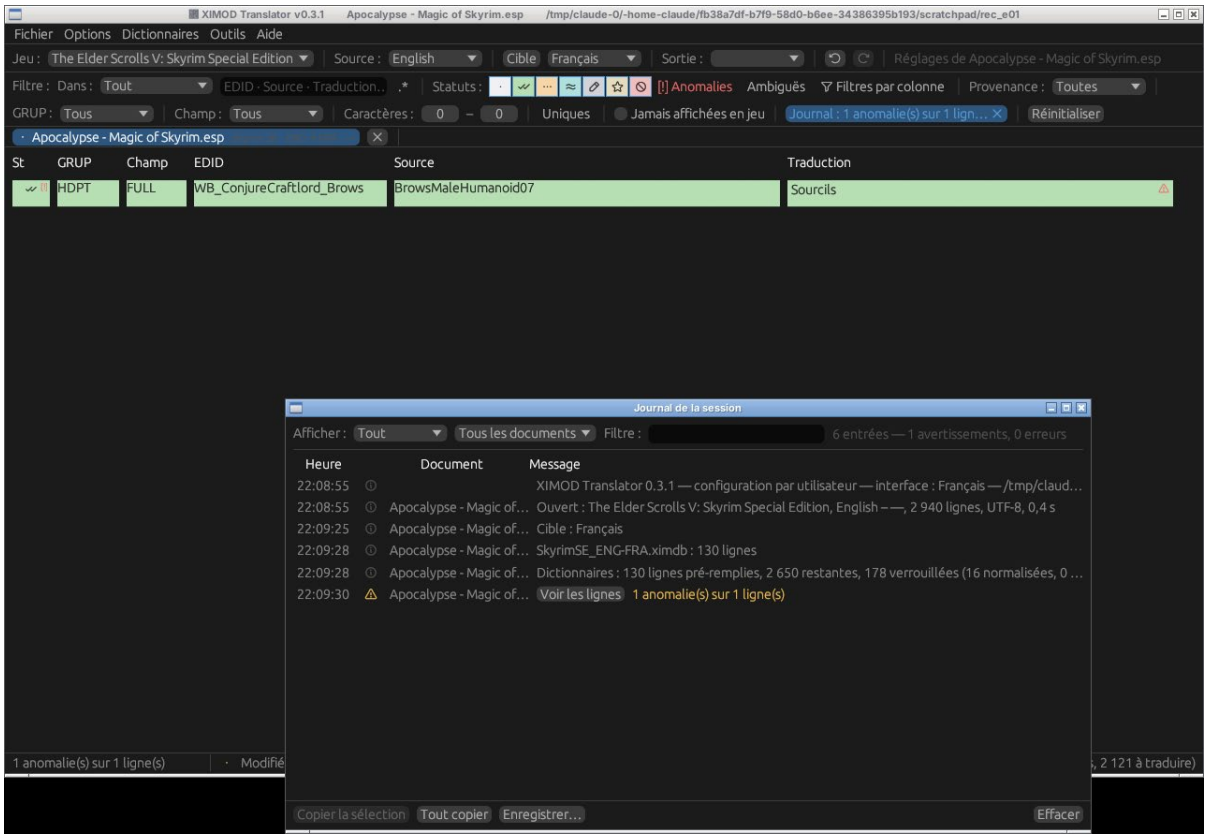


Figure 4.7 — The session log: the opening, the target, one entry per dictionary then the total, a check with its lines; at the top, the chip set by Show lines (interface in French).

5. Quick start

To translate your first mod, from start to finish:

1. Choose the *Target* language in the toolbar, by country (§ 6.4); the *Game* and the *Source* are recognized from the file when it opens.
2. Open the mod's file (*File* ▸ *Open…* or drag and drop). The game is recognized from the file; the lines already known to this mod's dictionary, to your personal dictionary and to the game database are pre-filled automatically.
3. In the grid, click in the *Translation* column and type the text, line by line (*Ctrl+Z* undoes): a line you translate becomes *Validated*. Use the filters to see only the *To translate* lines; the lines filled by the dictionaries remain to be reviewed and validated (F5): only Validated lines are exported by default.
4. Check the spelling with *Options* ▸ *Spell checker…*.
5. Choose the *Output mode* suited to the game, then *File* ▸ *Export translation…*.
6. Install the result in the game (see chapter 11 and the MO2/Vortex rules specific to each game family).

6. The toolbar: Game, Source, Target, Output

6.1 Output modes

The *Output* selector chooses what *Export translation…* produces:

Mode	What it produces
Full plugin	A standalone, fully translated .esp/.esm plugin; for a localized plugin, the strings from the STRINGS are reinjected into the plugin (delocalization).
Override patch	A patch plugin that contains only the translated records and loads after the original (it depends on it).
STRINGS	The translated .STRINGS/.DLSTRINGS/.ILSTRINGS files, to be installed next to the localized plugin. Also works from a STRINGS document on its own. On a non-localized Skyrim SE, Fallout 4 or Starfield plugin, becomes “STRINGS (localize the plugin)”: the plugin is localized and its STRINGS written (§ 19.4).
Hybrid	Like STRINGS, but from a loaded plugin (record/field context). Requires an open plugin.
Text file	The translated interface file (MCM, INI, JSON…, § 7.7), written in the target language next to the source file: the only mode offered for text documents, absent from the list for plugins.
DSD (SKSE)	A Dynamic String Distributor file (JSON, Skyrim SE/AE) injected at launch through SKSE, without redistributing the original mod.

The output names are those of the interface language (the French interface says Hybride, Correctif and Module complet). Like any label, they can be changed in the language file

(`assets/locales/<code>/main.ftl`, keys `output-mode-hybrid`, `output-mode-override`, `output-mode-full`).

The outputs follow the game. The list only offers the outputs the document's game can receive; the others stay in it, greyed and not selectable, and their tooltip says so (“Output not available for The Elder Scrolls IV: Oblivion”). With no game chosen, all are offered. Text and “scripts” documents keep their single output.

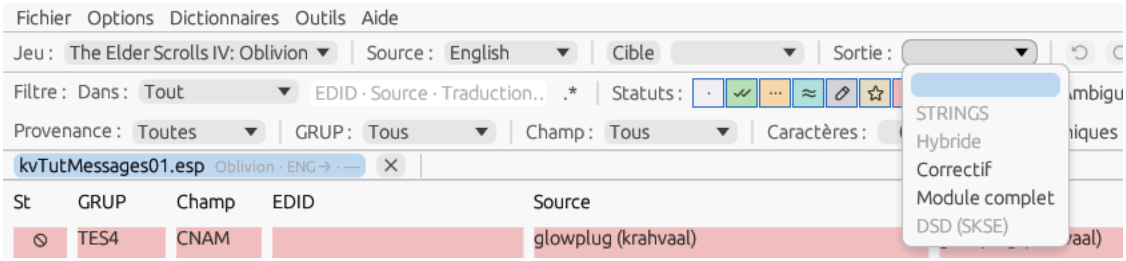


Figure 6.1 — The Output list of an Oblivion plugin: STRINGS, Hybrid and DSD are greyed.

Game	Outputs offered
Morrowind	Full plugin
Oblivion, Oblivion Remastered, Fallout 3, Fallout: New Vegas	Override patch, Full plugin (texts in the plugin, no STRINGS)
Skyrim, Fallout 4, Fallout 76, Starfield	STRINGS, Hybrid, Override patch, Full plugin
Skyrim Special Edition / Anniversary Edition	STRINGS, Hybrid, Override patch, Full plugin, DSD (SKSE)

> **Note —** An output the game cannot receive — the one of the previous session on an Oblivion file, or after a change of *Game* — empties the *Output* list; the status bar explains it and the export will ask for it (§ 6.5). The list follows the game's sheet (`assets/data/games/<Game>.json`, field `outputModes`).

Which mode for publishing? To distribute the translation of a mod you did not write, choose *Override patch* (or *STRINGS* for a localized plugin, or *DSD* on Skyrim SE): the original mod stays intact, your plugin contains only the translated records, copied as overrides, and it keeps applying when the author updates the mod. This is what authors expect from a translation, and it is the norm on Nexus. *Full plugin* rewrites the original: keep it for your own mods, or for games without STRINGS when the author accepts it. The confirmation shown at export reminds you of this when this mode is selected.

Each open document carries its own Game, Source, Target and Output settings. On opening, the game is the one recognized in the file (or the one in the *Game* list if recognition is impossible), the *Source* is deduced from the file (§ 6.3), and *Target* and *Output* take the values of the lists. Changing a list while a document is active changes **only that document**; with no document, it sets the setting of the next file. You can thus keep a Skyrim SE mod and a Fallout 4 mod open side by side, each with its own game and export mode.

6.2 Changing the Game

Changing the *Game* of a document **reloads it as that game**: the file is reread with the new game's rules (translatable fields, encoding), the translations already entered are kept by matching (FormID, then EDID, then context, then text), and the new game's dictionaries complete what is still empty. If the document has been modified, a confirmation comes first: *"Reload MonMod.esp as Fallout 3? Translations will be kept by matching."* The other documents are not affected; nothing is closed. This is useful for ambiguous cases (Fallout 3 / New Vegas, Oblivion / Oblivion Remastered) that automatic recognition cannot settle.

> **Note —** This reload clears the document's undo history, since its lines have been re-extracted.

6.3 The Source: the languages the file provides

The *Source* list offers only the languages that the open file actually provides; with no document, it is empty and inactive. For a **localized plugin**, these are the languages of the STRINGS files found next to the plugin or in its archive (a plugin can offer several). For a STRINGS file on its own or an xTranslator XML, it is the language written in its name or in its `<Source>` tag. For a **non-localized plugin** or a binary database (SST, EET), which do not state their language, the application **detects it in the text** using the Lingua library (75 languages): the list receives the plausible language or languages, with the most likely one selected. The status bar reports the language chosen when it differs from the one that was displayed.

Changing the *Source* of a document re-resolves its STRINGS in the new language and completes its lines that are still empty. Detection is an estimate: on a mod made only of short names, check the proposed choice.

6.4 The Target: by country

Target is a button: it opens the flags window (the same one as in the Settings); the cursor is in its Filter field: type the start of the country's name, Enter takes the first flag (the arrows move it). Clicking a flag closes the window and places in the list to the right of the button the **official languages of the country**, under their endonyms ("Deutsch", "Português"…). A country with a single language is selected immediately; for a country with several official languages (Belgium, Switzerland, Canada…), the list offers them and the status bar mentions them: choose one. Every language thus has its place, whether or not Bethesda publishes it (§ 19.3).

Changing the *Target* language of a document rebuilds (or retrieves from memory) the dictionary index of the game / target pair and completes its lines that are still empty **without ever touching the translations already entered**. As long as no target is chosen, the dictionaries do not apply (the status bar says so), and the commands of the *Dictionaries* menu that name a language pair ask for the target.

6.5 The Output: requested at export

The *Output* list can stay empty. *File ▸ Export translation…* is what decides: if the list is empty, an alert asks you to choose the output; if it is set, a confirmation recalls the selected output ("The selected output is 'STRINGS'. Is it the right one?") — *Yes* continues to the window of exported lines (§ 11.2), *No* returns to the document. Changing the *Output* changes only the document's export mode; the outputs offered follow its game (§ 6.1).

6.6 Undo and redo

The ↺ and ↻ buttons of the toolbar, the shortcuts *Ctrl+Z* / *Ctrl+Y* (or *Ctrl+Shift+Z*; ⌘ on macOS) and the commands of the grid's context menu undo and redo changes to the content: typing in a cell (one operation per cell, from the start to the end of the input), status change, application of a database or dictionary, automatic re-translation, propagation to duplicates, spelling correction, resumption of an automatic save. Each document has its own history; the button's tooltip names the operation and the number of lines ("Undo: Application of the base game (3,214 lines) — Ctrl+Z"). After an undo, the grid scrolls to the first affected line, and the status bar indicates how many lines are outside the current filter. The tab's "modified" dot disappears when you return exactly to the saved state. The number of operations kept (200 by default) is set in *Settings* ‣ *Advanced*.

7. Opening files

7.1 Open, drag and drop, recent files

- Menu *File* ‣ *Open…*: translation databases, plugins, and text files (MCM, INI, JSON…, § 7.7).
- **Drag and drop**: drop one or more files on the window. A `.bsa` / `.ba2` opens the archive explorer; everything else is loaded into the translator.
- Menu *File* ‣ *Recent*: reopens a recently used file (the number is adjustable in the Settings).

> **Note —** You do not need to choose a game before opening a file: a plugin opens with the game its header states, and that game goes into the *Game* list for the next files; a file that cannot state its game (text file, STRINGS alone…) opens with the game of the list, and, the list being empty, the application asks you to choose it. As long as no game is known, the *Dictionaries* menu offers only *View a dictionary…*: the game determines the dictionaries loaded and the destination folders. The other three lists never block opening. From a command line or a shortcut, `--game SkyrimSE MonMod.esp` presets the game.

7.2 Localized plugins and STRINGS

For a localized plugin (Skyrim SE, Fallout 4, Starfield…), the fields contain string identifiers rather than the text. On opening, the application automatically looks for the `.STRINGS` files of the *Source* language next to the plugin (or in a neighboring `.bsa`/`.ba2` archive) to display the original text; the languages found make up the *Source* list (§ 6.3). The encoding is detected automatically (UTF-8, otherwise Windows-1252).

A compiled script (`.pex`) opens the same way: it gives a "scripts" document of that script alone (§ 8.13).

7.3 Auto-translation: the dictionary stack

On opening, the lines already known are pre-filled from the document's active dictionaries, in the priority order of its tree (§ 12.8): the first dictionary that knows the line wins, each level filling only what the previous ones left empty. In each dictionary, an exact match (identical record, field, FormID, EDID and source text) gives the Validated status — or the remembered status for the mod dictionary —, then a match by text (same record and field, same text) gives By similarity. The initial order of the roles:

1. the **mod dictionary**, `mods/<game>/<mod name>_<SRC>-<DST>.ximdb` — your own working memory on this mod, written at each save and export (§ 7.6), or the translation of an earlier version of the same mod; a line it recognizes exactly gets back the status it had (Validated, Pending, Custom, Ignored…);
2. your **personal dictionary** (`dict/personal_<SRC>-<DST>.ximdb` in the configuration folder, § 12.10), only if the *Use my personal dictionary* box of the tab's context menu is checked (unchecked by default; the initial value can be set in `Config.ini`); this is your terminology: a pair that matches exactly by text gives the Custom status;
3. the **text-only** databases: text → text pairs with no record or field (§ 12.9), which recognize a line whose source text is identical, whatever its field, with the By similarity status;
4. the **game dictionaries** of the game / target pair, `jeux/<game>/*.ximdb` whose declared language is the target, in alphabetical order;
5. the **external dictionaries**: those of the other mods in `mods/<game>`, and the files or folders you have added to the tree;
6. the **backups** of the mod dictionary (`backups/`), unchecked at first.

The status bar names the dictionaries that filled something, in order, with their number of lines ("Dictionaries: MonMod_ENG-FRA (212), personal_ENG-FRA (3), Skyrim_ENG-FRA (1,004) — 1,219 lines pre-filled"). You therefore only type what remains. Each pre-filled line remembers its **origin**: the tooltip of its status cell says "By similarity — Skyrim_ENG-FRA.ximdb, same text", and the Record tab of the edit window repeats it; a line filled by the Approximate mode (§ 11.3) says "approximate (75 %)"; this provenance is saved with the line and covers every way a translation arrives (§ 9.1). Each file is read only once per session (and reread if it changes on disk), so switching from one tab to another rereads nothing. *Options ▸ Reload translation databases* rereads the dictionaries from disk and replays the stack on the empty lines of the active document; this application can be undone.

Normalized matching. In each dictionary, after the text-only level and before the old data, the stack tries one last level: the **normalized form** of the source — case ignored (Unicode NFKC), each number replaced by a token, sentence punctuation (". , ; : ! ? …" and dashes) removed, spaces reduced. Engine tags (`<Alias=…>`, `<mag>`…) are compared as they are, and brackets, quotes and symbols must be the same: "[BACK]" is not "Back", the translation would lose its brackets. "You have 5 days left." thus finds "You have 3 days left!" → "Il vous reste 3 jours !", adapted into "Il vous reste 5 jours.": the numbers of the new source replace those of the old one in the translation, by their value (even in another order, with the translation's decimal comma); the final punctuation is carried over (full-width after a Chinese or Japanese text); so is the case, that of the first letter, or the whole text when only one of the two sources is in capitals ("AMBER" → "AMBRE" gives "Ambre" for "Amber"). Nothing else is touched. If the translation does not contain the numbers of its source ("trois jours"), it is set as it is with the mention "numbers not carried over", and the Numbers quality check (F12) flags it.

A line filled at this level is always *Pending*, never Validated: the Provenance column says "normalized: numbers, punctuation" ("normalized" alone when nothing was carried over: the two sources only differed by spaces, inner punctuation or the case of another letter); the status cell tooltip and the Record tab give the dictionary, the source found and what was carried over ("Translated by normalized matching —

SkyrimSE_ENG-FRA.ximdb, “Carrots.”: final punctuation carried over”); the *Provenance ▸ Normalized* filter isolates them and the status bar counts them (“… — normalized lines left Pending, to check: 24”). When several sources of the same form give different adapted translations, the line is marked ambiguous (§ 11.4). A text without a letter never matches. The level also applies to the old data of the mod dictionary and to the personal dictionary, with the Pending status (not Custom: it is not the line you typed); Ignored, locked and never-shown lines are not filled. The *Normalized matching at opening (punctuation, numbers, case)* box of Settings ▸ Advanced (`[Fuzzy] Normalize`, checked by default) removes it from the opening; it then remains available in Apply the dictionaries, Normalized mode (§ 11.3).

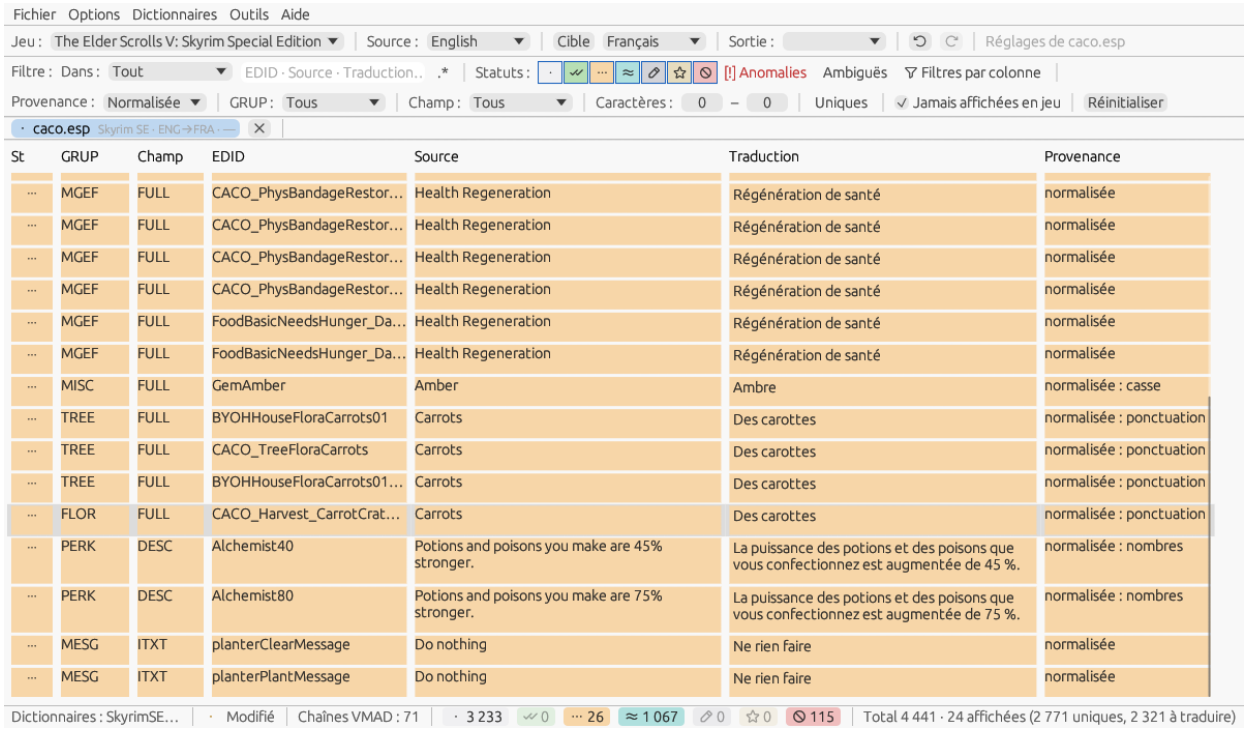


Figure 7.3 — Complete Alchemy and Cooking Overhaul opened with the Skyrim text-only base: the Provenance ▸ Normalized filter keeps the 24 lines found by their normalized form, Pending, with what was carried over (numbers, punctuation, case).

7.4 Several documents

You can open several files; each has its own tab. The grid, the filters and the export apply to the active document (the highlighted tab).

Ctrl+Tab and *Ctrl+Shift+Tab* go to the next and previous tab. A mod often comes with patches (compatibility, options) that repeat part of its texts: open them together and they are translated in one go.

Propagation between files

When an operation writes the translation of a line — typing, Use, online translation, applying the dictionaries, propagation to duplicates, accepted review — the lines with the same source (identical text, outer spaces aside) in the other open files receive that translation, as *Pending* and with the provenance “propagation (MonMod.esp)”, provided they have no translation or one “by similarity” and both files have the same source and target languages. Validated, Pending, Custom or Ignored lines are never touched,

nor lines never shown in game. It works both ways: from the mod to the patches as from a patch to the mod. The status bar says so (“Translation propagated to the lines of the same source of MonMod - Patch A.esp — lines: 2”), and the tab of the file touched gets the “modified” mark.

In each file touched, the propagation is an operation of its own (“Propagation from MonMod.esp”), undone with Ctrl+Z in that file; undoing the original typing does not undo what it propagated. The option *Propagate a translation to the open files with the same source* (Settings ▸ General, `[Editing] PropagateAcrossFiles`, checked) switches it off; the line menu then keeps *Apply to the open files* to do it on demand on the selection.

The “All mods” tab

As soon as two files are open, an *All mods (n)* tab appears right of the others: one grid with the lines of every file, in the order of the tabs, and a *File* column in second position — a column filter “Patch A” under File shows only the lines of that patch. Filters, optional columns, status buttons, line menus, the edit window (its title names the line's file) and shortcuts work there; every change is made in the line's own file: F5 on a selection covering two files validates the lines in each, one operation per file. *Ctrl+Z* in this tab undoes the last of these operations in its file, and the status bar names the file (“Undone in MonMod - Patch A.esp: Status Validated”); a second Ctrl+Z goes back to the previous file.

The toolbar there shows, read only, the settings of the current line's file; saving, export, closing, applying the dictionaries, the mod update and the dictionary tree are done in the file's own tab. Closing a file removes it from the synthesis; when only one is left, the tab disappears.

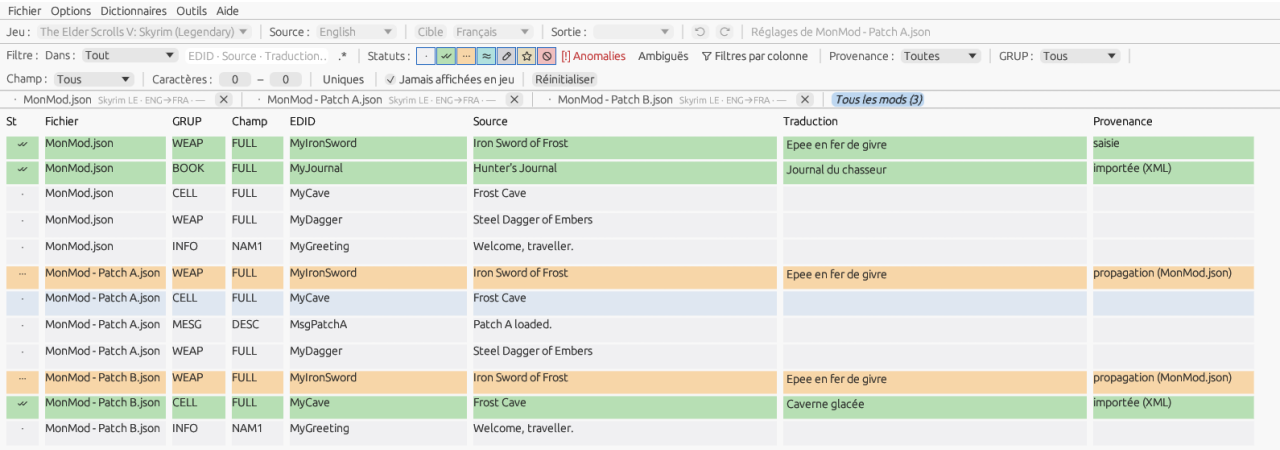


Figure 7.4 — The “All mods” tab: a mod and two patches, the File column, a translation typed in the mod and received by patch A (“propagation (MonMod.json)”), the Validated line of patch B left as it was (French interface).

7.5 Recovery after an abnormal shutdown

Every five minutes (adjustable), each document modified since its last automatic save is written in the background to the backup folder (`autosave/` in the configuration folder), without touching the original file or the “modified” state; the status bar shows it discreetly (“Automatic save 10:41”). When a tab or the application is closed normally, these files are deleted. After a crash or a power cut, the next startup offers *Resume your work?* with the list of files, the date and the number of lines: *Resume* reopens each file with its settings and its saved translations (an undoable operation; the document is marked modified); *Ignore*

keeps the backups for thirty days without offering them again. If the original file has changed in the meantime, what matches by identifier is recovered and the rest is reported.

7.6 The mod dictionary: the working memory

Each open mod has its dictionary, `mods/<game>/<mod name>_<SRC>-<DST>.ximdb` in the mod dictionaries folder. It is rewritten at each *Save as…* and at each successful *Export translation…* (option *Write the mod dictionary at each save and export* in Settings ▸ Advanced, on at first), and on request through *Dictionaries* ▸ *Save the mod dictionary*. It contains **all** the lines that are no longer "To translate" — translated, whatever their status, or ignored — with their status; lines never shown in game are not included. The status bar says so: "Mod dictionary written: 214 line(s) → MonMod_ENG-FRA.ximdb". The previous file goes into `backups/` according to the number of timestamped copies (§ 16.3).

When the mod — or a later version of it — is reopened, this dictionary is applied first (§ 7.3): a line recognized exactly (identical record, field, FormID, EDID and source text) gets back its translation **and its status**, including Pending with no text or Ignored; a line whose source text alone matches (another record) gets the translation with the By similarity status, as from any other database. You therefore find your work as you left it, with no working file to reopen. The dictionaries created through the Dictionaries menu (§ 12.4), intended for sharing, carry no status: they contain only what is validated and are applied as Validated.

Nothing is lost when a mod is updated. At each write, the lines of the previous file that the document no longer has (a removed line of dialogue, a renamed object) are not discarded: they move to the **old data** section of the same dictionary, with the current date, without duplicates; a line that returns in a later version comes back out and gets its status back. This old data is used after the current lines: a line recognized exactly gets back its translation and its status, an identical text is taken up by similarity, with the origin saying "old data"; a line whose text alone has changed is never filled on opening — the Mod update window (§ 12.11) or the FormID only mode (§ 11.3) proposes it. *Keep the old data of the mod dictionaries* (Settings ▸ Advanced, in months, 0 = forever, `[Backup] OldDataMonths`) purges it by age at write time; the editor (§ 12.10) shows it and deletes it entry by entry.

The plugin version. The mod dictionary keeps in its header the identity of the plugin it was written for: name, size and CRC-32 of the file as it was opened (likewise for a text file). When the mod dictionary is known — at opening, or as soon as the *Target* is chosen — the application compares: same size and CRC-32, the status bar adds "mod's dictionary recognised"; different, a message says the dictionary was written for another version of the plugin, with both sizes and CRC-32, the date of writing and the number of lines whose text changed, and offers *Continue with this dictionary* (normal opening: the stack applies it by record and by text), *Mod update…* (opening then the window of § 12.11) or *Open without the mod's dictionary* (it is set aside for the session; at the next write of the mod dictionary, the application asks whether to rewrite it for the open version). The message appears only once per plugin and per dictionary version. A dictionary written before this version has no identity: nothing is said, and the header is completed at the next write.



Figure 7.6 — The mod dictionary was written for another version of the plugin: continue, open the mod update, or open without the mod's dictionary (French interface).

Renamed plugin. When there is no dictionary under the plugin's name, the application looks in the same folder for a dictionary of the same language pair written for an identical file (same size, same CRC-32) and offers: “This plugin is identical to MonMod.esp; use its dictionary MonMod_ENG-FRA.ximdb (it will be renamed)?”. *Yes* renames the dictionary, its `.ximhist` history and its timestamped copies; *No* opens without. The check is switched off in *Settings* ▸ *Advanced* (*Check at opening that the mod's dictionary matches the plugin (CRC-32)*, `[Dictionaries] CheckCrc`); the header is written in all cases.

7.7 Interface files and text formats

Almost every Skyrim mod with a configuration menu (SkyUI MCM) and many Fallout 4 mods keep their interface texts outside the plugin, in `Interface/Translations/<mod>_<language>.txt`: an identifier “$Key”, a tab, the text, in UTF-16LE with a byte order mark; the game itself has `translate_<language>.txt`. Other mods use INI or JSON files. The program opens these files as full documents — grid, statuses, dictionaries, undo, edit window, checks — and writes them back in the target language while respecting the file: encoding, line endings, comments and foreign lines.

Opening. *File* ▸ *Open…* offers “Text files (MCM, INI, JSON…)”; drag and drop and *Recent* work the same way. The format is recognized from the file name (format patterns, § 15.2): a precise pattern (“Interface/Translations/*_*.txt”, “MCM/Config/*/config.json”) decides alone — the most precise one when several take the name; a .txt, .ini or .csv file that a single format by line takes by its extension opens the same way; in the other cases — several formats, none, or a JSON or XML file that no precise pattern takes — the *Import with a format* dialog opens (§ 7.8). A `.json` or `.xml` file is first read as a database (pivot, DSD, xTranslator XML) and becomes a text file only if it is not one.

The grid. The columns become *Format* (the format identifier in capitals: MCM, INI, JSON…), *Field* (always TXT) and *Key* (the key in the file, “$MonMod_Title”, with its instance number when it is repeated). A line not recognized by the format's expression — comment, empty line, JSON header — does not appear but is kept in place for writing. The tab badge shows the format and the encoding (“MCM · UTF-16 · ENG→FRA”); the status bar shows “MCM / translate file: 184 keys · 120 translated · target: MonMod_french.txt (existing, 180 keys taken)”. Keys are never translated; the “{0}”, “{}” and “%s” placeholders in the values count as tags (Aliases and numbers tab, quality checks, `tags` list in `assets/data/fields/Text.json`); there is no length limit and line breaks are allowed. The spell checker, the dictionaries, the word heuristic, instant translation and propagation apply as they do to a plugin.

Languages. An MCM file name carries its language: lowercase English names for Skyrim (`english`, `french`, `german`…), codes for Fallout 4 and Starfield (`en`, `fr`, `de`…, `ptbr`) — the table is the one used for STRINGS (§ 19). The *Source* list offers the languages of the sibling files present (MonMod_english.txt, MonMod_german.txt), and choosing another one opens that file in place of the document (if it has no unsaved changes); *Target* is chosen as usual. When the target file already exists, its values pre-fill the translations with the Validated status, origin "target file" (status tooltip, Record tab), before the dictionaries; the keys it has and the source does not are counted and ignored. A `translate_<language>.txt` file, or a format without a language in its name, takes the languages from the toolbar — or, failing that, the language detected in the text.

Writing. *File ‣ Save as…* proposes `<mod>_<target language>.txt` in the file's folder (the file itself when its name carries no language), and *Export translation…* — Output "Text file" — does the same after the exported-lines dialog (§ 11.2), where the *Untranslated lines: keep the source* checkbox (`[Export] UntranslatedCopy`, checked by default) decides what happens to lines without a translation: the source is written, or the line is omitted. The file is written in its original encoding (MCM: UTF-16LE with byte order mark) and with its original line endings, the translated values replaced in place and everything else copied as is: a comparison with the original differs only in the translated values. Save as… also keeps the database formats (.sst, .eet, .xml…) so you can keep a record of the work. The mod dictionary (§ 7.6) is written as for a plugin, under the file's name (`MonMod_english_ENG-FRA.ximdb`), and joins the plugin's stack through the External role — and conversely: the plugin's dictionary pre-fills the MCM keys whose text is known.

Link with the plugin. When you open a plugin, the program looks for `Interface/Translations/<plugin name>_<source language>.txt` in the plugin's folder (all the mod's interface files when none is in the source language); if it finds any, the status bar says so ("Interface file found: MonMod_english.txt — File ‣ Open the mod's interface files") and the *File ‣ Open the mod's interface files* entry opens it in a new tab, with the plugin's game and languages; for a plugin without an interface file, the entry is grayed out. Nothing opens automatically.

7.8 Import with a format

Files whose texts lie deep in a structure — the `config.json` of MCM Helper, an SKSE mod's JSON with arrays of objects, an XML configuration file, a CSV with the source and the translation side by side — are read with a **structured format** (§ 15.2): JSON or XML by paths, CSV by columns. *File ‣ Import with a format…* picks the file and opens the dialog; *Open…* leads there too for a text, JSON or XML file that no precise pattern takes and that no native reader (pivot, DSD, xTranslator XML) recognizes.

The dialog. *Format* lists the text formats with their type: the one chosen last for this kind of file first (`[Import] Last.<extension>`), then those with a pattern that takes the name, then by number of values recognized. *Edit…* opens the Text formats editor on the chosen format (a shipped format is duplicated), *New format…* creates there a format of the type guessed from the extension, with the file name as its pattern. The *Description of the fields* recalls what the format looks for; the *Preview summary* gives the values read (the texts of the file), the lines to translate per field, the values left out and their names, whether the identifiers are unique — "Identifiers not unique: 4, told apart by their rank" when two lines of the same field have the same one — and the target file; the preview shows the first fifty lines as the

grid will show them. Changing the format runs the analysis again; a format that recognizes nothing says so and greys out *Import*. *Import* opens the document; *Save the format* keeps among your formats a new format not yet saved.

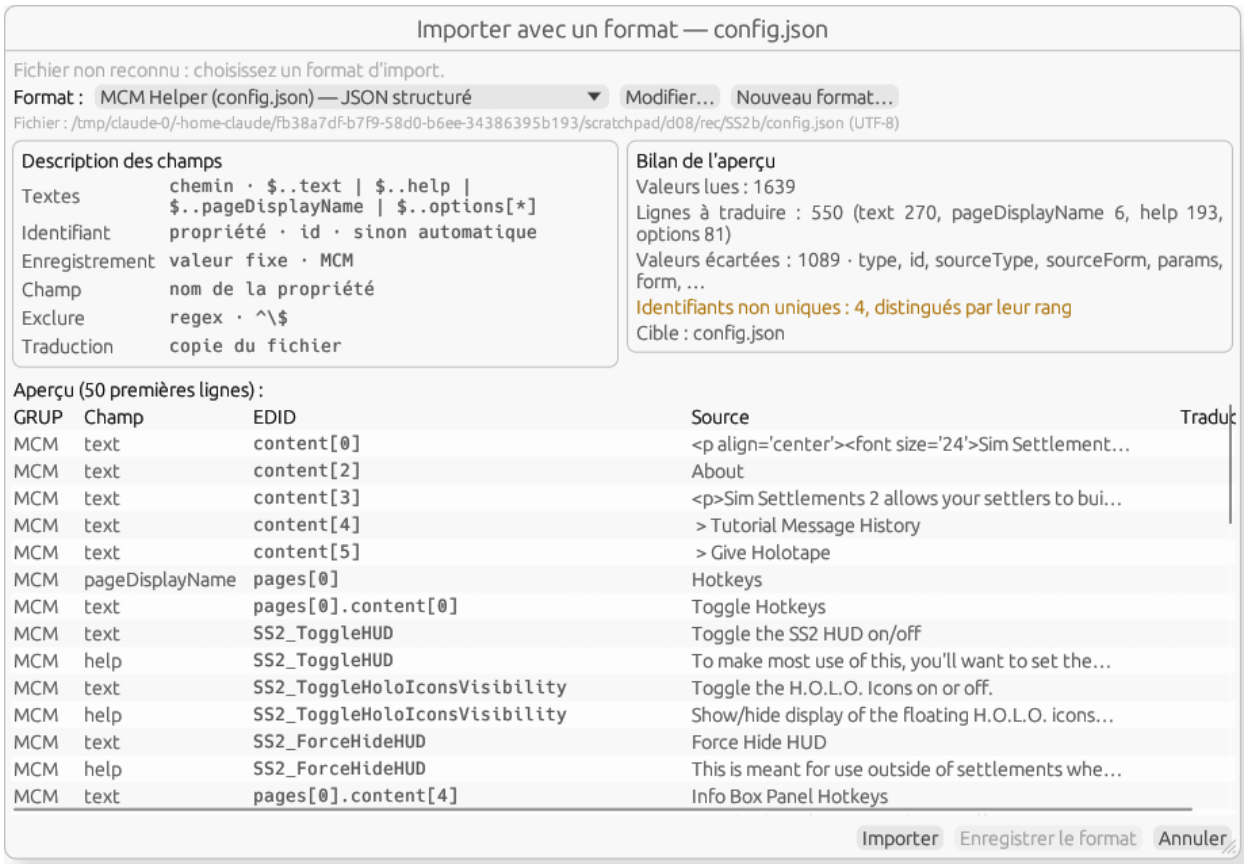


Figure 7.8 — Import with a format on the config.json of Sim Settlements 2: the MCM Helper format, its description of the fields, the summary and the preview of the lines.

The document. Each value recognized is a line: *GRUP* = the record (a fixed value, or the format identifier in capitals), *Field* = the field (the property name: text, help, options…), *EDID* = the identifier (the object's "id" property, else the path of the object — content[2], pages[1] —, followed by the rank in an array: SS2_Opt_Difficulty.0); the rank tells apart two lines with the same field and identifier. Empty values, excluded values and values outside the paths stay invisible in their place. A translation the file already carries (a sibling property, a translation column) is read with the Validated status, provenance "target file". The exclusion rules, the dictionaries and the review apply as for a plugin: *Load into review…* (§ 8.11) reads a translated copy of the file with the same format and pairs it by field, identifier and rank.

Writing. *Save as…* and *Export translation…* write `<name>_<language>.<ext>` (`config_french.json`) — or the file itself when the translation is a property or a column of the file — replacing only the translated values, in their place, escaped according to the type (JSON, XML entities, CSV quotes): indentation, key order, numbers, comments, line endings, encoding and byte order mark remain those of the original, and a comparison with it differs only in the translated values. A file modified outside the application since it was opened (size or date) makes the writing refused: open it again before writing.

7.9 Analyse a folder

File ▸ *Analyse a folder…* opens an independent window that tells, of a mod installation, what is left to translate without opening each file. *Folder* (typed or *Browse…*, remembered: `[Scan] LastDir`) and the kinds taken: plugins (esp, esm, esl), interface files and text formats recognized by a precise pattern (§ 7.7, § 7.8), compiled scripts (pex, unchecked: costly), DSD (`SKSE/Plugins/DynamicStringDistributor/…/*.json`), subfolders (`[Scan] Subdirs, Scripts`). Archives (BSA, BA2, zip, 7z) and STRINGS are not walked, but a localized plugin takes its STRINGS as at opening; a XIMOD dictionary (ximdb) is listed without analysis; beyond 500 files, the window asks for confirmation. The game and the pair are those of the toolbar, the game being taken from each plugin's header when it says so.

The analysis. *Analyse* reads the files one by one in the background (progress bar, *Stop* keeps what is done): the lines as at opening, the dictionary stack resolved as for a document (mod dictionary, personal, game and mod bases, external — each base indexed only once for the whole analysis) and applied dry, the exclusion rules laid. Nothing is opened nor written: no mod dictionary is born from an analysis. The summary totals; the table gives one line per file: path under the folder, game, lines, translated by the stack (and the percentage; the tooltip details per dictionary), left, locked, mod dictionary (recognized, other version, absent, or "dictionary" for a translation) and state: *translated* (nothing left), *almost done* (less than 5%), *to translate*, *update* (dictionary of another version and texts changed), *dictionary source* (a DSD, a plugin whose language is the target), *unreadable*.

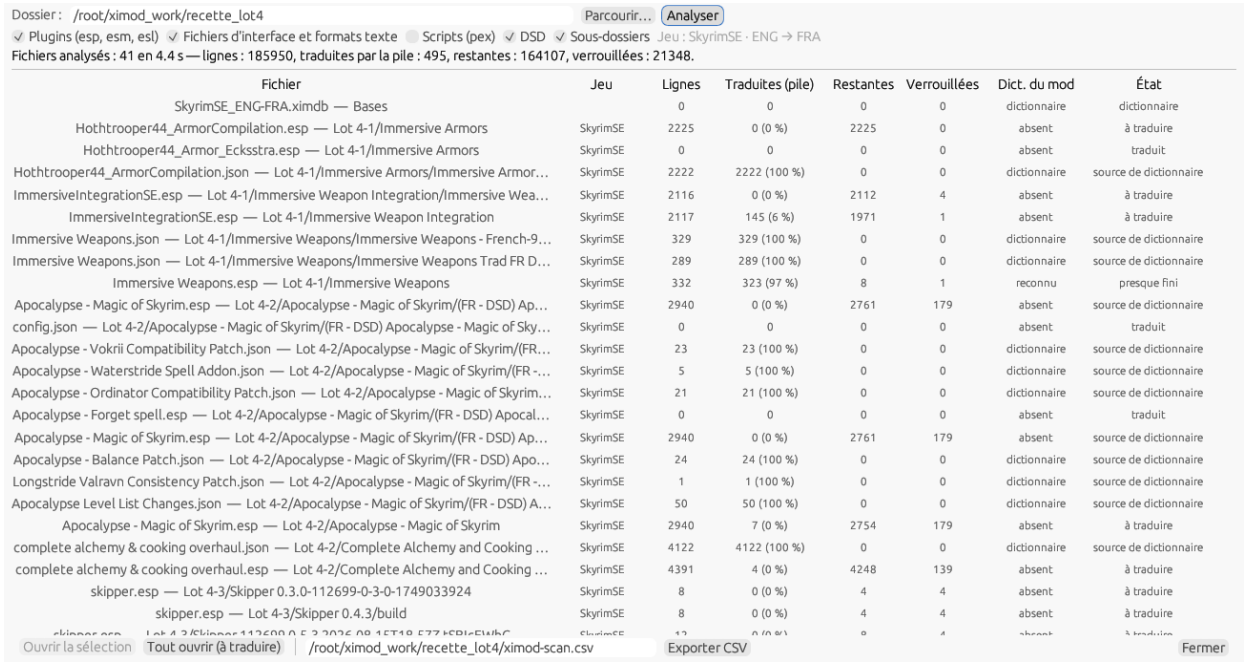


Figure 7.9 — Analyse a folder on the recette mods: one file per line, what the stack would make of it, the mod dictionary and the state.

Then. A double click opens the file in a tab with its game and the pair (hence its stack); *Open the selection* (click, Ctrl+click, Shift+click) and *Open all (to translate)* open several plugins at once. *Export CSV* writes the file whose path is next to the button (UTF-8 with BOM, semicolon; columns `file;game;lines;translated;left;locked;dictionary;state`, dictionary and state as stable codes:

known, other, none, is; done, almost, todo, update, source, error). The window remembers its geometry (`[Window] ScanX/Y/W/H`).

8. The translation grid

8.1 The columns

The grid shows one line per string, with six fixed columns, to which the *Options* ▸ *Columns* sub-menu adds optional columns, all hidden at first — the same menu opens with a right-click on the grid header; the choices are remembered in `[Columns]`:

Column	Content
Status	Colored dot + glyph showing the state of the line (see chapter 9).
Status buttons	Optional (Options ▸ Columns ▸ Status buttons), right of St. Four small buttons in the colour of their status — ✓ Validated, ⋯ Pending, ★ Custom, ⊘ Ignored — that give the status to the line or to the selection in one click (§ 8.2).
GRUP	Signature of the record (e.g. MESG, BOOK, INFO).
Field	Translated subfield (e.g. FULL, DESC); VMAD for a string of a script attached to the record (§ 8.13).
EDID	Editor ID of the record, when the format provides it.
Source	Original text (Source language), on several lines if needed.
Translation	Input area for the translation (Target language).
Review	Optional (Options ▸ Columns ▸ Review), shown automatically when a review is loaded. The text proposed by a reviewer, editable like the translation; in blue when it differs from the translation, in grey when it is the same (§ 8.11).
Provenance	Optional (Options ▸ Columns ▸ Provenance). Where the translation came from: “dict. game (exact)”, “online (DeepL)”, “typed”, “identical”…; the full label in the tooltip (§ 9.1).
Comment	Optional (Options ▸ Columns ▸ Comment). The author's comment, when the file carries one (EET, XML), then the generated context of the line: speaker and gender, quest, topic, emotion, response rank for a reply; model of an object; journal entry of a quest; master file of an override.

The Comment column. *Options* ▸ *Columns* ▸ *Comment* (`[Columns] ShowComment`, unchecked by default; the former `[Editing] ShowComment` key is taken over) adds the last column, after Translation and Provenance. Its text is generated on display from the dialogue map (§ 8.8) and from the subrecords read by the context pass, cached per line as long as the document and its map do not change, and costs nothing for lines off screen: a 50,000-line file stays smooth. The text appears once the context pass has finished (the column starts it on first request). Its content depends on the line type, unknown elements being omitted: a **reply** (INFO) gives “Ralof (m) · MQ101 · MQ101RalofTopic01 · Anger 60 · response 1 / 2” —

speaker and gender, quest, topic (or category / subtype), emotion and intensity, response rank —, then "voice ✓" or "voice ✗" once the voice map is built (§ 8.9); a **topic** (DIAL) "MQ101 · MQ101RalofTopic01 · 3 replies"; a **quest** "quest MQ101 · journal entry, stage 20" for a journal text (CNAM) or "quest MQ101 · objective 10" for an objective (NNAM); a message, perk or spell tied to a quest "quest MQ101"; any **object with a model** (weapon, armor, book, container, plant, light, door…) "model: Weapons\Iron\IronWarAxe.nif"; an **NPC** "male · NordRace · voice MaleNord"; and any **override** of a record from a master file ends with "master: Skyrim.esm". The author's comment, carried by EET and XML files since batch 1, comes first, separated by "·"; it cannot be edited here and is kept as is when saving. A text longer than 200 characters is truncated, and the tooltip shows it in full; the column width follows the window width.

8.2 Entering a translation

Click anywhere in the *Translation* cell — even at the bottom of a tall line — and type the text; the line's status changes when you leave the cell: a line without translation becomes Validated, an automatic translation edited (By similarity, Pending) becomes Modified proposal — to be reviewed —, Validated and Custom stay as they are, and erasing the translation brings the line back to To translate. The *Source* and *Translation* columns wrap long texts, so the input column is never pushed off screen. The grid is virtualized: even a file of several thousand lines stays smooth.

The line menu. A right-click anywhere on a line — dot, identifiers, source or translation — opens a single menu; if the line is not in the selection, it becomes the selection, otherwise the menu acts on the whole selection and says so ("Status of the selection (3 lines)", "Paste into the 3 selected lines"). Five groups: *Edit* — Edit… (Enter: opens the edit window, § 8.5), Copy the source, Copy the translation (Ctrl+C), Paste into the cell or into the selected lines (Ctrl+V), Translation = source (Ctrl+F8: copies the source into the translation without changing the status, skipping Ignored lines); *Status* — Validated, Pending, Custom, Ignored (F5 to F8) and Validate then next (Ctrl+↓); *Dictionaries* — Apply the dictionaries to the selection… (§ 11.3), Add to my personal dictionary (Ctrl+M, § 12.10) and Apply to every line with the same source… (Ctrl+F6, § 8.4), then Concordance… (Ctrl+K, § 10.3); *Navigation* — Filter on this record (all lines with the same FormID) and Filter on this source (all lines with the same source text); these filters appear as chips in the filter bar, and clicking a chip removes it; *Line tools* — Book preview (§ 8.3); then Undo / Redo. An entry that does not apply is grayed out and its tooltip says why (empty clipboard, not a book); the Review (§ 8.11) and History (§ 8.12) groups complete the menu. Any menu action that modifies the document can be undone in one step, whatever the number of lines. The menu shortcuts Ctrl+C / Ctrl+V / Ctrl+F8 / Enter act only when no cell is being edited: inside a cell, they keep their usual meaning.

Selection and statuses from the keyboard. Clicking a line selects it (bluish tint); Shift+click extends the selection, Ctrl+click adds or removes a line; clicking in the *Translation* cell also selects the line. The function keys give a status to the whole selection: *F5* Validated, *F6* Pending, *F7* Custom, *F8* Ignored. A line without a translation cannot become Validated or Custom: it is skipped and the status bar says so ("3 lines → Validated; 1 without translation skipped"). *Ctrl+*↓ validates the line (or the selection) then moves to the next line in the view, *Ctrl+*↑ to the previous one; if you were typing, the input cursor follows: type, Ctrl+↓, type… translates a file without touching the mouse. At the end of the view, the status bar says "Last line of the view.". Each key press is an undoable operation ("Undo: Status Validated (3 lines)"), and a line

whose status has just changed stays displayed even if the status filter now excludes it, until the next change of filter or selection.

The status buttons. For whoever reviews a list with the mouse, *Options* ▸ *Columns* ▸ *Status buttons* (`[Columns] ShowStatusButtons`, unchecked at first) adds, right of the St column, four small buttons in the colour of their status: ✓ Validated, ⋯ Pending, ★ Custom, ⊘ Ignored — the glyphs of the St column and of the line menu. A click gives the status to the line; when the line is part of a selection of several lines, to the whole selection, exactly like F5 to F8: one undoable operation, the count in the status bar, and the propagation proposal to the duplicates after ✓ when that option is on (§ 8.4). Clicking a button of a line that is not selected selects that line alone, then applies the status; the click neither opens the edit window nor starts typing. The button of the current status is framed; an impossible status — ✓ and ★ on a line without translation — is greyed and its tooltip says why ("A line without translation cannot be Validated."); lines never shown in game have no buttons. The tooltips recall the shortcut: "Validate — F5", "Set pending — F6", "Set custom — F7", "Ignore — F8". The column takes its width from the text columns and gives it back when hidden.

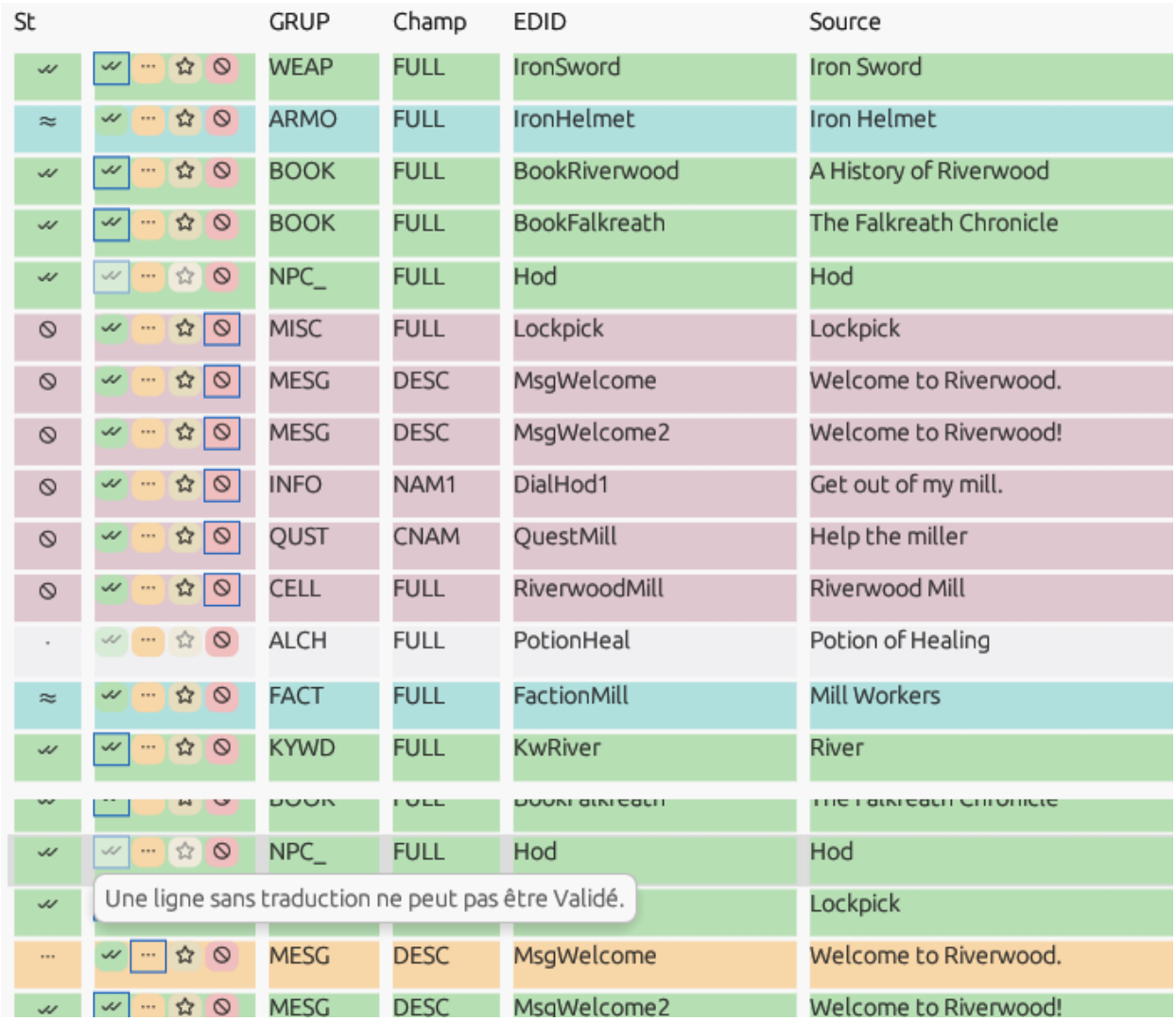


Figure 8.2 — The status buttons column: six selected lines set to Ignored with one click on ⊘; below, the tooltip of a greyed button (French interface).

Length and line breaks. As long as the translation respects the limits of its field, the cell shows nothing. When a limit is exceeded, the cell shows the count in red at its right edge (“34 / 32”: characters against a character limit, or “37 / 32 o” against a byte limit); a line break in a field declared single-line is flagged by *[!]* in orange; an engine tag or a number that differs between the source and the translation is flagged by ⚠ (red: missing or unknown tag or number; orange: same tags in a different order, a mere warning). The cell's tooltip gives the details (field, limit, bytes, tags involved); the Aliases and numbers tab of the edit window (§ 8.5) offers the repairs. The limits and whether line breaks are allowed come from the game's field table (§ 15.1); a field with no known limit is never flagged. Exceeding a limit prevents neither input nor validation: you stay in control.

8.3 Book preview

For book records, a 📖 *Book preview* button opens a window showing the formatted *Source* and *Translation* side by side, with the *Game fonts* option for a faithful rendering.

8.4 “Unique lines” view

The *Unique* filter shows only one line per distinct source. Translating this unique line **automatically propagates** the translation to all its hidden duplicates — handy when the same text recurs many times.

Explicit propagation. From the normal view, *Ctrl+F6* (line menu ▸ Apply to every line with the same source…, in the grid and in the edit window) on a translated line opens a dialog that lists the document's other lines with the same source text — excluding Ignored lines and lines never displayed — with record, field, EDID, current translation and status. All are checked, except Validated and Custom lines whose translation differs; Check all / Uncheck all help. *Apply to the N checked lines* (Enter) copies the translation **and the status** of the original line onto the checked lines, in one undoable operation (“Undo: Propagation to duplicates (19 lines)”); the status bar gives the count. When there is no duplicate, the status bar says so (“No other line has this source.”). On a selection of several lines, each translated line forms a group, each source once. The source text is compared without its leading and trailing spaces. When other files with the same languages are open (§ 7.4), their lines with the same source join the list, marked with their name in a File column, with the same checking rule; Apply writes one operation in each file it touches, which Ctrl+Z undoes file by file, and the status bar says so; the automatic propagation between files (§ 7.4) does not go over what the dialog decided. The *Propose the propagation to the lines of the same source after a validation* option (Settings ▸ General, `[Editing] ProposePropagation`, unchecked by default) opens the same dialog after F5 or Ctrl+↓ on a single line that has differing duplicates; Esc closes it without doing anything and does not keep you from moving on.

8.5 The edit window

For long texts, or to translate a file in one go without leaving the keyboard, a separate edit window accompanies the grid. *Enter* on the selected line, a double-click on a line (outside its translation cell) or *Edit…* in the line menu opens it on that line — or brings it to the front if it is already open; the cursor is

placed at the end of the translation. There is only one, tied to the active document: switching document tabs switches the window's document, and *File* ▸ *Close* closes it along with the document.



Figure 8.5 — The edit window: line identity, source, translation with its counter, Record tab, navigation and statuses.

From top to bottom: the line identity (record and field, EDID, FormID, instance number, originating plugin for a DSD); the *Source*, read-only but selectable and copyable; the *Translation*, multiline, where your input is carried over to the grid live — the carry-over becomes an undoable operation (§ 6.6) when the line, the status or the focus changes, and *Ctrl+Z* in the window undoes the last one; below the area, the counter: “146 / 512 chars · 151 bytes · line break: allowed” when the field limit is known, “146 chars · 151 bytes” otherwise — orange from 90% of the limit, red beyond it, *[!]* on a forbidden line break; bytes are counted in the game's encoding (UTF-8 for Skyrim SE, Fallout 4, Fallout 76 and Starfield; Windows-1252 or the code page of the target language for the older games, where one character is one byte, § 19.5); the context tabs; finally *Previous* / *Next* and the four status buttons, the current status shown raised.

From the keyboard. *Ctrl+Enter* validates the line then moves to the next line in the filtered view: type, Ctrl+Enter, type…; *Alt+Right* / *Alt+Left* change line without touching the status; *F5* to *F8* set the status of the displayed line; *Esc* closes the window and keeps your input. The window and the grid stay in step: the displayed line is the selected line, the grid scrolls to show it, and clicking another line in the grid changes the window's line. At the end of the view, the window stays on the last line and says so. *Enter* alone inserts a line break when the game's field table (§ 15.1) allows it, or as long as the table says nothing; in a field declared single-line, nothing is inserted and the window reports it.

The *Record* tab first shows, below the origin of the translation, the line's comment (“Comment: Ralof (m) · MQ101 · …”, § 8.1 — the author's comment, then the generated text, the latter as soon as the dialogue map is built), then lists the other translatable fields of the same record (same FormID), with their

status, source and translation; clicking a field name goes to it. The *Aliases and numbers* tab compares the engine tags and numbers of the source with those of the translation: "`<Alias=Victim>` missing from the translation", "`<Alias=Victime>` missing from the source", "Number 3 missing from the translation", "Tag order differs from the source" (a warning: word order changes from one language to another), or "consistent". *Restore aliases* replaces the i-th tag of the translation with the i-th tag of the source, adds the missing ones at the end of the text when the translation has fewer, and says so; *Restore numbers* does the same for numbers; each repair is an undoable operation. The recognized tags — `<Alias…>`, `<Global…>`, `<Token…>`, `<BaseName>`, `<Pronoun…>`, book formatting tags — are the `tags` key of the game's field file (§ 15.1), compared case-insensitively; an empty translation is never an anomaly. The *Dialogue* tab (§ 8.8), on an INFO reply, summarizes its context — speaker, gender and voice, quest, topic, response rank, emotion — then gives the player's question, the reply before and the reply after, the conditions in plain language, the voice and the author's note; *Previous* / *Next* (Ctrl+Alt+Left / Ctrl+Alt+Right) change the window's line by following the conversation, and *Dialogues…* opens the Dialogues window; on a topic or a quest, the tab gives its label and counters; on any other line, "This line does not belong to a dialogue.". The *Vocabulary* tab offers, a tenth of a second after you arrive on the line, the sources close to the line in the active dictionaries (§ 12.8), with their translation: a **word heuristic** compares the words of the two texts (ignoring case and punctuation; an engine tag counts as a "tag" word, a number as a "number" word) and keeps the sources whose number of differing words — missing, extra or replaced — does not exceed the *Word heuristic threshold* (Settings ▸ Advanced, 20% by default, from 5 to 50; `[Fuzzy] Threshold`). Each line shows the score (green at 90% and above, yellow from 70 to 89%, orange below), the source with its differing words underlined, the translation, the dictionary and the original field ("(old)" for old data, § 7.6), and a *Use* button that copies the translation into the line with the Pending status — to adapt, then validate — in one undoable operation. Eight suggestions at most, sorted by score, then by stack order; words that are too frequent in a dictionary (present in more than 5% of its lines) are not used to preselect candidates. A dictionary's word index is built in the background on first request ("Word index in progress (42 %)…"), a few seconds for a hundred thousand lines, and lasts for the session. Below the suggestions, the **Known fragments** line lists the parts of the source that the dictionaries know ("Satchel → Sacoche (Skyrim_ENG-FRA) · of Moon Sugar → de sucrelune (personal_ENG-FRA)"): this is what instant translation will produce. Between the suggestions and the fragments, the **Character-by-character differences** frame compares the source of the line with that of a suggestion — the first one when you reach the line, then the one you click (its row is framed) — or, after a click on a candidate translation (§ 11.4), the translation of the line with that candidate: in green what the line has in addition, in red what the entry has in addition, identical runs plain; inside a differing run a space reads "·", a line break "↵", a tabulation "→". ⇄ *Swap* exchanges the two lines, the colours and the legend, to read the difference from the entry. The comparison works on characters (Unicode graphemes, Myers' algorithm from the similar library) and is then cleaned up: a common run shorter than two characters between two differences is absorbed, and a difference inside a word or a number grows to the whole word or number — "12 gold" against "120 gold" highlights "12" and "120", and two lines that differ only by a number and a punctuation mark highlight those alone. Beyond 5,000 characters, or when the comparison takes more than 50 ms, it is done word by word and the frame says so. The frame changes nothing; Use remains the only gesture that takes a translation. A right-click in the Translation area opens the Edit and Status groups of the line

menu (§ 8.2). The window geometry is remembered (`[Window] EditX/EditY/EditW/EditH`, 760 × 520 by default, opened offset near the main window), as is the active tab (`[Editing] EditTab`).

The *Common (n)* tab, between Vocabulary and Dialogue, lists the other lines that have the same source as the line — same text, without its leading and trailing spaces; n is their number in the files. First the lines of the file, in its order, then those of the other open files with the same source and target languages (§ 7.4), in the order of the tabs — in the "All mods" tab, the lines of the other files come after those of the line's own file; lines never displayed in game are left out. Each line gives its status (swatch, the name as a tooltip), its file, its record and field, its EDID, its translation and its provenance (§ 7.3): a translation that differs from the line's is in blue, an identical one in grey. Further down, folded, the *Dictionaries (n)* section gives the distinct translations of this source in the active dictionaries, with the dictionaries and the number of lines, like the candidate translations of the Vocabulary tab (§ 11.4). A click on a line (or a dictionary translation) selects it and, when its translation differs, shows below the **character-by-character differences** between the line's translation and its own. *Go to the line* — or a double-click — makes the tab of its file active, selects the line there and puts the edit window on it. *Apply to all (Ctrl+F6)* opens the propagation dialog (§ 8.4) with the lines of the same source in the file and in the other open files; greyed out when the line has no translation. The summary below the list counts "2 here, 1 in other files · 1 in the dictionaries"; beyond 200 lines of files (20 dictionary translations), the header says how many are shown. With no other line: "No other line has this source." and "Common (0)". The list is recomputed when you arrive on a line and after each operation, from an index of the sources of each file built on first use: a few milliseconds for 150,000 lines. The three context tabs of a dialogue line complement each other: Dialogue for the linked lines (player prompt, previous and next lines), Common for the lines with the same source, Vocabulary for the dictionary suggestions.

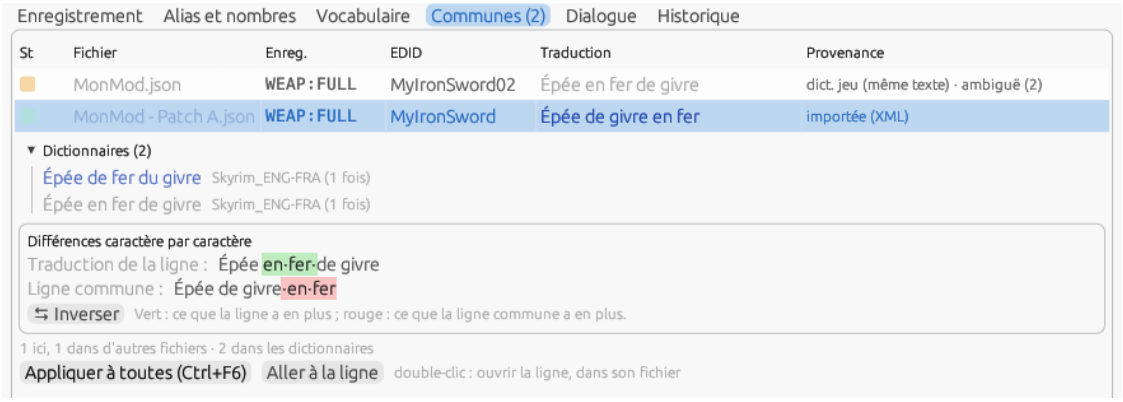


Figure 8.6 — The Common tab: the line of the file with the same source, that of an open patch with its different translation in blue, the Dictionaries section, and the character-by-character differences of the clicked line.

Instant translation. You can select text in the Source area with the keyboard and the mouse (it remains read-only), and its selection is kept when the focus moves to the Translation area. *F9* translates the source selection — or the whole source when nothing is selected — by assembling the **fragments** known to the active dictionaries (§ 12.8), text-only and personal dictionaries first: the whole text first, otherwise, word after word, the longest known fragment starting there (case-insensitive); a word that no fragment covers is copied as is and counted as unknown; engine tags, numbers and punctuation are copied. With a selection, the result is inserted at the cursor of the Translation area (or replaces its selection); without a

selection, it replaces the translation, after confirmation if the translation is not empty (“Do not ask again during this session” checkbox). The initial capital follows the destination: at the start of the translation, it follows the source; in the middle of a sentence, a fragment taken from the start of the source loses its capital (“Une” + “Steel Sword” → “Une épée en acier”). The line is set to Pending, the operation is undone like typing, and the message line gives the summary: “2 fragment(s) translated, 1 unknown word(s) (open)”. *Ctrl+F9* translates in two parts: the cursor in the Source area splits the text (a cursor inside a word splits after that word), each part is translated and the two results are joined by a space; without a cursor, Ctrl+F9 is the same as F9. In the grid, F9 does nothing: line menu ▸ Apply the dictionaries to the selection… (§ 11.3) remains the grid's entry point.

Concordance. *Ctrl+K* (or the context menu of the Source or Translation area ▸ Concordance…) looks for the selected text, or the word under the cursor, in the active dictionaries; *Into the line* takes the chosen translation in place of the selection or at the cursor of the Translation area (§ 10.3).

Glossary. When every word of the source is covered by the glossary of the language pair (§ 15.7), *F9* without a selection gives the glossary's translation, composed (“Tribunal Heavy Boots White” → “Solerets blancs du Tribunal”), with the provenance “rule: composition” or “rule: glossary”; otherwise a pattern with variables (§ 15.7) when the source is short (“Iron Sword of Burning” → “Épée en fer de feu”, provenance “rule: pattern 2”); otherwise the glossary is the first level of fragments, before the personal dictionary. The *Vocabulary* tab shows the glossary terms that cover the line, with their group: “Glossary: Heavy Boots → Solerets (Armures) · White → blanc / blanche / blancs / blanches (Couleurs)”, and the pattern that would translate the line: “Pattern 2: %A% of %B% → Masse de Molag Bal”. *Ctrl+Alt+G* — or the context menu of the Source area ▸ Add to the glossary — takes the text selected in the source and in the translation (the whole texts without a selection) and opens a small dialog: group (an existing one, or a new group you name), original, translated, class, gender, whole word, case, and the four forms of an adjective or a colour. *OK* writes the term into the user copy of the pair's glossary, without opening the window; F9 and the try field know it at once, and the chosen group is offered again for the next term.



Figure 8.5 b — Add to the glossary from the edit window, on "Einherjar Brigandine Dark Boots": the adjective "Brigandine" and its four forms, in the group Modificateurs; F9 then gives "Bottes brigandines sombres d'Einherjar".

8.6 Find and replace

Tools ▸ *Find and replace…* (*Ctrl+H*) opens a separate window, only one, tied to the active document, pre-filled with the selection of the cell being edited or, failing that, the filter text. Two fields, *Find* and *Replace*, each with the ▾ button that drops down the last ten patterns of the session. *In* chooses the target — Translation, Source, EDID or FormID, Signature — but only the translation can be replaced in a plugin: on another target, the Replace buttons are grayed out and say so, while searching remains possible. *Scope*: the whole document, the filtered view or the selection (with the number of lines). Options: *Match case*, *Whole word*; modes: *Normal*, *Extended* (`\n`, `\r`, `\t`, `\\` interpreted in the pattern and the replacement) and *Regex* (Rust syntax, groups `$1`…`$9` and `${nom}` in the replacement: "(\w+) of (\w+)" → "$2 de $1").

The preview — "37 occurrences in 29 lines — first: '…' → '…'" — is recomputed a quarter of a second after you type; an invalid expression is shown in red, with the buttons grayed out. *Next* (F3) and *Previous* (Shift+F3), from the window or from the grid, select the next or previous occurrence in the grid, which scrolls; the edit window follows it if it is open. *Replace* replaces in the current line then moves to the next one; *Replace all* processes the whole scope in a single undoable operation ("Undo: Replacement (29 lines)"). Ignored lines are never modified; a modified Validated line is set to Pending, for review, and the status bar says so: "29 lines replaced, 12 set Pending".

Matrices. At the top of the window, *Matrix* offers the replacement matrices of the document's target language (§ 15.6), then the matrices common to all languages, suffixed "(all languages)"; *(none — single rule)* goes back to replacing one pattern. *Direction*: *Direct* applies the rules in order; *Inverse* undoes them, from the last to the first — greyed out, with the rule at fault in the tooltip, when a regex rule has no inverse. The preview gives the number of rules, of lines touched and the first transformation; *Apply the*

matrix processes the scope in a single undoable operation (“Undo: Matrix Typographie française”). Only translated lines, neither ignored nor never shown in game, are touched; they **keep their status** — a matrix fixes the form, not the meaning — and take the provenance “replacement (Typographie française)”, which the *Provenance* ▸ *Replacement* filter isolates. The chosen matrix is remembered per language (`[Replace] LastMatrix`). It also applies in the All mods tab, file by file.

Add to a matrix…, below the single rule, appends the current pattern (with its mode, case and whole word) to a matrix of the target language, or to a new matrix named in the menu's field then *Create*; the matrices file is saved at once. The former “Apply a rules file…” and “Save as rule…” buttons are gone: an existing rules file is imported as a matrix (§ 15.6). The window remembers its geometry (`[Window] ReplaceX/Y/W/H`).



Figure 8.6 b — Find and replace with the French matrix Apostrophes typographiques: 32 lines touched, the direction, then the single rule.

Special characters. *Options* ▸ *Show special characters* — a checkbox also found in the window — makes visible, in the grid and in the edit window, the line feed “¶”, the carriage return “↵”, the tab “→” and the non-breaking space “°”, in a dimmed color. The text is not modified and input ignores these glyphs: the saved file does contain the real characters. The setting is remembered (`[Editing] ShowSpecialChars`).

8.7 Quality checks

Tools ▸ *Quality checks…* opens a separate window, only one, tied to the active document; *F12* opens it and starts the check right away, and a second F12 runs it again. Nine checks, each with its own checkbox, whose state is remembered (`[Quality] Checks`): *Aliases* (engine tags of the source missing from the translation, unknown or in a different order), *Numbers* (numbers of the source missing or unknown), *Field maximum length* (the character or byte limit of the field table is exceeded, § 15.1), *Forbidden line break*

(line break in a single-line field), *Different final punctuation* (the source ends with ".", "!", "?", "…" or ":" and the translation with something else, or the reverse), *Capital after a period* (lowercase letter after ".", "!" or "?" — unchecked by default, as proper nouns and abbreviations trigger it too often), *Double or edge spaces*, *Unencodable character* (characters the document's writing code page cannot represent, § 19.5: the list of offending characters is given, with the page) and *Different translations of one source* (the same source translated in several ways: the variants with their number of lines). Ignored lines, lines never displayed in game and lines without a translation are not checked.

Scope: the whole document, the filtered view or the selection. *Check* (F12) works in the background with a progress bar — about one second for 50,000 lines and all checks — and the interface remains usable. The result, "37 anomaly(ies) on 29 line(s)", heads a table with the columns *Line*, *EDID / source*, *Check*, *Detail*, sorted by clicking a header (an arrow shows the direction). A double-click or Enter opens the edit window on the line; Ctrl+click and Shift+click select several lines; *Fix aliases* and *Fix numbers* apply the repair of the Aliases and numbers tab (§ 8.5) to the selected lines, in one undoable operation. An "Inconsistency" line has the *Unify on the most frequent* button: all lines of the group, Ignored excluded, receive the majority variant, in a single undoable operation (a Validated line goes back to Pending). The window remembers its geometry (`[Window] QualityX/Y/W/H`, 700 × 520 by default).

In the grid, each flagged line shows *[!]* in red in the status cell, with a tooltip listing its anomalies; the *[!] Anomalies* button of the filter bar (§ 4.3) shows only those lines. The indicator follows your corrections: validating an input, repairing, replacing or unifying recomputes the checks of the line (and the window's table) without rerunning the whole set — only the inconsistency check, which compares lines with each other, waits for the next F12. The anomalies belong to the checked document: switching tabs clears them, F12 recomputes them.

8.8 Dialogues

A dialogue is scattered across the plugin: a quest (QUST) owns topics (DIAL), each topic contains replies (INFO), and each reply carries one or more responses (NAM1 text, emotion, author's note), the player's paraphrase (RNAM), a speaker, conditions and links to the neighboring replies; scenes (SCEN) chain topics by phases, with actors designated by alias. The grid shows these lines in file order. *Tools ▸ Dialogues…* (*F10*) opens a separate window, only one, tied to the active document, which rebuilds the tree and lets you follow a conversation reply by reply. It lives with the document: switching tabs makes it follow, and a document without dialogue opens it on "No dialogue in this document.".

The context pass. On first request — Dialogues window, Dialogue tab of the edit window, NPC filter — the program re-reads the plugin in the background and derives the dialogue map from it: quests and aliases, topics, replies, scenes, NPCs (name, gender, voice type), voice types, factions, keywords and global values named by the conditions. About one second for 50,000 lines, with progress shown in the window. Forms from master files (an NPC from Skyrim.esm, another mod's quest) are named when the master is found — in the plugin's folder, then in the Data folder of the game detected or set in Settings ▸ Paths: an index of its names is built once, in the background, and kept in `cache/names/` in the configuration folder (key: file size and date, source language); without the master, the form is shown as "000A2C8E (Skyrim.esm)".

The tree. First level: the plugin's quests (EDID — name), then “No quest” for topics that have none. Under a quest: the scenes and their phases (“Phase 1 — Ralof”), each phase leading to the replies of the action played, then the topics by category and subtype (“Topic — Custom”, “Misc — Hello”, “Combat — Attack”…), each topic with its EDID and its text (“MQ101RalofTopic01 ‘What's going on?’”); a reply shows its speaker, the start of its first response and the emotion; a reply with several responses can be expanded. Nodes carry the status dot of their lines (gray when the responses do not all have the same status) and the “translated / total” counters. The labels of categories, subtypes and emotions come from the `assets/data/dialogue/<family>.json` file (skyrim, fallout4, fallout3, oblivion, starfield, morrowind).

The reply. The right-hand pane shows the chosen reply: FormID, topic, rank in the topic; the **speaker** with gender, voice type and how it was resolved — *ANAM* when the reply names its NPC, *scene* when it is a scene actor (the quest alias, resolved through its unique actor or its forced reference), *condition* when a condition says so (GetIsID → the NPC, GetIsAliasRef → the alias, GetIsVoiceType → “voice MaleNord”, GetInFaction, GetIsRace), several possible speakers being listed with “or”; otherwise “undetermined speaker” (generic greetings); the quest, the topic and its category, the listener; **the player's question** (RNAM, or the topic text) with its translation; the **responses** in order, each with its number, its emotion and intensity (“Anger 60”), its source, its translation or “(to translate)”, and the author's NAM2 note; the **conditions** in plain language; the **voice** (§ 8.9). Clicking a response selects its line in the grid; double-clicking opens it in the edit window.

Conditions in plain language. Each condition of the reply is decoded through the function table of its game family (`assets/data/conditions/<family>.json`: index, name, parameter types) and stated in plain English for the sixty or so common functions — “if the speaker is Ralof”, “if quest MQ101 is at stage ≥ 20”, “if the player is a woman”, “if quest MQ101 is completed”, “if the speaker has the keyword …” — or otherwise in the generic form “Function(parameters) op value”; a function missing from the table is shown as “Function #4096”. Conditions are joined by “and”, and the plugin's OR flag by “or”; a condition on the listener or on a reference says so first (“on the listener: …”); a global value is named. Hovering over a condition shows its raw form (“GetIsID([01000A01 Ralof], 0) == 1”). The Skyrim and Fallout 4 tables are complete; Oblivion, Fallout 3 / New Vegas and Starfield have the names of the common functions, and the others are shown by their number.

Following a conversation. *Next* goes to the next response or reply of the topic (file order is the game's priority order), then to the first reply of the first linked topic (TCLT); in a scene, to the next action of the phase, then to the next phase; *Previous* follows the PNAM link when there is one, otherwise goes to the previous reply. *Ctrl+Alt+Left* / *Ctrl+Alt+Right* do the same in the Dialogues window and in the Dialogue tab; *Enter*, a double-click or *Open in the editor* open the edit window on the line, and it then follows the navigation. Selecting a reply selects the line of its first response in the grid; with *Follow the grid* (checked by default, `[Dialogues] FollowGrid`), selecting a line in the grid selects the corresponding node.

Filters and NPCs. The *Quests* list and the *NPCs* list (the document's resolved speakers, with their gender and number of responses, the most talkative first), the search in the sources and translations of the replies (a quarter of a second after you type) and the *Unvoiced* checkbox (§ 8.9) reduce the tree to the matching nodes; the window's status bar counts “60 replies · 12 translated · 4 unvoiced · 3 NPCs · filter: …”. *Filter the grid on this NPC*, next to the NPC list, and *Filter on this NPC* in the line menu (on a reply whose speaker

is resolved) set a grid filter that combines with the others, shown as the "NPC: Lydia ⨯" chip in the filter bar; Reset removes it. The window remembers its geometry (`[Window] DialogX/Y/W/H`, 900 × 580 by default) and the position of its splitter (`[Dialogues] Split`); Esc closes it.

Flat, in the grid. The Comment column (§ 8.1, *Options ▸ Columns ▸ Comment*) summarizes this context line by line — speaker and gender, quest, topic, emotion, response rank —, and *In: Comment* in the filter bar (§ 4.3) searches it: "(f)" for replies spoken by a woman, "Anger" for lines played in anger, the name of an NPC.

8.9 Voices

You translate a voiced reply with in mind the subtitle that scrolls during the English voice; a reply without a voice is a gap the mod author will want to know about. On first request — Dialogues window, Dialogue tab, Comment column, "voice ✗" filter — the program builds the mod's **voice map** without extracting or reading the audio: it lists the loose files under `<Data>/Sound/Voice/<plugin name>/` (Data is the plugin's folder, which covers a mod opened from its Mod Organizer or Vortex folder) and all the archives in the plugin's folder whose name starts with the plugin's name (`MonMod.bsa`, `MonMod - Voices.bsa`, `MonMod - Voices_en.ba2`…), of which only the directory is read. The game's archives are not consulted. The map is kept with the document; *Refresh*, at the bottom of the Dialogues window, rebuilds it after files are added.

Matching. A voice file is recognized by the suffix "_<FormID as eight hexadecimal digits>_<response number>.<ext>" (fuz, xwm, wav, wem, mp3, ogg, lip depending on the game): the low twenty-four bits of the FormID are compared with those of the reply — the load index written in the name depends on the author's load order, so "0501A3F2" matches "0101A3F2" —, and the number with that of the response. The parent folder gives the voice type (Skyrim, Fallout 4) or race / sex (Oblivion, "Imperial/M"). When a response has several voice types, all are listed. A `.lip` file alone is not a voice ("lips only"); a `.fuz` file contains both voice and lips. Starfield names its `.wem` files by hash: nothing is matched, and the window says "unresolved (Starfield: N .wem files)". These conventions are in the `voice` section of `assets/data/dialogue/<family>.json` (root folder, extensions, naming mode, folder depth).

Display. In the Dialogues window, each response has its own line: "✓ 1 · MaleNord · MQ101_MQ101RalofTopic01_01000301_1.fuz (loose)", "✓ 2 · malenord · … (MonMod - Voices.bsa)", or in red "✗ 2 · no voice file" and "✗ 1 · lips only (…lip)"; the tooltip gives the full path, or the archive and the internal path; below the conditions, the *Voice* section repeats them, with *Open the folder* for a loose file. The Dialogue tab of the edit window gives the same line, abbreviated; the Comment column (§ 8.1) ends the reply with "voice ✓" or "voice ✗", which you can filter with *In: Comment*. A reply counts as **unvoiced** as soon as one of its responses has no voice file: the window's status bar counts them, so do the tree counters, and the *Unvoiced* checkbox keeps only them. *Copy the list of unvoiced replies* puts one line per reply on the clipboard — "FormID ⇥ topic ⇥ speaker ⇥ text" — for you to send to the mod author.

No source. A mod without a Sound/Voice folder or archive (or with an empty folder) has no known voice: nothing is marked ✗, the counters stay at zero and the window says "No voice file for this mod (neither Sound/Voice/MonMod.esp nor an archive).". Audio playback and the 3D viewer are planned for step 3.

8.10 Translating online

A machine translation service takes ten seconds to produce the first draft of a three-thousand-word book and leaves you the review, where your value is real. The program connects to **DeepL** (free or pro), **Azure Translator**, **OpenAI** and any server that speaks the same API (LM Studio, Ollama, vLLM… on your machine) and **Google Cloud Translation**. Three principles: nothing is sent without an action on your part; what is sent is announced (lines, characters, requests); what comes back is marked *Pending* with its provenance and is undone with Ctrl+Z. The dictionaries (§ 7.3) remain the first resort; the service handles what they leave.

Configuring. *Settings* ▸ *Online* (§ 16.4): choose a service, paste its key, set its endpoint and options, then *Test…* — DeepL answers with your usage for the month, the others with a control sentence; an error is reported with the service's code and message. A service is usable when its key is present (or when its local server responds); the *Default service* is the one Ctrl+G calls. Each service has its own terms and pricing: DeepL offers 500,000 characters per month with a free key, the others are billed by usage. The key is stored in the system keyring — Windows Credential Manager, macOS Keychain, Secret Service on Linux — and, without a keyring, in `secrets.json` in the configuration folder, readable by you only; never in Config.ini, never in the log.

OpenAI-compatible profiles. OpenAI, Mistral, an Ollama or LM Studio server on your machine speak the same language ("Chat Completions") with different settings: each has its **profile**, a JSON file that describes the connection — endpoint, model, headers —, the *Prompt* sent to the model, the answer format, the sampling (temperature, tokens, penalties) and its own guards. Three profiles are shipped in `assets/data/online/profiles/`: `openai` (api.openai.com, gpt-4o-mini), `local` (Ollama, http://localhost:11434/v1, model to fill in, batches of 20 lines and 3,000 characters, 120 s timeout) and `mistral` (api.mistral.ai); a change copies them into `data/online/profiles/` of the configuration folder, which wins, and *New profile* or *Duplicate* creates others. Each profile is a service of its own: it has its line in the list of services, its key in the keyring, its cache of answers (`cache/online/openai-<id>_<SRC>-<DST>.ximdb`) and its name in the provenance of the lines received ("online (Ollama local)"). For an Ollama server: install a model (`ollama pull mistral-nemo`), copy its exact name (`ollama list`, for example `mistral-nemo:latest`) into the *Model* of the "Ollama local" profile, tick *Usable without a key*, *Test…*: the server's list of models is queried, then the control sentence is translated with the profile's prompt.

The prompt. It is sent as the system message; `%lang_source%` and `%lang_dest%` — the syntax of ESP-ESM Translator, so a prompt written for it pastes as is — or `{src}` and `{dst}` become the English names of the languages of the pair ("from English to French"), `%game%` the name of the document's game, `%n%` the number of texts of the batch; saving warns if the prompt does not name the target language. The shipped prompt asks for a JSON array of the same length and describes the protection tokens (`<a_A>`…), always checked on return. For a model that writes JSON badly, the *Separated lines* format sends each text after a numbered marker "⟨⟨1⟩⟩" (or the separator chosen) and reads the answer back on the same markers, in any order. An answer that does not have as many texts as the batch is refused and the batch is sent again once line by line. The parameters ticked (temperature, max tokens, frequency and presence

penalties) are sent as they are, the others left out; the profile's ticked guards (lines per batch, characters per request, requests per minute, timeout) replace for it those of the Guards group.

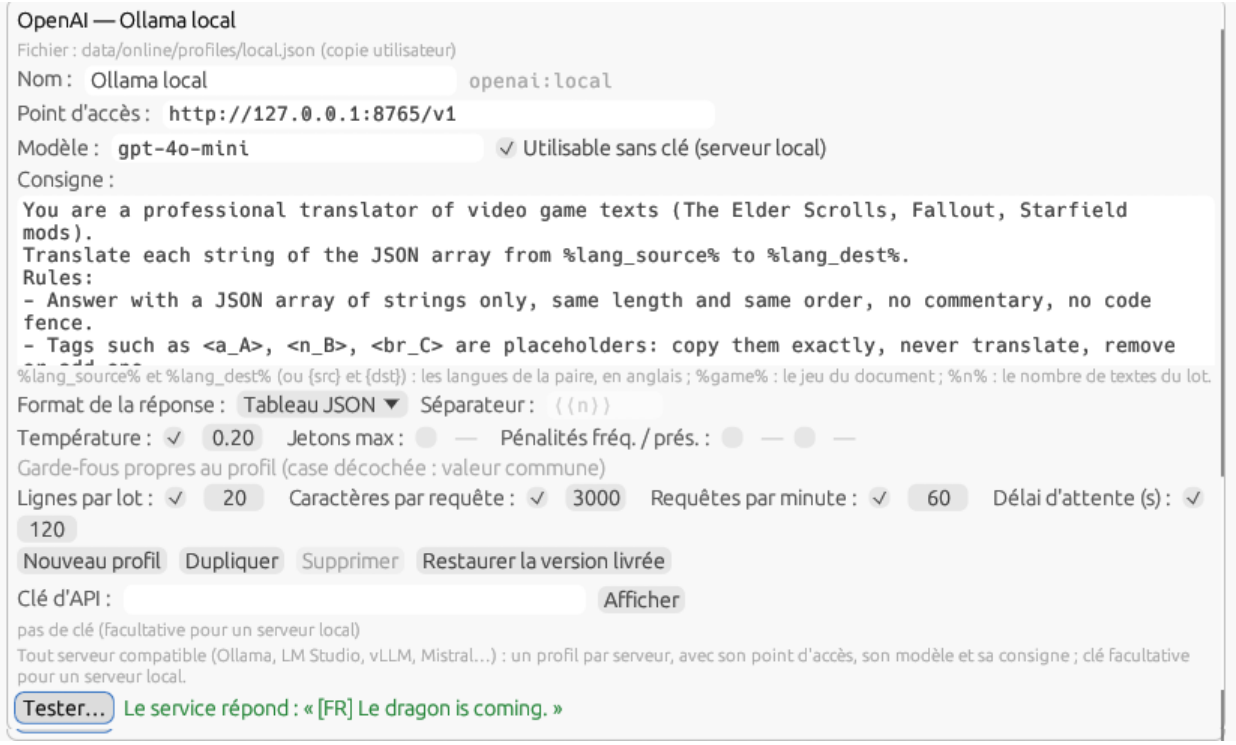


Figure 8.10 — Settings ▸ Online: the "Ollama local" profile pointed at a local server, its prompt, its guards, and the result of Test….

The line. *Ctrl+G*, the line menu ▸ *Translate online* or the *Online* button of the edit window send the source of the current line — or of the selected lines, as one batch — to the default service, in the document's Source → Target pair. The answer replaces the translation, with the status *Pending*, provenance "online (DeepL)" (status tooltip, Record tab, Comment column), as one undoable operation ("Undo: Online translation (1 line)"); a *Validated* or *Custom* line is replaced only after confirmation. Without a usable service, Ctrl+G opens the browser on Google Translate, as in the Translate a FOMOD window, and the status bar points you to the Settings; in that window, Ctrl+G also uses the service when there is one.

The batch. *Ctrl+Shift+G* or *Tools* ▸ *Translate online…* opens the dialog: the *Scope* (current line, selection, filtered view, whole document), the *Only the lines to translate* box (checked: the To translate lines and the empty Pending lines; unchecked: the whole scope except Ignored and never-shown lines), the service, then the announcement of what will be sent: lines and distinct sources (a repeated source is sent only once and its answer is placed on all its lines), characters, number of requests according to the safeguards, number of aliases, numbers and line breaks protected, the session counter, and the lines the cache will serve without sending anything. *Translate* starts sending in a background thread: the progress bar advances request by request, *Stop* interrupts after the current request and keeps what has been received, and the log lists each line received or refused (service error, lost token, empty answer or answer identical to the source, line too long). The received lines come in as one undoable operation ("Undo: Online translation (184 lines)"). The mod dictionary is touched only on save, as for any line.

Mass translation. The *Whole document* scope takes every line of the document, whatever the filters set, and says how many are to translate out of the total ("Whole document (2761 lines to translate out of 2968)"); it is the scope proposed on opening when nothing is selected, then the one chosen last (`[Online] LastScope`); a selection of several lines wins. The "What will be sent" block explains the count: the lines locked by the exclusion rules, Ignored or never shown in game are never sent; the lines already translated are not sent again while the box is checked; the lines the cache will serve go without a request. During the sending, *Close* is greyed out; *Stop* lets the current request finish. The account gives the lines received, refused and not sent ("Received: 858 · refused: 0 · not sent (stopped): 1903"), then the refusals by reason. The lines received are Pending, provenance "online (service)" — none becomes Validated — in a single operation: one Ctrl+Z takes them all back; the lines not sent stay as they were, and a new sending picks them up.

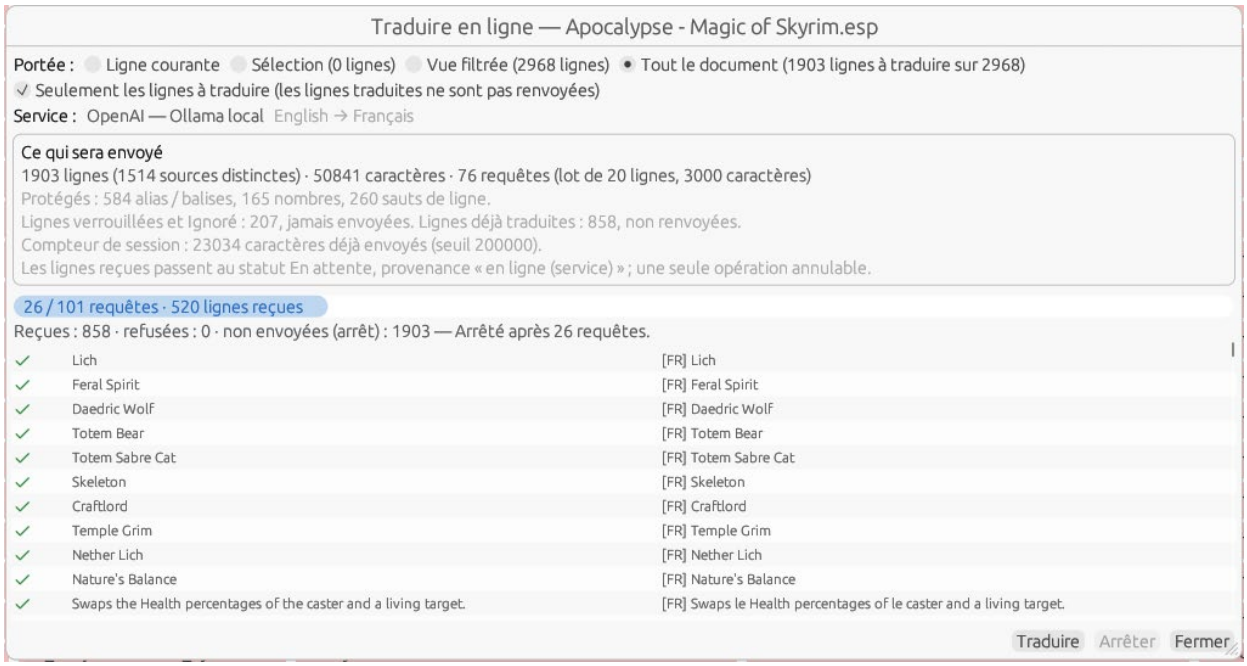


Figure 8.10 b — Translate online on the whole document, stopped after 26 requests out of 101: the scope, the lines left aside, the account (858 lines received, 1,903 not sent).

What is protected. Before sending, in each source, the game's aliases and tags (`<Alias=…>`, `<Global=…>`, book tags…) and the placeholders of text files (`{0}`, `%s`) become `<a_A>`, `<a_B>`…; numbers become `<n_A>`…; line breaks become `<br_A>`…. The services that allow it receive these tokens as tags not to be translated (DeepL, Azure, Google); OpenAI receives the instruction. On return, each token gets its value back; a missing, duplicated or altered token makes the answer refused: the line is not modified and the log says so. The order of the tokens may have changed — the target language sometimes requires it —, which A-07 reports as a warning, not as an error. Each of the three protections can be unchecked in the Settings (numbers, for example, for a language where the service reformats them usefully).

Safeguards. Characters per request (5,000), lines per batch (50) and requests per minute (20) split and pace the sending; a line longer than the limit is sent alone, never truncated, and is refused if it still exceeds it. A counter of the characters sent since launch is shown in the dialog; beyond the threshold (200,000, 0 to never ask), each new batch asks for confirmation. Each answer is kept in

`cache/online/<service>_<SRC>-<DST>.ximdb` in the configuration folder: a source already translated by the same service in the same pair is not sent again, even after a restart; *Clear the cache* erases it. The proxy from the Settings, or failing that the HTTPS_PROXY environment variable, is honored; the timeout is adjustable; requests and answers are not logged.

8.11 Reviewing a translation with someone else

Review by a second person is where a mod translation gains its quality. Rather than passing the same file back and forth — whoever saves last overwrites the other —, the reviewer has a column of their own, *Review*, and the translator accepts the proposals line by line. *Options* ▸ *Columns* ▸ *Review* (`[Columns] ShowReview`, unchecked at first) shows it between Translation and Provenance; the edit window then shows a Review field below the translation.



Figure 8.11 — The Review column: in blue the proposals that differ from the translation, in grey an accepted proposal; the Review sub-menu of the line menu (French interface).

- **Typing a review**: the Review cell is edited like the Translation cell, and so is the field of the edit window. Changing the review touches neither the translation, nor the status, nor the provenance; it is an undoable “Review” operation.
- **Accepting**: *Ctrl+Shift+R* or the line menu ▸ *Review* ▸ *Translation = Review*, on the line or the selection: a non-empty review that differs replaces the translation, the line becomes Validated (Pending if it equals the source) with the provenance “review” (§ 9.1). The review stays, in grey, until *Clear the review*. A single undoable operation for the whole selection; the status bar gives the number of lines validated.
- **Starting from the translation**: *Review = Translation* copies the translation into the review, so the reviewer corrects instead of retyping everything; *Clear the review* empties it.
- **The lines to look at**: the ≠ button of the Review cell of the column filter row (§ 10.1, Ctrl+Shift+F), present while the column is shown, keeps only the lines whose review differs from the translation; it combines with the other filters, and the eraser of the row and Reset release it.

Loading the reviewer's copy. The reviewer works on their copy and saves it as XML or JSON (Save as…). *File* ▸ *Load into the review…* opens any file the application reads as a translation — XML, pivot JSON, SST, EET, DSD, .ximdb, or STRINGS of the target language (matched by string number) — and puts its

translations into the Review column, without touching the translations. *Matching*: *Same record, field and source* (default; the FormID and EDID too when both sides have them) or *Same source (text only)*. *Fill only the lines whose review differs from the translation* (checked) leaves empty the lines the reviewer did not change; *Replace an existing review* (unchecked) protects a review typed in the meantime. Before loading, the preview counts the lines of the file, the matched ones, the reviews different from the translation, the lines without a match (source changed or missing) and the reviews already present. Loading is an undoable operation; the column shows if it was hidden.

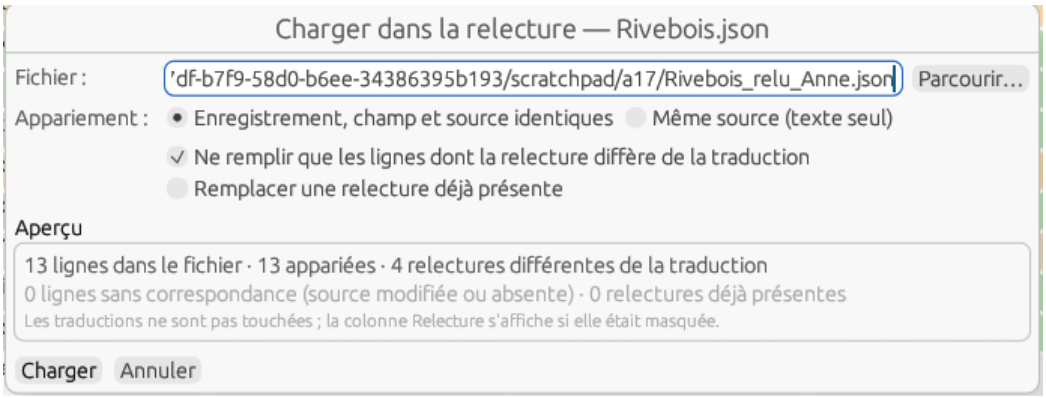


Figure 8.12 — File ▸ Load into the review…: the reviewed copy, the matching and the preview (French interface).

Where the review goes. It is saved to the mod's dictionary (format 4), the pivot JSON and the native XML; Save as… in a format that does not carry it (SST, EET, xTranslator XML, DSD, CSV, STRINGS) warns: “The review of 37 lines is not saved in this format.” Game exports, creating and applying dictionaries ignore it: only the translation counts.

8.12 History and going back

Undo (§ 6.6) reverts the last operations of the session and is emptied when the file is closed. The **history** keeps, per file, the dated list of the successive translations of every line: it survives saving and reopening, can be looked at, and lets you go back to an earlier version, line by line, even days later.

- **What is recorded**: every undoable operation leaves one entry per line it changes — date and time to the second, identity of the line (signature, field, FormID, EDID, rank), translation and status before and after, provenance after, review before and after when it changes, and the name of the operation (“Typing”, “Applying dictionaries”, “Online translation (DeepL)”, “Translation = Review”…). Typing counts as one entry per cell, like undo. The log only grows: undoing or redoing adds an entry (“Undo: Typing”) instead of removing one. The pre-filling done when a file is opened is not recorded.
- **Where it is kept**: in memory during the session and in the autosave — resuming after an abnormal stop finds the entries of the lost session (§ 7.5) —, then, every time the mod's dictionary is saved, in a companion file with the same name in the same folder, `<mod dictionary>.ximhist` (one JSON entry per line). A document without a mod dictionary (no target language) keeps its history in the pivot JSON when you save it in that format. The history never goes into a game output, a shared dictionary or a mod archive.

- **How much**: at most *Entries kept per file* (Settings ▸ Advanced, 20,000 by default, from 1,000 to 200,000); beyond that, the oldest are removed when writing. Unchecking *Keep the history of changes* stops it: existing entries are kept, nothing more is added.

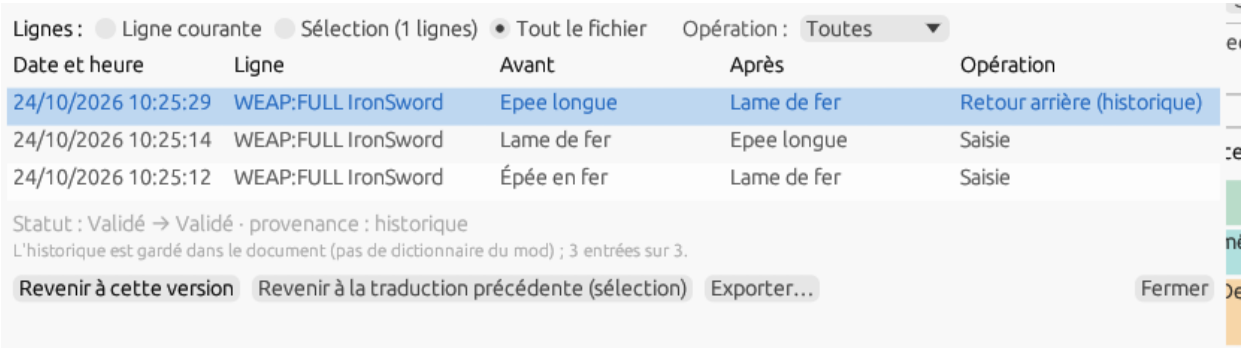


Figure 8.13 — Tools ▸ History…: the scope, the operation filter, the entries from the most recent to the oldest and the detail of the chosen entry (French interface).

Tools ▸ History… opens an independent window: scope *Current line*, *Selection* or *Whole file*, filter *Operation*, then the entries from the most recent to the oldest (date and time, line, before, after, operation). Clicking an entry selects its line in the grid and shows below it the statuses, the provenance and the review. A line whose source changed since the entry (mod update, § 12.11) is in orange, “different source”. *Export…* writes the entries shown as CSV. The position and size of the window are remembered (`[Window] HistX/Y/W/H`).

- *Go back to this version* (window, or the button of each entry in the *History* tab of the edit window): the line takes the “after” translation and status of the entry, with the provenance “history”. If the source of the line changed since, a confirmation is asked.
- *Go back to the previous translation* (*Ctrl+Alt+Z*, line menu ▸ *History*, window button): each line of the selection takes back the translation and status it had before the change that gave it its current translation; lines without an entry stay as they are and are counted in the message.
- Each of these returns is an undoable “Going back (history)” operation, itself recorded in the log.

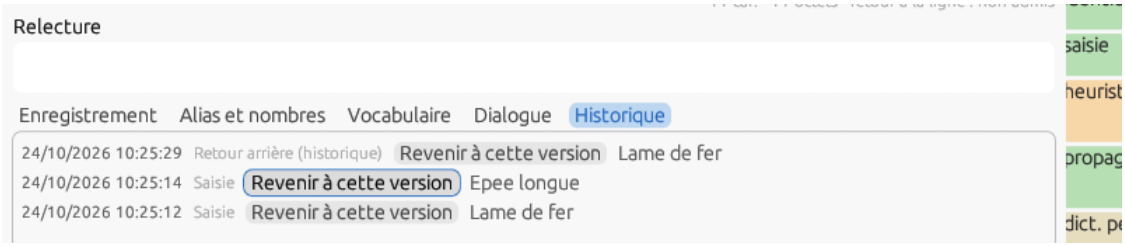


Figure 8.14 — The History tab of the edit window: the entries of the line, each with Go back to this version (French interface).

8.13 Scripts

Skyrim, Fallout 4 and Starfield mods put part of their texts through Papyrus scripts. This section describes what the application reads and translates of them.

VMAD strings

A record may carry a `VMAD` subfield: the scripts attached to it and the values of their properties — and, for a quest, a dialogue response, a package, a perk, a scene or a terminal, its script fragments. Among the

properties, the string ones (and string arrays, and the string members of Fallout 4 structures) sometimes carry a displayed text — an MCM menu title or page, a message, a name given to an object created by script — most often an identifier: a file name, a "$…" key, an event or menu name. Each one gives a line of field VMAD, index = rank of the string in the block; the tooltip of the Field cell and the Comment column give the script and the property (`MCM_skipper.Pages[0]`). Script, property and fragment names are never lines.

Locked by default. At opening, every VMAD line is *Ignored*, provenance "locked: VMAD disabled", not editable: translating an identifier would break the script. *Tools ▸ Scripts ▸ Translate the VMAD strings of this file* (figure 8.15), a check box of the document, unlocks them; it is an undoable operation, and unchecking it locks them again without losing anything: the translations stay in the mod dictionary and come back when it is checked again. The state of the box is kept in the header of the mod dictionary and in the JSON pivot; it comes back when the file is reopened.

Rules. With the box checked, the lines without translation that look like an identifier stay locked, with the rule as provenance: "$ key" (a key resolved by the interface files), "path or file" (`DLCRobot.esm`, a path, `_root.`), "no letter", "single word" (one word without space, like `weaponSwing` — but also "Help", hence the check). Such a line is unlocked by hand: a status (*F6*, *F5*…) or a typed translation gives it the provenance "typed". These rules are those of the exclusion rules (§ 8.14), in `assets/data/rules/exclusions.json`; a copy of the same name in the `data/rules/` folder of the configuration replaces it. The *Provenance ▸ Locked* filter isolates the locked lines.

Writing. The full plugin, the override patch and the delocalized or localized plugins (§ 19.4) rewrite the blocks that carry an exported line: only the translated string changes, with its length, the rest of the block is copied as it is; a block without exported line is copied byte for byte. VMAD strings are never in the STRINGS: with the STRINGS, hybrid or DSD output, the export dialog counts the exported VMAD lines the output does not write (then export the plugin as well).

Status bar. "VMAD strings: n" for a file that has some. A block of a version or a type the application does not know is not read: it gives no line, it is kept as it is when writing, the log names it, and the status bar shows "VMAD blocks not read: n".

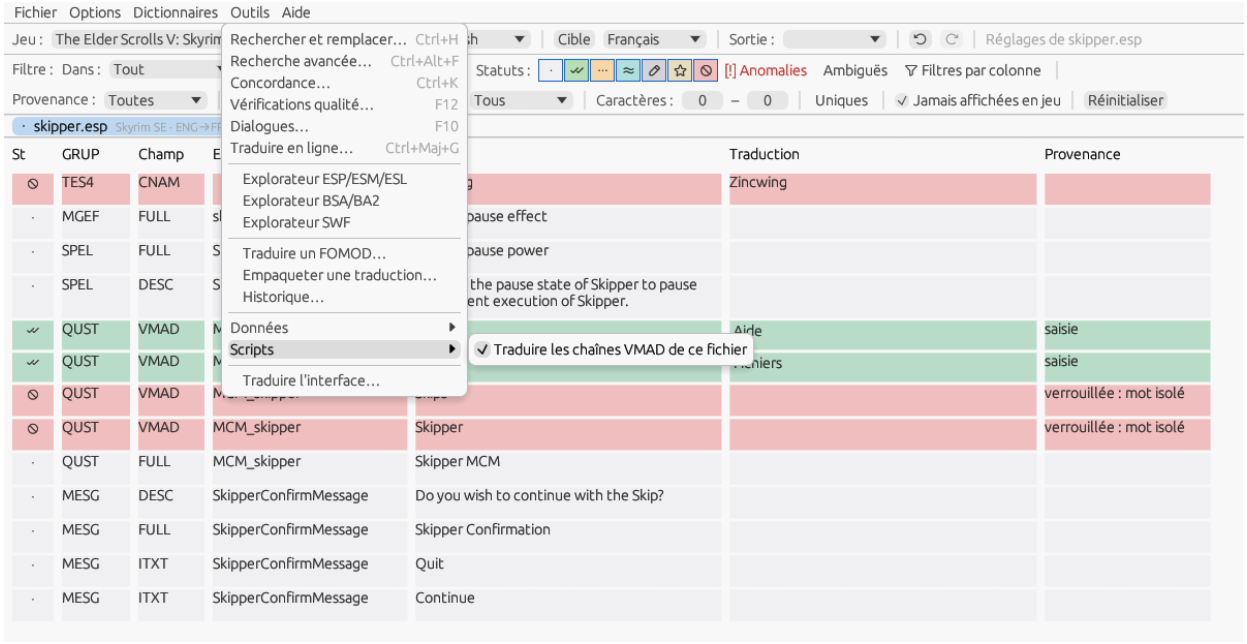


Figure 8.15 — The VMAD lines of Skipper, box checked: one line translated, the others locked by the "single word" rule; the Tools ▸ Scripts menu.

Compiled scripts (.pex)

A compiled Papyrus script carries a string table where identifiers (names of objects, functions, properties, variables, states) sit next to the literals of the code: texts of `Debug.Notification` and `Debug.MessageBox`, MCM menu options built by script, names given by `SetName`, but also file, event and menu names, "$" keys. The application reads the whole script, knows where each string is referred from, and shows only the literals as lines. Formats: Skyrim LE and SE (3.1, 3.2), Fallout 4 and 76 (3.9), Starfield (3.12); another number is not read.

Mod scripts. *Tools ▸ Scripts ▸ Mod scripts…*, on a plugin (figure 8.16), opens an independent window that looks for the mod's scripts: the loose `.pex` of the `Scripts/` folder next to the plugin (subfolders included, sources aside), those of the mod's archives (`<plugin>.bsa`, `<plugin> - Main.ba2`…), a loose script winning over the archive's one of the same name, as for the engine, and the `.psc` sources (`Scripts/Source/`, `Source/Scripts/`, `Scripts/Source/User/`, archives). The table gives for each script its origin, its format, its strings, its texts (literals not locked), its translated lines, whether its source is there and its state (no text, modified, translated, unknown version: not read, unreadable). Opened from the game's own `Data` folder, where every mod puts its scripts, the window keeps only the scripts the plugin names (VMAD blocks, fragments).

The "scripts" document. *Open in a tab* opens a "<plugin> — scripts" document, one per plugin, that gathers the texts of all its scripts, fragments included (`QF_`, `TIF__`, `PRKF_`, `TERM__`…): field `STR`, EDID = the script, index = the index of the string in the table; the Comment column gives the function using it and the function called (`MCM_skipper.OnPageReset · MCM_skipper.AddTextOption`). It has its mod dictionary (`<plugin>_scripts_<SRC>-<DST>.ximdb`), its languages, undo, propagation and "All mods" like the others. *File ▸ Open* on a `.pex` gives the same document for that script alone.

Locking. The literals that are not texts stay *Ignored* with the rule as provenance: “script identifier” (the string is also a name of the script: translating it would change the name), “argument of …” (argument of a function whose strings are identifiers: `Game.GetFormFromFile`, `RegisterForModEvent`, `Debug.Trace`, `StorageUtil.*`…), then the rules of the VMAD strings ($ key, path or file, no letter, single word). A status or a typed translation unlocks the line. The functions and patterns are those of the exclusion rules (§ 8.14); *Tools ‣ Scripts ‣ Open the script rules* opens their editor, and a user copy of the former `papyrus_rules.json` is merged into it once.

The Script tab. For a script line, the edit window has a *Script* tab (figure 8.17): the function and the call, the number of uses of the string in the compiled script, whether it is shared with an identifier, and, when the `.psc` source is found, the line that contains the literal, in blue, with two lines of context on each side, comments in grey, read-only.

Writing. Output “Scripts (.pex)”, the only one of a “scripts” document: *Export the translation…* asks for a folder (the plugin's by default) and writes there, in `Scripts/`, the scripts that have exported lines — loose files, never in the archive —, an existing original being first copied to `backups/` (§ 11.2). Only the string table changes: a literal takes its translation in its place; a string shared with a name stays in place and its translation is added at the end of the table, only its literal uses being redirected; the rest of the file is copied. A script without exported line is not written. The window's *Save the translated scripts…* button does the same.

> **Note —** Fallout 4 prefers loose files over the mod's archive only if `Fallout4Custom.ini` contains, in the `[Archive]` section, `bInvalidateOlderFiles=1` and `sResourceDataDirsFinal=`. Skyrim and Starfield prefer them by themselves.

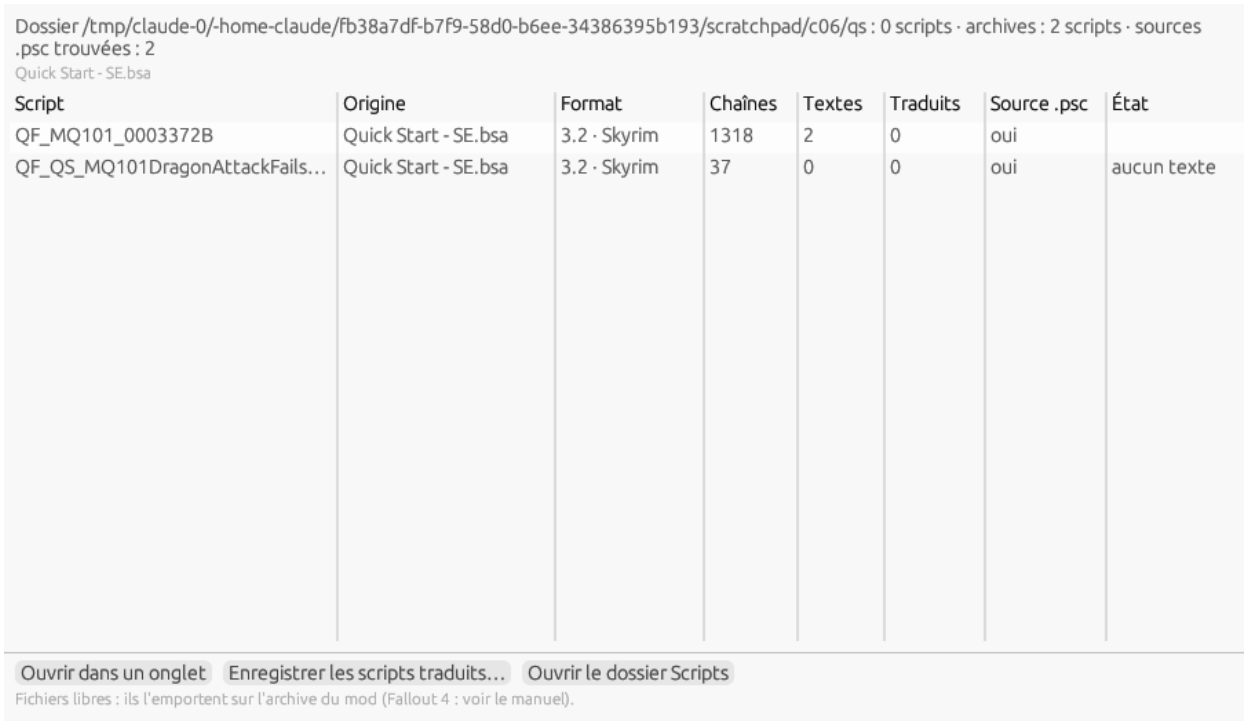


Figure 8.16 — Tools ‣ Scripts ‣ Mod scripts… on Skipper: the places searched, the scripts found, the actions.

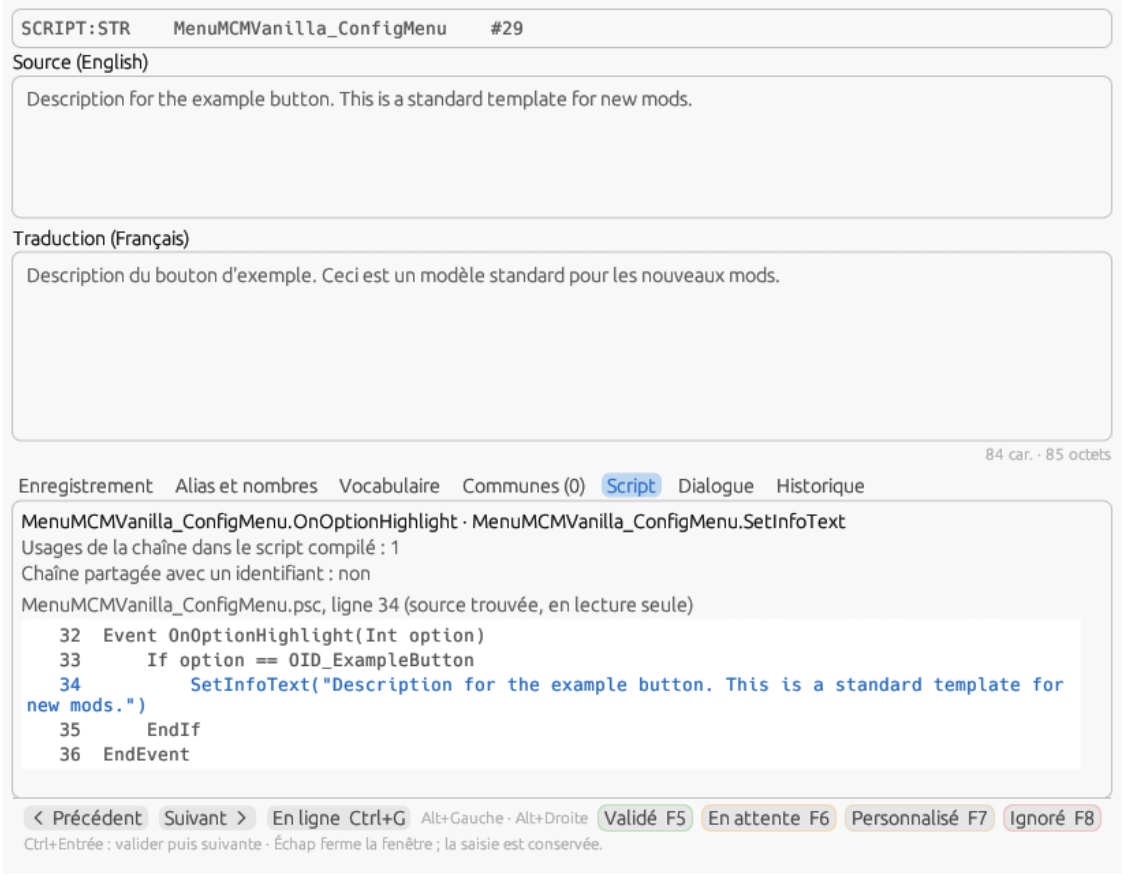


Figure 8.17 — The Script tab of the edit window on a script line: the function, the uses, the .psc source around the literal.

Copy into the source. Under the source, the *Also copy the translation into the .psc source* box (unchecked by default, `[Scripts] CopyToSource`) also writes, at export, the source of each written script, the translated literals replaced between their quotes, at its own place under the chosen folder (`Scripts/Source/`, `Source/Scripts/` or `Scripts/Source/User/` depending on the mod). The game does not need it; it is for whoever recompiles the mod.

Scripts of the old games (SCTX)

Oblivion, Fallout 3 and New Vegas keep their scripts in the plugin: a SCPT record, and the result script of a quest stage (QUST) or a dialogue response (INFO), carries the source (SCTX), the compiled form (SCDA) and its header (SCHR). The texts shown there are the arguments of `Message` and `MessageBox` — the message, then each button — and of their script extender forms `MessageEx` and `MessageBoxEx` (xOBSE, NVSE), where the buttons follow the message in the same string, separated by "|": each piece is a line. The lines are in the plugin's document, field `SCTX`, index = rank in the record; the Comment column gives the instruction, the rank of the message or button, and the ";" comment above the instruction. The other strings of the scripts (names, variables, arguments of other functions) are not extracted; format codes (`%.0f`, `%r`…) stay in the message and must be kept.

Source and compiled form. The engine runs the compiled form: at opening, each text is looked for in the SCDA in the form the compiler writes there ("%r" turned into a line break). At export (full plugin or override patch), the translation replaces the text in the source, quotes kept, and in the compiled form, with the

lengths of the instruction, of the `Begin` block and the compiled size in SCHR; the rest of the script is identical, a script without exported line is copied byte for byte. A text not found in the compiled form (script not compiled, unexpected form) gives a line locked “compiled form not found”: unlocked and translated, only the source changes, and the script must be recompiled in the Construction Set or the GECK. Messages are limited to 255 bytes (length counter).

The Script tab and the window. On a SCTX line, the *Script* tab (figure 8.18) shows the script and its FormID, the lines the script gives, the place of the text in the compiled form (or the warning “literal not found”), then the source read-only with the instruction line in blue and the comments in grey. For these games, *Tools ▸ Scripts ▸ Mod scripts…* lists the plugin's scripts that have messages, with their counts; *Open in a tab* selects the first line of the chosen script in the plugin's grid; the save buttons are greyed, the script is written with the plugin.

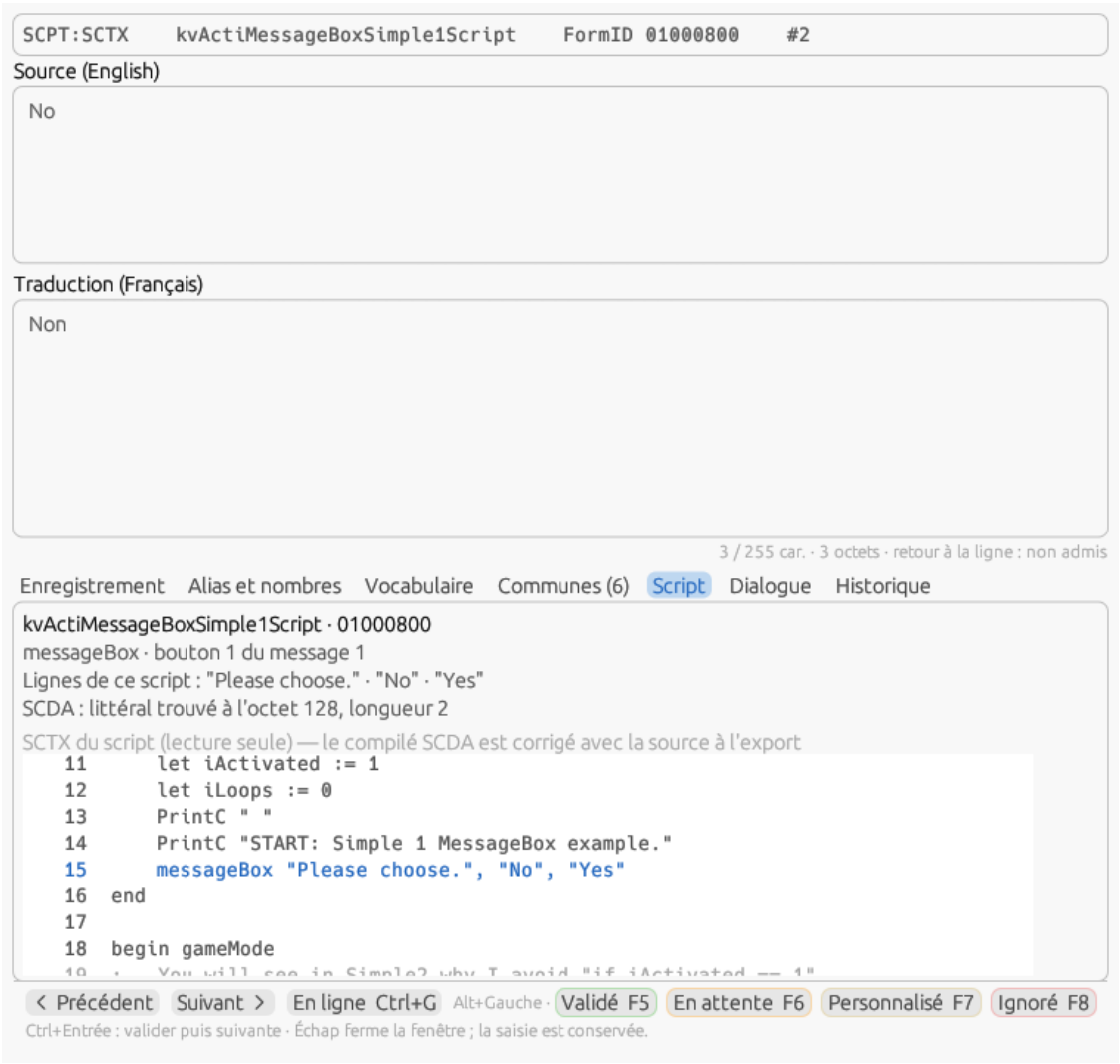


Figure 8.18 — The Script tab on an Oblivion SCTX line: the script's source, the instruction in blue, the lines of the script and the place of the text in the compiled form.

8.14 Exclusion rules

Part of the lines of a mod are not texts: identifiers (“xCACO_FireGrenadeEffect01Impact”), interface keys (“$Key”), paths, FormIDs, addresses, tags alone (“<Alias=Player>
”), technical values of MCM files. The **exclusion rules** recognize them and lock them: status *Ignored*, provenance “locked: identifier”, “locked: $ key”…; they are neither translated nor exported translated. The rules only touch the lines nobody decided (no translation, To translate, no provenance); they are lifted before the dictionaries fill the document and laid again after, at opening as at each reapplication. A line you unlock by hand (a status F5, F6… or a typed translation, provenance “typed”) is never locked again in this document.

Three levels. The **shipped** rules are in `assets/data/rules/exclusions.json`; a copy of the same name in the configuration's `data/rules/` folder replaces it (it is the one the editor writes) — it takes over the former `papyrus_rules.json` of the scripts, whose user copy is merged once then renamed `.bak`. The shipped rules: $ key; path or file (a line without space with a backslash, two slashes or a known extension — “Themes/Tags” and “” are not); no letter; identifier (no space nor punctuation, with an underscore, a letter followed by a digit or three inner capitals: “ITMPotionUse”, not “MrHandy”); hexadecimal FormID or colour; web address; tags alone (“<Unknown>” is a text); for text files, the technical keys of `MCM/Config/*/config.json` (type, id, sourceType, form, script…) and the identifier keys of JSON files (“*ID*”, “*Type”); for script strings, in addition, the single word and the functions whose arguments are identifiers (§ 8.13). The **deduced** rules are computed at opening: a script string or a text-file line equal to an EDID of the plugin (never a displayed field like FULL or DESC), a key of the mod's interface file, a file name of the mod's folder. The **learned** rules come from you, through the generalization below: local, they live in the header of the mod dictionary; global, in your copy of the file.

Tools ▸ *Exclusion rules…* opens a separate window with three tabs. *Active rules* sums up the document (“locked lines: 1,745; rules acting: 4; lines unlocked by hand, which win: 0”) and lists each rule with its *Active* box, its origin (shipped, deduced, learned for this mod or global, user, fields file, game sheet), its scope, its game, the number of lines it locks and an example. Unchecking a rule turns it off for this document (remembered in the mod dictionary): its lines go back to To translate and through the dictionaries, in one undoable operation. For the selected rule: *See the lines* restricts the grid to its lines (a “Rule: …” chip in the filter bar), *Unlock the lines* turns it off for the document, *Make global* / *Make local* moves a learned rule; *Apply the rules again* replays everything. The fields marked Locked in the fields file (§ 15.1) and, for Oblivion Remastered, the LOC_* keys are listed without a box: they are set where they are.



Figure 8.14 — Exclusion rules, Active rules tab, on Sim Settlements 2: the shipped and deduced rules, their counts and an example.

Editor. The *Editor* tab edits the user copy: on the left the list of rules, in order (the first that recognizes a line names it), with their box and *Add* / *Duplicate* / *Delete* / ▲ ▼ — deleting a shipped rule only turns it off, so that the restoration finds it; on the right the rule: identifier, type (regex on the text, REC:FLD field, EDID pattern, script function, text-file key), pattern, “And also” (a second expression that must also recognize the text), scope (text, script), files (for a key), game (* for all), comment; below the rule, the trial on the open document (“Trial on the document: 652 lines — first: “Airplane_Debris_02””); at the bottom, the script functions, one per line. *Save* validates the file (unique identifier, compilable expression, AAAA:BBBB signature, non-empty scope) — an invalid file is never written — then applies the rules again to the open documents, without restart; *Restore the shipped version* deletes the copy. The *Suggestions* tab computes, on demand, rules without applying them: a field the game's base never translates (more than fifty lines with source = translation), a group of at least ten lines Ignored by hand that share a field, a first word or an extension; *Add to the editor* puts it in the editor.

Generalize. When you set a line to Ignored (F8, button or menu) and other lines of the document share its text or its field, the *Generalize the exclusion?* window proposes to also ignore: the other lines with the same text in this mod (local rule, default choice); the lines starting with the same beginning (“zzz”, “Bld Deco”) in this mod or in all mods; the other lines of the same field — or the same key for a text file — in this mod or in all mods of the game, greyed out for a field shown in game (FULL, DESC, ITXT, NAM1…), whose exclusion would make real texts disappear; or nothing. The rule created locks its lines in one undoable operation; the origin line keeps its own state. The *Stop proposing generalization* box (like *Settings ▸ Advanced ▸ Propose to generalize an exclusion set by hand*, `[Rules] ProposeGeneralization`) replaces the window with a “Generalize…” link in the status bar. The window remembers its geometry (`[Window] RulesX/Y/W/H`).

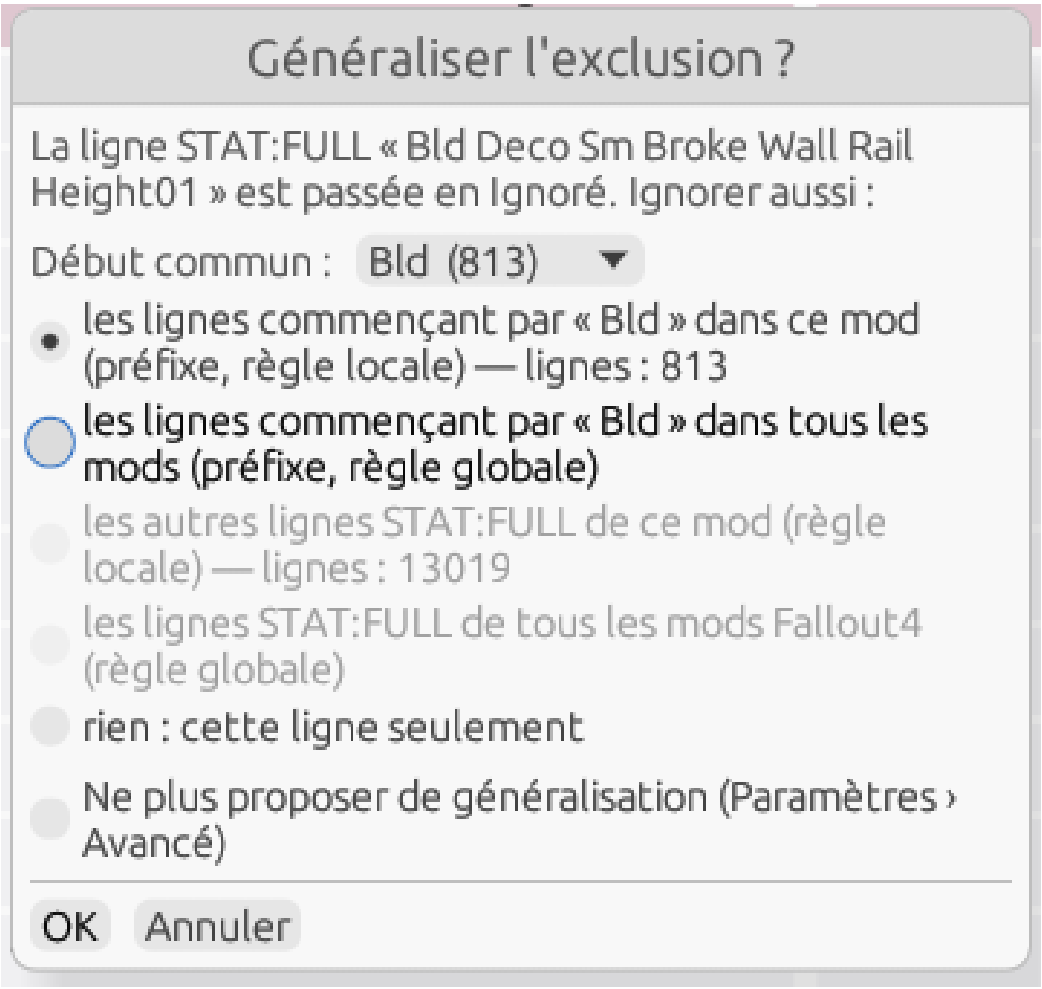


Figure 8.14 b — Generalize the exclusion after an F8 on an internal name of Sim Settlements 2: the exact text, the common beginning, the field greyed out.

9. Statuses and their colors

Each line has a status, shown by a background color and a glyph:

Status	Glyph	Meaning
To translate	•	Line not yet translated.
Validated	✓✓	Translation typed in an empty line, or validated (F5).
Pending	…	Translation incomplete or in progress.
By similarity	≈	Filled automatically from a dictionary.
Modified proposal	•▬	Automatic translation edited by hand: to be reviewed.
Custom	★	Marked as custom.
Ignored	∅	Line excluded from translation.

A special case: the **lines never shown in game** — the plugin header (author and description, `TES4:CNAM` / `TES4:SNAM`), the names that only Bethesda's editor uses (explosions, projectiles, hazards, and depending

on the game classes, factions, trees or lights), and, in Fallout 4, Fallout 76 and Starfield, the names of the **object templates** of weapons and armor (the predefined combinations of modifications, extra `FULL` lines of a `WEAP` or an `ARMO`) marked “editor only” — only the template of a unique legendary, which names the object in game, stays translatable and carries the comment “Object template”. They have the Ignored status, are shown on a red background (the Ignored color) with the source text in place of the translation, cannot be edited and receive nothing from the dictionaries: translating them would be wasted work. They are hidden by default; the *Never shown in game* box of the filter bar shows them (§ 10). The list is per-game data (“Never shown in game” mode in the field editor, § 15.1), deliberately cautious: if a mod author tells you about other useless lines, add them to it.

The colors are fully customizable via *Options* ▸ *Status colors…*. The picker offers an *Index color* tab (AutoCAD ACI palette) and a *True colors* tab (RGB). *Reset colors* restores the default palette.

9.1 The provenance of translations

The status tells where a line stands; the **provenance** tells where its translation came from. Every operation that writes a translation sets it on the lines it changes, and only on those: *typed* (typed in the grid or the edit window, pasted, Translation = source); *dictionary*, with the file, its role (game, mod, personal, text, ext., backup) and the match level (record and text identical, same text, text only, old data, same record with text changed, normalized form with what was carried over — § 7.3); *heuristic* with its score (Use button of the Vocabulary tab, instant translation F9 on the whole source); *online* with the service; *propagation* (copied from a duplicate, § 8.4); *replacement* (Find and replace, alias and number repairs, quality-check unification, spell checker); *target file* (§ 7.7); *review* (Translation = Review, § 8.11); *imported* with the format, for a translation read with the file opened (SST, EET, another tool's XML). A line without translation has no provenance.

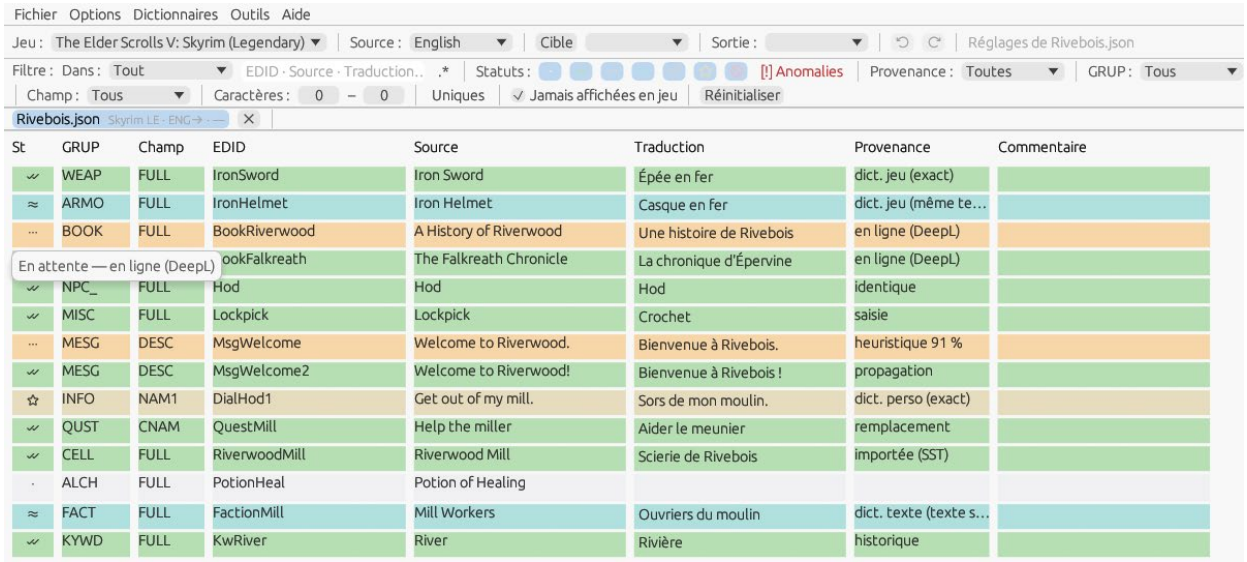


Figure 9.1 — The Provenance column (Options ▸ Columns) and, on the status cell, the “Status — provenance” tooltip (French interface).

- **Changing the status does not change the provenance**: a line received from DeepL then validated (F5) stays “online”; the tooltip says “Validated — online (DeepL)”. Editing by hand a translation that

came from elsewhere turns it into “typed”; clearing the translation clears the provenance; Ctrl+Z and Ctrl+Y restore it with the translation and the status.

- **Where to read it**: the tooltip of the status cell (“Status — full provenance”, for example “Validated — Skyrim_ENG-FRA.ximdb (dict. game), record and text identical”); the Record tab of the edit window (§ 8.5); the Provenance column, as a short label (“dict. mod (same text)”, “heuristic 91 %”).
- **Identical**: a non-empty translation equal to the source (outer spaces aside) shows as “identical” in the column and falls under “Same as source” in the filter, whatever its saved provenance — which stays in the tooltip. This is the case to review: a proper noun left as is, or an oversight.
- **Saved with the work**: the provenance is written to the mod's dictionary (.ximdb format 4, an eighth element per line; formats 1 to 3 read as before and a file moves to format 4 at its first save), to the pivot JSON (save and autosave) and to the XIMOD native XML. A line found again at reopening through the mod's dictionary gets back the provenance it had (“online (DeepL)”), instead of “mod dictionary”. The formats of other tools (SST, EET, xTranslator XML, CSV, STRINGS) and shared dictionaries carry none.
- **No new color**: the provenance is read in the text; the seven status colors are unchanged.

10. Filters and counters

The filter bar restricts the lines shown; the criteria combine:

- *Filter* (text): shows only the lines containing the text entered, in the target chosen with *In* (§ 4.3) — all, source, translation, EDID or FormID, signature, or the generated comment (§ 8.1).
- *Statuses*: one button per status, dark glyph on the status colour, framed while the status is shown, grey when hidden; a click hides or shows it again.
- *Ambiguous*: only the lines the dictionaries left ambiguous, marked ≠ (§ 11.4).
- *Advanced search (n)*: chip set by the advanced search (§ 10.2); a click removes it.
- *Column filters* (Ctrl+Shift+F): shows or hides the filter row under the grid headers (§ 10.1).
- *Provenance*: All, Typed, Game dictionary, Mod dictionary, Personal dictionary, Other dictionary (text-only, external, backups), Heuristic, Online, Propagation, Replacement, Review, History, Imported (target file included), Locked (script strings, § 8.13), Same as source, Untranslated (§ 9.1). It combines with the statuses: “Online” + Validated shows the lines of a service that you have already validated.
- *GRUP*: limits to one record type.
- *Field*: limits to one subfield.
- *Characters*: filters by text length.
- *Unique*: a single line per distinct source (with propagation, § 8.4).
- *Never shown in game*: also shows the lines the game never displays (§ 9), in red and not editable; unchecked at first, the box is remembered (`[Editing] ShowUnused`).
- *Record …* / *Source …*: chips placed by the line menu (§ 8.2) — all the lines of one FormID, or of one source text; a click on the chip removes it.

- *Reset*: clears all the filters, those of the column row included.

10.1 Column filters

The filter bar has a single text field, applied to one target. For a search that crosses several columns — “the names (FULL) of books (BOOK) whose translation still contains ‘Iron’” —, the *Column filters* button of the filter bar (*Ctrl+Shift+F*, `[Editing] ColumnFilters`, hidden at first) shows a row of fields under the grid headers: one under each visible text column — GRUP, Field, EDID, Source, Translation and, when shown, Review, Provenance and Comment; nothing under St nor under the status buttons, statuses being filtered in the bar. The fields combine with AND among themselves and with every criterion of the bar (text, statuses, exact GRUP and Field, provenance, length, unique, never shown); each acts a quarter of a second after the last keystroke or at once on Enter, *Esc* in a field clears it, and the eraser ⌫ left of the row clears every field in one click. Without the .* box of the bar, a field searches “contains”, case-insensitive; with it, each field is a regular expression (Rust syntax, case-insensitive, `(?-i)` in front to respect case) — one mode for the whole row. An invalid expression turns its field red, the tooltip gives the error, and that filter alone is ignored while the others keep acting. The EDID field searches the EDID and the FormID in hexadecimal, like the “EDID or FormID” target; the Provenance field searches the short label of the column (“game dict.”, “online (DeepL)”…), the Comment field the author's comment and the generated text. The Review cell also carries the ≠ button (§ 8.11). When no line is left, the grid names the active filters (“filters: GRUP « ARMO », Field « FULL »”); the same list is the tooltip of the “shown” count in the status bar. Hiding the row clears its fields: no filter acts unseen. The selection and the current line are kept while they stay visible.

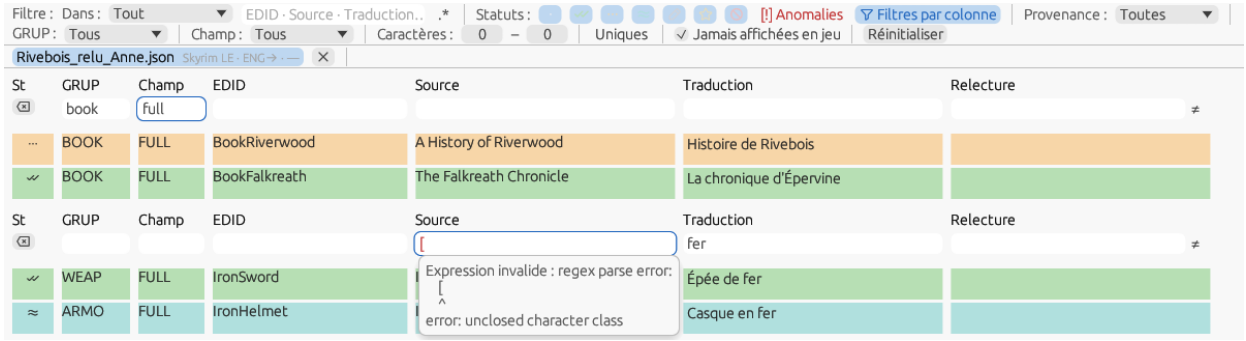


Figure 10.1 — The column filter row: “book” under GRUP and “full” under Field keep the book titles only; below, an invalid expression in . mode (red field, error in the tooltip), the other field still acting (French interface).*

The status bar counters (§ 4.5) reflect the result of the filtering live: “M shown” follows the view, the per-status pills count the whole document, and a click on one of them sets the matching status filter.

10.2 Advanced search

Tools ▸ *Advanced search…* (*Ctrl+Alt+F*) opens an independent window that combines six criteria with AND: *Source*, *Translation*, *EDID or FormID* (the EDID, or the FormID in hexadecimal like the “EDID or FormID” target of the bar), *GRUP*, *Field* and *Keyword* (searched in the source, the translation, the EDID, the signature and the comment). An empty field does not filter. Each field has its .* box — regular expression, Rust syntax — and its *Case* box; without .*, the field searches “contains”. An invalid expression

turns its field red, the error in the tooltip, and the search waits until it is corrected. The statuses and “Never shown in game” are chosen in the window as in the filter bar.

Captured groups. When Source and Translation are both expressions with at least one group in parentheses, the *Captured groups* option compares what each one captures, in order, without the edge spaces: *Identical between source and translation* or *Different*; a line where one of the expressions finds nothing is left out. With Source “(\d+)” and Translation “(\d+)”, Different finds the lines whose first number changed in translation — “You have 3 days left” translated “Il reste 5 jours”. Without a group in one of the expressions, the option is greyed.

Preview and result. A quarter of a second after the last change, the preview gives the number of lines found and, with Different, the first group pairs that differ (““3” / “5””). *Apply to the grid* (Enter) sets the search as one more filter, combined with AND with the bar: the bar shows the chip “Advanced search (n)”, which a click removes, as Reset does; the chip belongs to the file it was set on. *Select* selects the lines found without touching the filter; *Copy the list* puts one line per result on the clipboard, “FormID ⇥ EDID ⇥ source ⇥ translation”. Esc closes the window without removing the chip; the window follows the active file and its geometry is remembered (`[Window] SearchX/Y/W/H`).

Presets. *Save…* keeps the six fields, their boxes, the groups option and the statuses under a name; choosing a preset in the list fills the window; *Delete* removes one after confirmation. Four presets are shipped — *Different numbers*, *Missing tags* (the first <…> tag of the source missing or changed in translation), *Translation equal to the source*, *Lines without EDID* —; they cannot be deleted, but a preset of yours with the same name replaces them. Yours are in `data/search_presets.json` of the configuration folder, the application's in `assets/data/search_presets.json`.

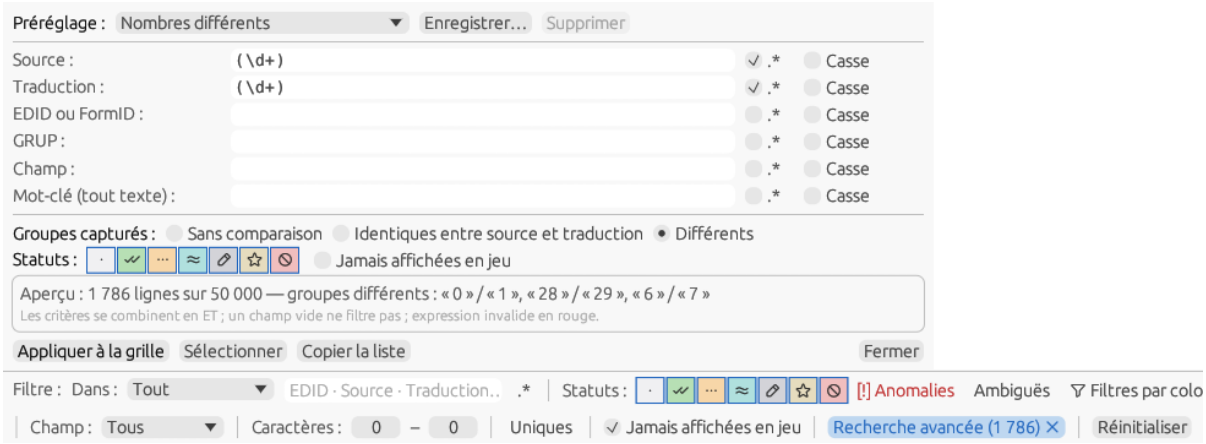


Figure 10.2 — Advanced search: the “Different numbers” preset, the preview and its group pairs; below, the chip set in the filter bar (French interface).

10.3 Concordance

The Vocabulary tab of the edit window offers the sources close to the line; the concordance answers another question: “how was this word translated elsewhere?”. *Tools ▸ Concordance…* (*Ctrl+K*, also in the line menu and in the context menu of the Source and Translation areas of the edit window) opens an independent window that looks for a term in the active dictionaries of the file (§ 12.8) — current lines and old data, signed or text-only lines — and lists the pairs found.

The term. At launch, it is the text selected in the Source or Translation area of the edit window; otherwise the word under the cursor of the area used last; otherwise the whole source of the current line. It can be changed in the *Term* field: the search follows the typing, *Enter* runs it again. The search is “contains”, case-insensitive; *Whole word* requires word boundaries (“Sugar” no longer finds “Sugarcane”); *In translations too* (checked at first) also searches the translations — useful to find the source of a French term; *Mod's dictionary only* restricts the search to the mod's dictionary. A one-character term is not searched (“Two characters at least.”). Ctrl+K on a window already open replaces the term.

The pairs. One row per distinct (source, translation) pair and per dictionary, sorted in the order of the dictionary stack then by source: the *Source* and the *Translation*, the term highlighted; the *Dictionary*; the *Record* — record and field of the first line carrying the pair, “— text” for a text-only pair; *N*, the number of lines of the dictionary carrying it. At most 500 pairs: beyond that, the header says so and invites you to refine the term. Below the options, a line recalls the dictionaries scanned, the number of lines read and the time taken. The search runs in the background — a million lines in a fraction of a second — and a new keystroke cancels the one running: the window never freezes.

Taking a translation. A click on a pair shows it in full in the *Chosen pair* frame at the bottom. *Copy* (or a double-click on the pair) puts its translation on the clipboard. *Into the line* inserts it in the Translation area of the edit window: in place of the selected text, otherwise at the cursor, or as the whole translation if the line has none yet — the concordance is for finding a term, not a sentence. The insertion is an undoable operation (Ctrl+Z); the line becomes Pending with the provenance “typed”. Without an open edit window, the button is greyed out. Esc closes the window; its geometry is remembered (`[Window] ConcordX/Y/W/H`).

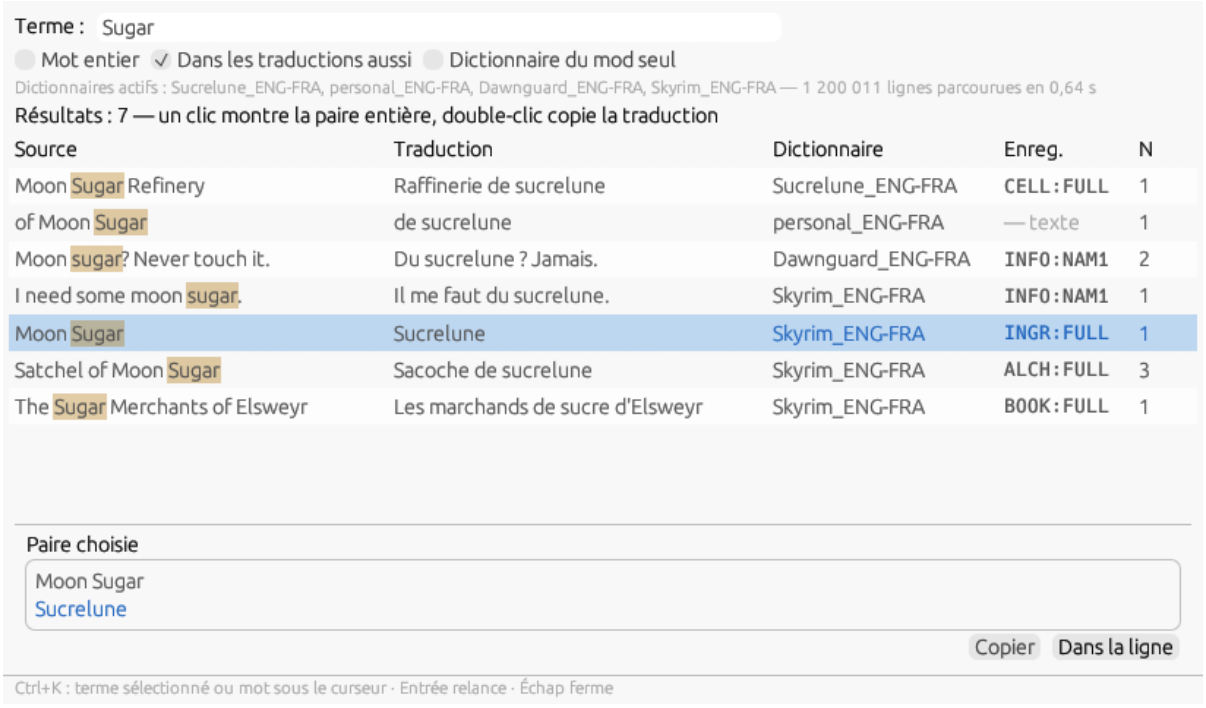


Figure 10.3 — The concordance of “Sugar”: the pairs of the mod's dictionary, of the personal dictionary then of the game dictionaries, the term highlighted, N lines per pair; at the bottom, the chosen pair, ready to be taken into the line (French interface).

11. Saving and exporting

11.1 Save as…

Menu *File* ▸ *Save as…* saves the work in a translation database format: `.sst`, `.eet`, `.dsd`, `.json`, `.csv`, XML or STRINGS. The format is chosen by the extension selected in the save dialog. Useful to keep a reusable memory or to resume later. The formats that have a comment column — EET, EET XML, XIMOD XML — receive the author's comment as it was read; with *Options* ▸ *Also export the generated comments (EET, XML)* (`[Export] GeneratedComments`, unchecked at first), the generated text of the Comment column (§ 8.1) is added after it, line by line — so a colleague's file receives no text they did not write, unless you ask for it; if the dialogue map is not built yet, it is built at that moment. The other formats and the mod dictionary never carry a generated comment.

Save a part…

Save as… writes the whole document. To send a reviewer only the Pending lines, submit the corrections of one GRUP or share the Validated part of a team's work, *File* ▸ *Save a part…* opens a dialog that chooses the lines before writing: the scope — *Selection*, *Filtered view* or *Whole document*, with their counts; the *Statuses* kept, the seven buttons of the filter bar; the *GRUP*, one chip per signature of the document (the number of lines in the tooltip) or *All*; the *Field*. Lines never shown in game are never written. The preview gives the number of lines and their breakdown by GRUP and by status, recomputed at each change; when there is nothing to write, the button is greyed out and the preview says so.

The file is proposed as "<document>_part.json" in the folder of the last save; *Browse…* picks another one. The format follows the extension, as for Save as…: `.json` (pivot, with provenance, review and the history of the lines written), `.sst`, `.eet`, `.dsd`, `.csv`, `.xml`; a STRINGS extension is refused, as a partial STRINGS file does not load in game. *Lines without translation: keep the source* writes the source as the translation of the kept lines that have none. The open document is neither modified nor renamed — it stays "modified" if it was — and the mod's dictionary is not written; the status bar says "Part saved in … — lines: N". The dialog remembers its choices for each file during the session.

The partial file comes back through *File* ▸ *Load into the review…* (§ 8.11) or is applied through *Apply the dictionaries…* (§ 11.3): the translator → reviewer → translator cycle, limited to the useful lines.

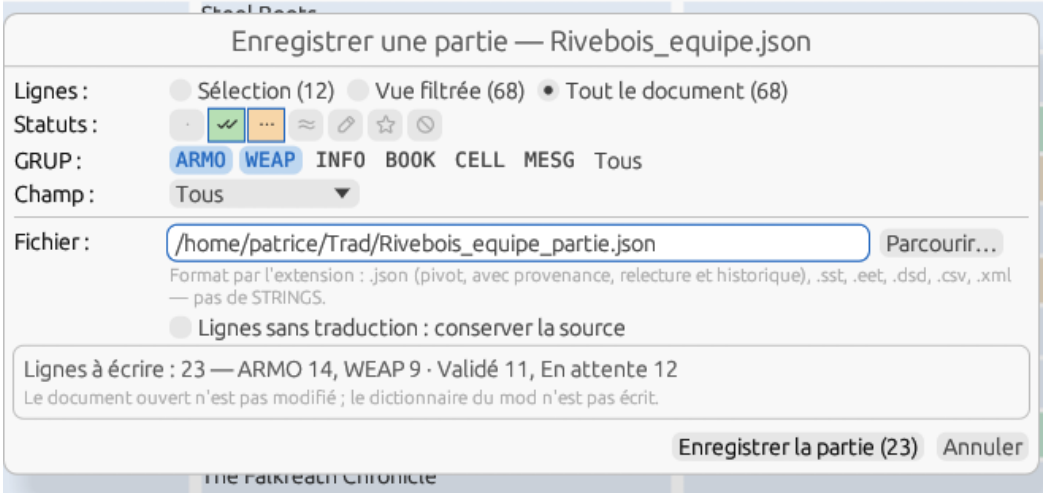


Figure 11.1 — Save a part: the Validated and Pending lines of two GRUP, the preview by GRUP and by status (French interface).

11.2 Export translation…

Menu *File* ▸ *Export translation…* produces the game-ready result, according to the document's *Output* (§ 6.1). A dialog comes before the choice of the destination file. For a text document (§ 7.7), the dialog adds the *Untranslated lines: keep the source* box and the proposed file is `<mod>_<target language>.txt`.

For a “scripts” document (§ 8.13), the Scripts (.pex) output writes the translated scripts as loose files in the `Scripts/` folder of the chosen folder.

Exported lines

Only the lines with the *Validated* status are exported; the others keep the source text. The *Also include* group lets you also take the *By similarity*, *Pending*, *Modified proposal* or *Custom* lines; the boxes are unchecked each time the dialog opens and a warning reminds you that these lines have not been reviewed. The count updates with each box (“1,022 lines will be exported (812 validated + 210 by similarity)”) and the end-of-export message repeats it. The *To translate* and *Ignored* lines are never exported. For the outputs that rewrite the text (STRINGS, hybrid, full plugin), a line that is not exported keeps its source text; for the override patch and the DSD, it is absent, and a record appears only if it has at least one exported field.

> **Note —** This behavior differs from previous versions, which exported every translated line: the lines filled by the dictionaries (“by similarity” status) are no longer exported without your review, unless you check the corresponding box.

Override patch of a localized plugin

For a localized plugin (Skyrim SE, Fallout 4, Starfield), the *Override patch* mode is available: the patch is written **delocalized** — translated records only, plain texts (the translation for the exported fields, the original text resolved from the STRINGS for the other text fields), “localized” flag removed, original plugin declared as the last master. The dialog shows the number of records and of unresolved ids (fields written empty if you continue) and offers to flag the patch as *ESL* (light) when the game and the patch allow it.

Keeping the original

When the file chosen for the export already exists — the open plugin itself, which the Full plugin mode proposes by default, or a previous output — it is first copied into the `backups` folder next to it, as “`<name>_<YYYYMMDD-HHMMSS>.<ext>`”, and only then replaced. If the copy fails (folder not writable, disk full), the export stops, the existing file is intact and the notification says why. After a successful write, only the N most recent copies of that file are kept. This applies to plugins, STRINGS files, text files and the DSD JSON; an export to a new path copies nothing.

The option *Keep the original before overwriting an existing file* and the number of *Copies kept per file* (5, from 1 to 50) are in *Settings* ▸ *Advanced* ▸ *Outputs* (`[Output] KeepOriginal`, `OriginalCopies`). Unchecked, no copy is made, except when the export replaces the open plugin: that case always copies, and the Exported lines dialog says so in Full plugin mode. The export notification names the copy (“Original kept: backups/MonMod_20261025-161203.esp”); *File* ▸ *Open the export backups folder* opens that folder in the file explorer.

STRINGS file names

The translated STRINGS files follow the game's convention: `MonMod_French.strings` for Skyrim and Fallout 4, `MonMod_fr.strings` for Starfield (two-letter codes). For Starfield, place them in `Data/Strings` and enable loose files in `StarfieldCustom.ini` (`[Archive] bInvalidateOlderFiles=1`, `sResourceDataDirsFinal=`).

11.3 Apply the dictionaries…

Options ▸ *Apply the dictionaries…* — or, from the line menu, *Apply the dictionaries to the selection…* — applies the document's active stack (§ 7.3, § 12.8) or a file of your choice (.ximdb, or a raw EET, SST, DSD, JSON, XML database read into memory; the chosen file is kept for the session) to a *Scope*: the whole document, the filtered view or the selection, with their line counts; the lines never shown in game and the Ignored lines are always out of scope. *Overwrite* says what may be replaced: Nothing (empty lines only, the behavior at opening), Everything but Validated and Custom, Everything but Pending, Pending only, Everything; a line the dictionaries do not know keeps its translation and its status, even with Everything. *Reset first* empties the translations of the scope (Validated and Custom excluded, except with Overwrite = Everything) before applying, in the same operation; this is how you regenerate the by-similarity lines from the current dictionaries (the former “Re-translate auto lines” entry no longer exists).

Matching: **Strict** — identical record, field, FormID, EDID and text give Validated (the stored status for the mod dictionary, Custom for the personal dictionary), then, in the same dictionary, the same text of the same field gives By similarity: the rule applied at opening; **Relax** — same record and field, same text → By similarity; **Text only** — same text, whatever the field, signed lines or not → By similarity; **Normalized** — same text but for numbers, punctuation and case, the translation adapted (§ 7.3) → Pending; Strict includes it when the Settings ▸ Advanced box is checked, as at opening; **FormID only** — same record, field, FormID and EDID but a different source text: the translation of the old text, with the Pending status since the line has changed — this is the mode for mod updates; **Approximate** — the word heuristic of the Vocabulary tab (§ 8.5): each line of the scope receives the best close source from all the dictionaries (on an equal score, the first in the stack), with the By similarity status and the provenance “approximate (75 %)”, above the threshold set in Settings ▸ Advanced and recalled in the label. The **preview** is recomputed a quarter of a second after each change (in the background with a progress indicator beyond 10,000 lines; in Approximate mode, after the word indexes are built, which is shown): “1,027 lines would receive a translation (812 strict, 215 relax), 0 overwritten”, and in Approximate mode the breakdown “1 at 90 % and above, 0 from 80 to 89 %, 0 below 80 %”; Apply is grayed out at zero. Applying is a single undoable operation (“Undo: dictionaries applied (1,027 lines)”), a toast gives the summary, each filled line carries its provenance (§ 7.3) and the anomaly indicators (§ 8.7) are recalculated. From the line menu, the dialog opens on the selection with Overwrite = Everything but Validated and Custom and Strict; the other settings are those used last time, except the scope. The glossary and the other assistants apply through a neighbouring dialog, Dictionaries ▸ Apply the assistants… (§ 11.5).

11.4 Ambiguous sources

When the dictionaries fill a line by the text of its source alone — the “same text” and “text only” levels, at opening as with Apply the dictionaries… —, they take the translation of the dictionary with the highest

priority. When the same source is translated in several ways in the active dictionaries (“Key”: “Clé” for the object, “Touche” for the keyboard), two translations in one dictionary included, the line is **ambiguous**: it still receives that translation, but in the *Pending* status instead of By similarity, with the ≠ mark in the status cell and the provenance “game dict. (same text) · ambiguous (2)” — the number is that of the distinct candidates (compared without their leading and trailing spaces). It is thus never filled silently. The levels that identify the record (identical record and text, FormID) are never ambiguous; a match of the personal dictionary wins and is not ambiguous: it is your choice. The end of the application says so (“… — 3 ambiguous lines left Pending”).

Deciding. In the edit window, the *Vocabulary* tab starts with the *Candidate translations (n)* section: each translation, the one the line carries in bold, with its dictionaries and the number of lines that give it (“Skyrim_ENG-FRA (3 times)”), and a *Use* button that takes it in the Pending status, with the provenance of that dictionary, like a suggestion. The mark goes as soon as you decide: typing, Use, Validated (F5) or Custom (F7); Ctrl+Z brings it back with the rest. It is saved with the provenance (mod dictionary, pivot JSON, native XML) and comes back when the file is reopened as long as the line has not been decided. The *Ambiguous* button of the filter bar keeps the marked lines only; Reset releases it. The ambiguities inside the document itself — two translations of one source in the mod — remain the job of the quality check (§ 8.7).

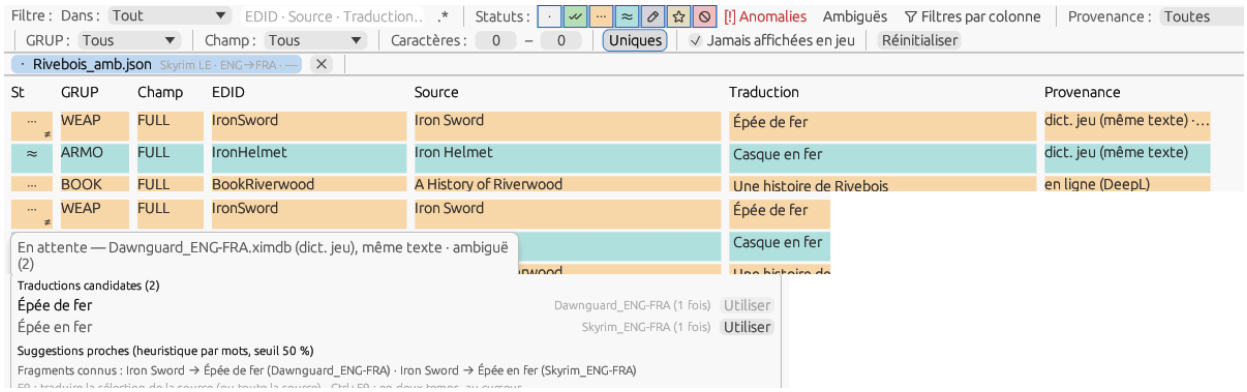


Figure 11.4 — An ambiguous line: the ≠ mark under the status, the provenance “… · ambiguë (2)”, the tooltip of the status cell, and the candidate translations of the Vocabulary tab (French interface).

11.5 Apply the assistants: glossary, patterns, derivations

The **assistants** translate without an existing dictionary, from rules you read and complete: the glossary of terms (§ 15.7), its composition by classes the patterns with variables and the derivations of objects (§ 15.7). They are **never** applied when a file is opened, only through *Dictionaries* ▸ *Apply the assistants…*, a modal dialog: the assistants checked (“Glossary (English → French, 290 terms)” and “Patterns with variables (6 patterns; composition: noun, bound, people, adjective, colour, origin, material)” when the pair has a patterns file, “Derivations (The Elder Scrolls V: Skyrim Special Edition, active rules: 10)” when the document's game has active rules for the target language), the *Scope* (whole document, filtered view, selection) and *Overwrite*, with the same choices as Apply the dictionaries (§ 11.3).

All or nothing. A line is filled only when **all** its words are covered by terms; tags, numbers and punctuation are copied. The order of the tries is fixed: the composition of the terms (pattern 1, when every word is covered and there is a noun), then the patterns of the file in their order, then — Glossary checked — the

glossary alone, in the source order. The line receives the first translation obtained, the status *Pending* and the provenance “rule: composition”, “rule: pattern 2”… or “rule: glossary”; the Provenance filter has its category *rule*. A line with a single word missing, that no pattern translates, is left alone: a half-done translation would look like a translation; F9 is there for those lines. Lines locked by the exclusion rules (§ 8.14) and Ignored lines are never filled.

Preview: “Glossary only: 1 line entirely covered”, “Composition: 837 lines”, “Patterns: 11 lines (pattern 2 “%A% of %B%”: 9, pattern 3 “%A% of the %B%”: 2)”, “Partially covered: 16 lines, left as they are”, then the total, the number of lines overwritten and that of the locked or ignored lines, left intact. *Apply* is one undoable operation (“Undo: Assistants (849 lines)”) and a toast gives the outcome. On Immersive Armors (865 armour names), the shipped glossary and patterns fill 849 lines, 408 of them identical to the French DSD translation (“Ritual Armor of Boethiah” → “Armure du rituel de Boéthia”, “Mantle of the Silver Hand” → “Manteau de la Main d'argent”); on Immersive Weapons (237 weapon names), a glossary of 100 terms and the patterns fill 153, 142 of them identical to one of the two DSD translations; the others differ mostly in vocabulary (“Bottes” where the human translation says “Solerets”), to be checked at review.

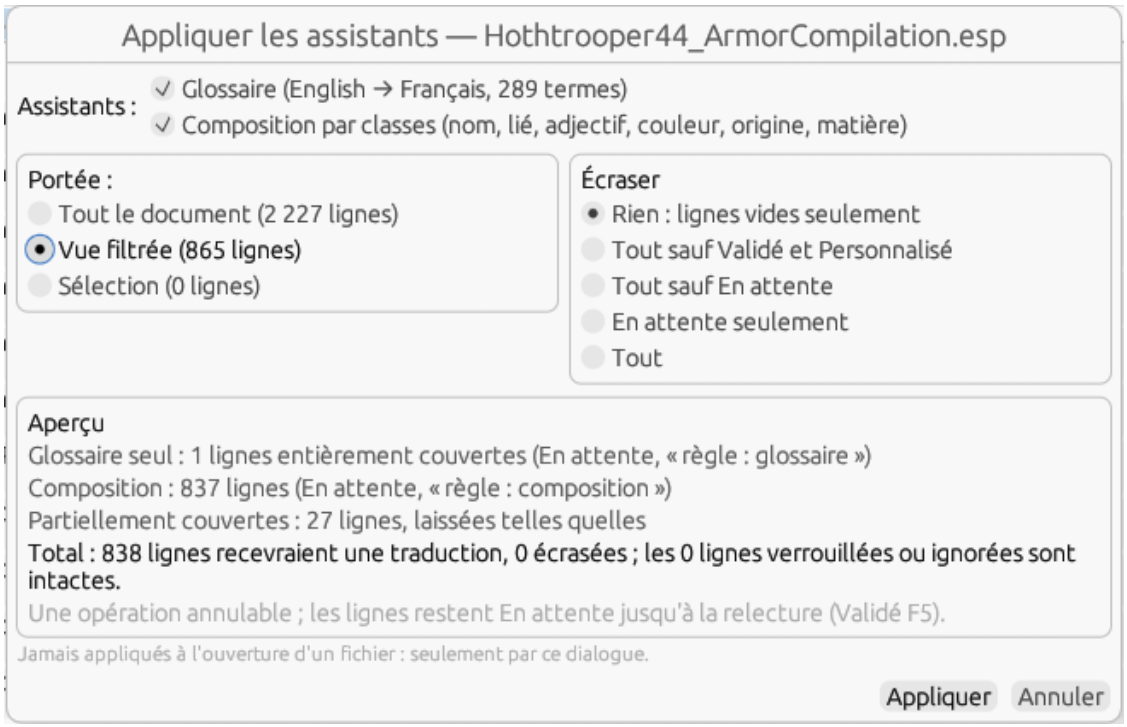


Figure 11.5 — Apply the assistants on Immersive Armors: the glossary and the composition, the scope and overwriting, the preview.

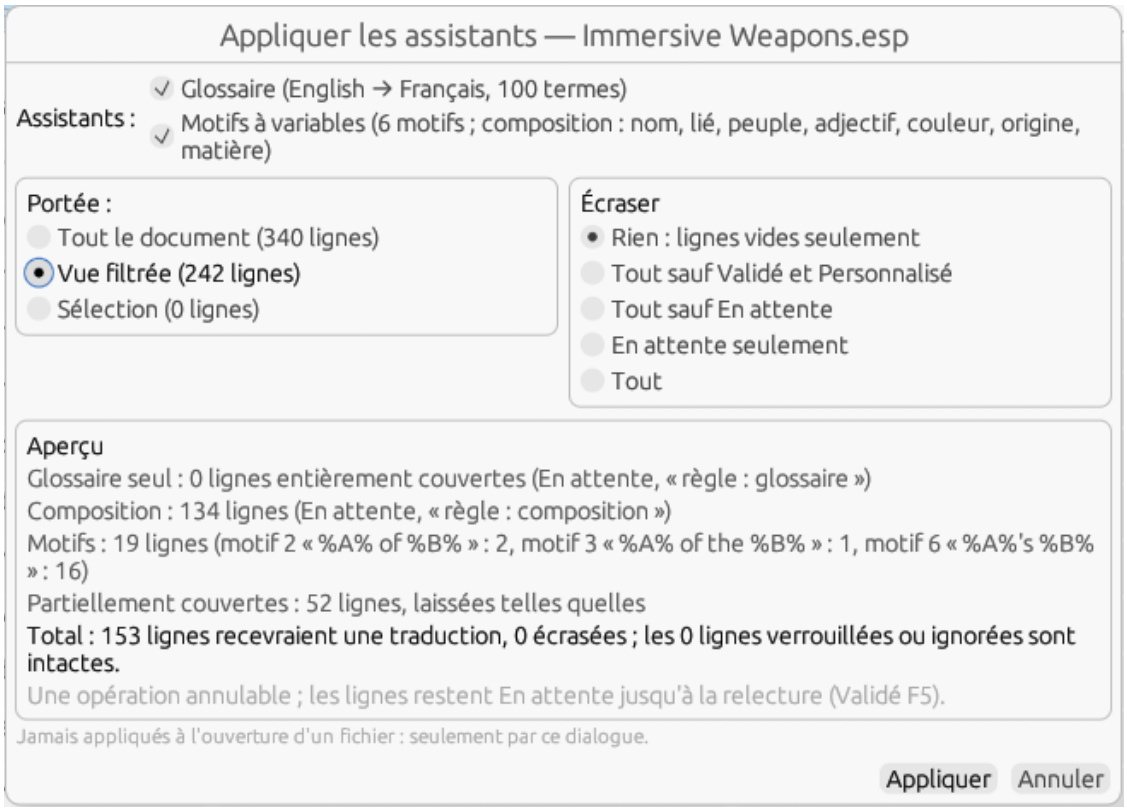


Figure 11.5 b — Apply the assistants on Immersive Weapons, view filtered on the WEAP, with a glossary of 100 terms: 134 lines composed, 19 by patterns 2, 3 and 6 (possessive), 52 partially covered left as they are.

Derivations. The derivations come after the glossary and the patterns, whose translations they take as parents: a spell the glossary has just translated serves at once. A derived object — scroll, tome, potion — takes its derivation rather than the generic pattern that would have translated it (“Scroll of %A%”): the derivation knows its parent, transforms it and follows it. The preview gives “Derivations: 326 lines (scroll: 136, tome: 190); parents without a translation: 0; parents not found: 1; ambiguous: 0”; a line whose parent is found without a translation, or is not found, is not touched. On Apocalypse - Magic of Skyrim, spells and effects translated by the shipped French translation, loaded as the mod dictionary, 326 of the 327 scrolls and tomes are derived, all identical to the human translation, “Invocation - ” prefixes stripped (“Scroll of Conjure Lich” → “Parchemin - Liche”); “Scroll of Conjure Dremora Herne” has no spell of that name. On Complete Alchemy and Cooking Overhaul, effects translated by the DSD translation and degrees in the game glossary, 345 potions, poisons and draughts are derived (“Weak Potion of Fortify One-handed” → “Potion de renfort d'armes à une main - Diluée”), 355 with the game base in the stack, which brings the game's effects (“Healing” → “Soin”); 256 are identical to the human translation, the others differ by the vocabulary the translator chose.

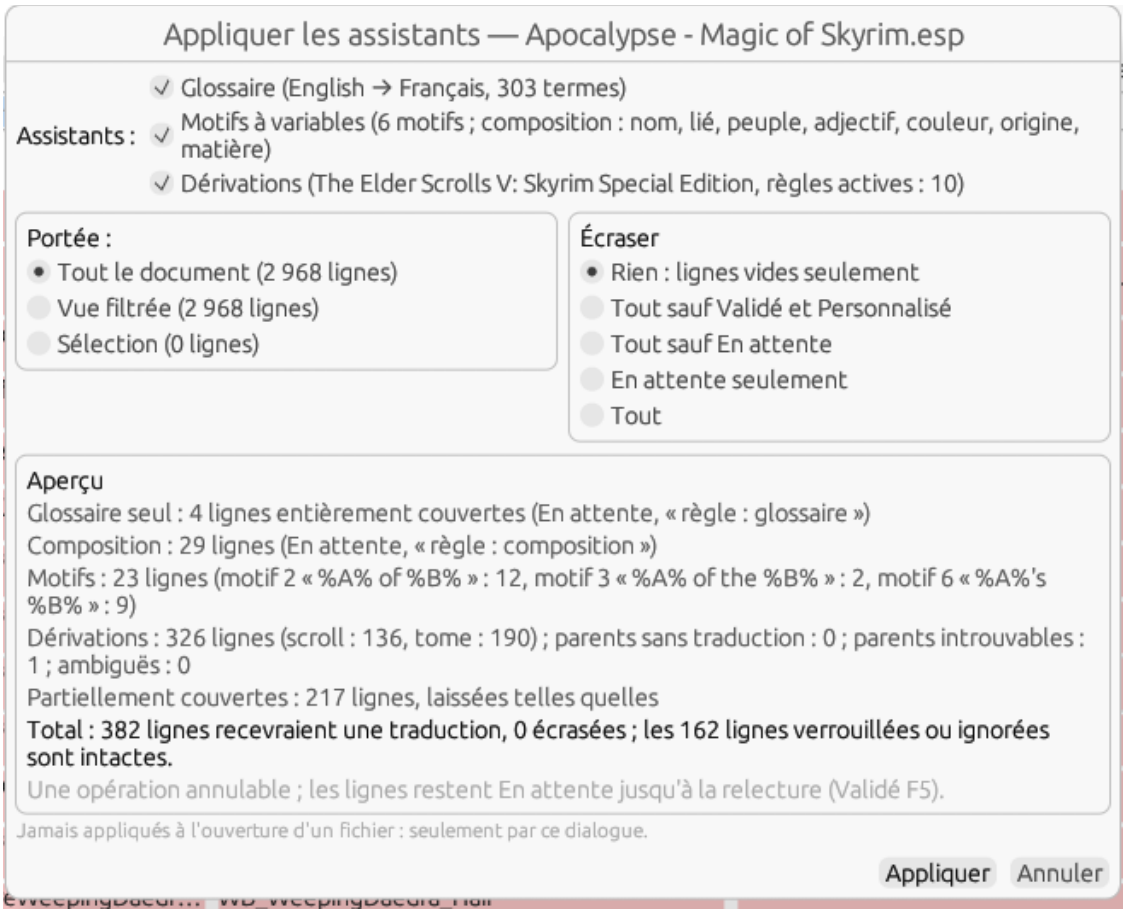


Figure 11.5 c — Apply the assistants on Apocalypse - Magic of Skyrim: 326 lines derived (136 scrolls, 190 tomes), one parent not found; the patterns only keep the lines that are not derived objects.

12. The Dictionaries menu

A **dictionary** is a file in the `.ximdb` format that associates a source text (original language) with its translation. XIMOD Translator uses it to automatically translate the entries of a mod that match the game's text or an existing translation. This menu gathers five families of operations: **converting** dictionaries between XIMOD and external tools; **building** a dictionary from a game, a mod, the STRINGS, a pair of plugins or a pair of locres files; **exporting** a game dictionary to STRINGS; **viewing** a dictionary read-only; and **managing** the list of installed games.

> **Note —** The source and target languages are **never** chosen in these dialogs: they always come from the *Source* and *Target* lists of the main window. They set the direction of the dictionaries created and the file suffix (e.g. `ENG-FRA`).

12.1 Before you start

The settings of this menu come from two places: the main window (languages and game) and *Options* ▸ *Settings* (tool and database folders). Some entries stay grayed out as long as the folder of the corresponding tool is not set:

- **Source / Target languages** — in the main window: they determine the direction and the naming of the dictionaries.
- *Game* — at the top of the main window: XIMOD cannot guess which game a mod belongs to (masters such as `Skyrim.esm` are shared). The chosen game loads its dictionary and sets the destination subfolder. **As long as the four lists are not set, the whole Dictionaries menu is grayed out.**
- **ESP-ESM Translator folder**: required for EET ↔ ximdb conversions.
- **xTranslator folder**: required for SST ↔ ximdb conversions.

Where the dictionaries are stored

Each game has its own subfolder under `assets\bases\jeux`. It contains the base game dictionary, named `<Game>_ENG-<LANG>.ximdb`, and a separate dictionary for each DLC, under its name. One file is created for each language pair (English being the source). For example:

- `assets\bases\jeux\SkyrimSE\SkyrimSE_ENG-FRA.ximdb` (base game)
- `assets\bases\jeux\SkyrimSE\Dawnguard_ENG-FRA.ximdb` (DLC)
- `assets\bases\jeux\SkyrimSE\SkyrimSE_ENG-DEU.ximdb` (another language pair)

Mod dictionaries, for their part, are stored in `assets\bases\mods\<Game>\`. The game subfolders are: `Fallout3`, `Fallout4`, `Fallout76`, `FalloutNV`, `Morrowind`, `Oblivion`, `OblivionRD`, `Skyrim`, `SkyrimSE`, `Starfield`. `OblivionRD` stands for Oblivion Remastered, kept separate from the original Oblivion. The list of DLCs per game is in `assets\data\GamesDLC.json`, which you can edit to add DLCs or Creation Club content.

12.2 Menu overview

Menu entry	Purpose
Convert an EET / SST to ximdb	Import a dictionary from an external tool into the ximod format (.ximdb).
Convert a ximdb to EET / SST	Export a ximod dictionary to the format of the external tool.
Build a dictionary from a game…	Compile into a ximdb the raw databases found in a game folder.
Build a dictionary from a mod…	Build a ximdb from mod files (EET/SST/DSD/JSON/XML).
Build a dictionary from the game STRINGS…	Pair source and target STRINGS into a game ximdb.
Build a dictionary from a pair of plugins…	Pair two plugins (same content, two languages).
Build a dictionary from a pair of locres files…	Oblivion Remastered: pair two locres exports (CSV/JSON).
Export a game dictionary to STRINGS…	Rewrite translated STRINGS from a ximdb.

Menu entry	Purpose
View a dictionary…	Browse a .ximdb (source → translation, with a filter). The only command of the menu available without a selected game.
Create a text-only dictionary from a file of pairs…	A “source<TAB>translation” file → a text-only .ximdb (§ 12.9).
Manage dictionaries…	The dictionary tree of the active document: roles, boxes, priority order, added files and folders (§ 12.8).
My personal dictionary… / Edit a dictionary…	The dictionary editor, on the personal dictionary of the pair or on a chosen .ximdb (§ 12.10).
Mod update…	The document compared with its mod dictionary: added, removed, changed; old translations taken back (§ 12.11).
Save the mod dictionary	Write the working memory of the active document now (§ 7.6), without waiting for a save or an export.
Detect installed games… / Add a game…	Search for or register the Bethesda games.

12.3 Converting between XIMOD and external tools

Four submenus, each with *Single file…* (one file) and *Batch…* (several files). **EET** conversions require the ESP-ESM Translator folder; **SST** conversions require the xTranslator folder.

Conversions concern only **mod dictionaries**: game dictionaries are built directly from the installed games (§ 12.4) or shipped with the application for the games that require external handling (Morrowind, Oblivion…). Each tool stores its mod dictionaries in one folder per game: `Bases Persos\<game>` for ESP-ESM Translator, `UserDictionaries\<game>` for xTranslator, `assets\bases\mods\<game>` for XIMOD Translator. It is this *<game>* folder that decides the destination.

Direction **EET/SST → ximdb**: the file picker opens in `Bases Persos` (EET) or `UserDictionaries` (SST). With *Single file…*, choose a file; with *Batch…*, choose either the root folder (all the *<game>* subfolders are processed) or a *<game>* folder. Each dictionary is written to `assets\bases\mods\<game>`, the mod name and the languages being derived from the file name. The *Convert to ximdb* dialog shows an *Example* of the result and asks for the game only if a file is not in a recognized *<game>* folder.

Direction **ximdb → EET/SST**: the file picker opens in `assets\bases\mods`; the converted file is written directly to the tool's `Bases Persos\<game>` or `UserDictionaries\<game>` folder, without further questions, under the name of the source dictionary. A `.ximdb` taken from outside a *<game>* folder is refused with a message.

Mapping of the *<game>* folders between ESP-ESM Translator and XIMOD Translator (xTranslator uses the same names as XIMOD):

ESP-ESM Translator	XIMOD Translator / xTranslator
Fallout 3	Fallout3
Fallout 4	Fallout4

ESP-ESM Translator	XIMOD Translator / xTranslator
Fallout 76	Fallout76
Fallout New Vegas	FalloutNV
Morrowind	Morrowind
Oblivion	Oblivion
Skyrim	Skyrim
Skyrim SE	SkyrimSE
Starfield	Starfield

> **Note —** If a conversion submenu is grayed out, the folder of the tool (ESP-ESM Translator or xTranslator) is not set in the Settings.

12.4 Building a dictionary

> **Note —** Because the subfolder follows the game of the active document (or the default game when no document is open), and not the actual content of the files, a warning *"Is this really a dictionary for <Game>?"* is shown when saving: *Validate* saves, *Cancel* abandons. This applies to the five "Build…" entries.
>
> **Note —** Timestamped copies — when a mod dictionary overwrites an existing file of the same name, the old one is first moved to the `backups/` subfolder under the name `<name>_<YYYYMMDD-HHMMSS>.ximdb`; the ten most recent copies are kept (number adjustable in *Settings* ▸ *Advanced*, 0 to disable). The `.ximdb` dictionaries are written in format 2, which can carry the status of each line; format 1 files are still read.

From a game…

Select a game folder; XIMOD compiles the raw databases it contains into a dictionary stored under `assets\bases\jeux\<Game>\`.

From a mod…

Select one or more mod files (`EET`, `SST`, `DSD`, `JSON`, `XML`). The dictionary is stored under `assets\bases\mods\<Game>\`, named after the first file.

From the game STRINGS…

Choose the source STRINGS (source language), then the target STRINGS (target language) — multiple selection is possible. The entries are paired by their identifier; only the pairs that are actually translated are kept.

From a pair of plugins…

Builds the dictionary from two identical plugins in two languages — typically for old games (Morrowind, Oblivion, Fallout 3/NV) whose text is in the ESM. The window offers two fields (*Source plugin* / *Target plugin*, with *Browse…*), a *Destination* choice (Game or Mod), and a reminder of the languages. The fields are remembered until the window is closed, and the window **stays open after each save** (you chain the master, then each DLC); it closes only with *Close*.

The file is named after the source plugin (e.g. `Anchorage.esm` → `Anchorage_ENG-FRA.ximdb`). Both plugins must have the **same file name**; a trailing language code (e.g. `MonMod_FR.esp` vs `MonMod.esp`) is ignored. If the names really differ, or if no entry matches, an alert is shown and nothing is saved.

From a pair of locres files (CSV/JSON) — Oblivion Remastered

For Oblivion Remastered, the game text is in `Game.locres` files (Unreal) packed in the `.pak` files; automatic generation and the plugin pair do not apply. First extract the text of each language with an external tool, in one of the two accepted formats:

- JSON from **FModel** (the simplest): export the `Game.locres` of one language; shape `{ namespace: { key: text } }`.
- CSV from **UEExtractor**: extract each language by targeting its folder, e.g. `--path=…/Localization/Game/fr/` (otherwise the languages merge and English wins).

Extract the source and the target in the **same format** so that the keys match (codes present: `de, en, es, fr, it, ja, pl, pt, zh-Hans`). In XIMOD, choose *OblivionRD* as the *Game*, set *Source*/*Target*, then *Build a dictionary from a pair of locres files…*: give the source file, then the target file, the destination, and *Build dictionary*. XIMOD pairs by key and saves only the translated entries; the file `OblivionRD_ENG-FRA.ximdb` is stored under `assets\bases\jeux\OblivionRD\`.

12.5 Exporting a game dictionary to STRINGS

Export a game dictionary to STRINGS… does the reverse of a generation: choose a `.ximdb`, then the source STRINGS folder, then an output folder. XIMOD rewrites translated STRINGS files from the dictionary.

12.6 Managing games

Detect installed games… runs a search on all the recognized platforms (§ 2.4: Steam, GOG and GOG Galaxy, Epic, Heroic, Lutris, Xbox, folder scan; when several find the same game, the most reliable source is kept), with a progress bar, shows a summary (game · source · path) and saves the paths in `Config.ini`. *Add a game…* registers a game folder manually, validated by the presence of its master file, then locked (kept by later detections).

12.7 Worked examples

Building the dictionary of an old game (Oblivion)

1. Set *Source* and *Target* (e.g. ENG → FRA), then choose *Oblivion* in the *Game* selector.
2. Menu *Dictionaries* ▸ *Build a dictionary from a pair of plugins…*, *Destination* set to *Game*.
3. Give the English `Oblivion.esm` (source), then the French `Oblivion.esm` (target).
4. *Build dictionary*: `Oblivion_ENG-FRA.ximdb` appears in `assets\bases\jeux\Oblivion\`. Repeat for each language.

Converting an EET dictionary to ximdb

Single file… converts a single `.eet`; *Batch…* takes a folder and gathers all the `.eet` files it contains (subfolders included).

1. Check the ESP-ESM Translator folder in the Settings, and set *Source* / *Target*.
2. Menu *Dictionaries* ▸ *Convert an EET to ximdb*, then *Single file…* or *Batch…*.
3. Choose the `.eet` file in `Bases Persos\<game>` (Single file), or the `Bases Persos` / `Bases Persos\<game>` folder (Batch).
4. In *Convert to ximdb*, check the *Example* (the destination *<game>* folder is derived), then *Convert*.

12.8 Managing dictionaries: the tree

Dictionaries ▸ *Manage dictionaries…* opens, for the active document, the tree of its dictionaries: six roles — Mod, Personal, Text only, Game, External, Backups — each with its files, a box per role and per file, and an order that is the priority order (§ 7.3). An unchecked role disables its files without losing their boxes. *Move up* and *Move down* move the selected role or file; *Add a file…* accepts a .ximdb — or a raw EET, SST, DSD, JSON, XML database, read into memory — from any folder, *Add a folder…* takes all the .ximdb files of the target language from it, without copying anything; these additions go into External and are the only ones that *Remove* removes (the files of the XIMOD folders can only be unchecked). A .ximdb that declares a language other than the target is reported and ignored. *View…* (or a double-click) opens the file in the viewer (chapter 13).

The detail line describes the selected file: role, number of lines, format, date, and the number of document lines that owe their translation to it. *Save* stores the tree for this game and this language pair (`[Dictionaries] Dicts_<Game>_<SRC>-<DST>`): a file that has appeared since then takes its default place, checked (except in Backups), and a file that has disappeared is removed silently. *Apply to the document* stores the tree, then replays the stack on the empty lines, as one undoable operation. *Reset the order* returns to the initial order, everything checked except the backups, additions forgotten (to be saved). Closing with changes asks: Save, Discard the changes, Cancel. The box of the personal dictionary is the same as the one in the context menu of the tab (§ 7.3). The window geometry is remembered (`[Window] DictsX/Y/W/H`).

12.9 Text-only databases

A **text-only** database is a dictionary whose lines have no record, no field, no FormID and no EDID: text → text pairs (“of Burning” → “de feu”). The .ximdb format allows this since its version 3; a file whose lines are all of this type takes the Text only role in the tree (§ 12.8), whatever its folder; a mixed file keeps the role of its folder and its unsigned pairs are still used. In each dictionary, after the exact match and the match by text of the same field, a text-only pair recognizes a line whose source text is identical (apart from leading and trailing spaces), whatever the field, and gives it By similarity; the provenance says “text only”. They are also the material for instant translation (B-14, upcoming). *Dictionaries* ▸ *Create a text-only dictionary from a file of pairs…* reads a UTF-8 text file, one “source`<TAB>`translation” pair per line (empty lines and lines starting with # are ignored), and writes it to `mods/<game>/<file>_<SRC>-<DST>.ximdb` after the game is confirmed, like the other builders. The personal dictionary (B-04, upcoming) will be a text-only database.

12.10 My personal dictionary and the dictionary editor

The personal dictionary of the language pair — `dict/personal_<SRC>-<DST>.ximdb` in the configuration folder — is your own terminology, which follows you from mod to mod (“Dwemer”, “Dovahkiin”, the ranks of a faction). It is fed in three ways. *Ctrl+M* (line menu ▸ Add to my personal dictionary, in the grid on the selection or in the edit window on the line) sends it the source → translation pair of each translated line, as text only by default (§ 12.9): the terminology then applies to any field; the *Ctrl+M keeps the field (REC:FLD)* option of Settings ▸ Advanced (`[Personal] KeepField`) stores the pair with its signature. A pair already known with the same translation is left as it is; with another translation, a dialog asks Replace, Keep or Cancel, with the “Remember this choice for the session” box; beyond twenty lines, a confirmation gives the number. The file is written at once, with a timestamped copy, and the toast says “3 pairs added (1 replaced)”. Second way, the *Fill my personal dictionary at each save* option (`[Personal] AutoFill`, unchecked at first): at each successful Save as… and export, the Validated and Custom lines whose pair is not yet known go into it as text only, and the status bar says so; Pending, By similarity and Ignored lines never go into it. Third way, the editor.

Dictionaries ▸ *My personal dictionary…* opens the dictionary editor on the personal dictionary of the current pair (created empty if it does not exist); *Dictionaries* ▸ *Edit a dictionary…* and the *Edit…* button of the tree (§ 12.8) open it on any .ximdb. An independent window, only one at a time: a filter (substring in the source or the translation, applied a quarter of a second after typing), a Source / Translation / Field / Status table that can be sorted by clicking the header, and a Source, Field and Translation editing area. A click on a pair loads it into the area; *Add* creates a pair, *Replace* modifies the selected pair (source and field included), *Delete* removes it; Enter means Replace when a pair is selected, Add otherwise; Del in the table deletes. An empty field makes a text-only pair, “REC:FLD” a signed pair. Everything stays in memory until *Save* (timestamped copy, dictionaries of the open documents reloaded); closing with changes asks. *New…* creates an empty text-only dictionary for the pair in `mods/<game>` or in the chosen folder. A game dictionary shipped with the software opens read-only; *Save as…* makes a copy of it in your folders. On a mod dictionary that has old data (§ 7.6), the *Old data (N)* box switches the table to it, with the disappearance date in the Status column; *Delete* removes one entry, *Put back among the current rows* returns it to the current lines. The geometry is remembered (`[Window] DictEditX/Y/W/H`).

12.11 Mod update

Dictionaries ▸ *Mod update…* compares the active document with its mod dictionary (current lines and old data): at the top, the date the dictionary was written, its number of lines and that of the open file; then the summary — **added** (unknown to the dictionary, record included), **removed** (in the dictionary, no longer in the file: the current lines will move to the old data at the next write, the old ones are already there with their date), **changed** (same record, field, FormID and EDID, different source text) and unchanged — and three tabs that list them; the list of changed lines gives the old source, the new one and the old translation, the differences between the two sources highlighted character by character (green on the old one, red on the new one, § 8.5), and a double-click opens the line in the edit window. *Take the N old translations* gives the changed lines whose translation is empty or by similarity their old translation, with the Pending status, as one undoable operation; the lines already Validated, Pending or Custom are kept and counted. *Forget the removed…* empties the old data section after confirmation (the

previous file goes to `backups/`). Without a mod dictionary, the window offers to write it. When you open a file some of whose lines have changed since the dictionary, the status bar says so without applying anything. The geometry is remembered (`[Window] UpdateX/Y/W/H`).

Lines whose source only changed by a number, punctuation or case do not wait for this window: at opening, normalized matching (§ 7.3) finds them in the mod dictionary, translation adapted, with the Pending status, and the *Provenance* ▸ *Normalized* filter gathers them for review.

Derived lines (§ 15.7) follow their parent: when the update takes back the old translation of a spell or an effect, or when you correct it, the scrolls, tomes and potions derived from it that are still Pending are recomputed in the same operation.

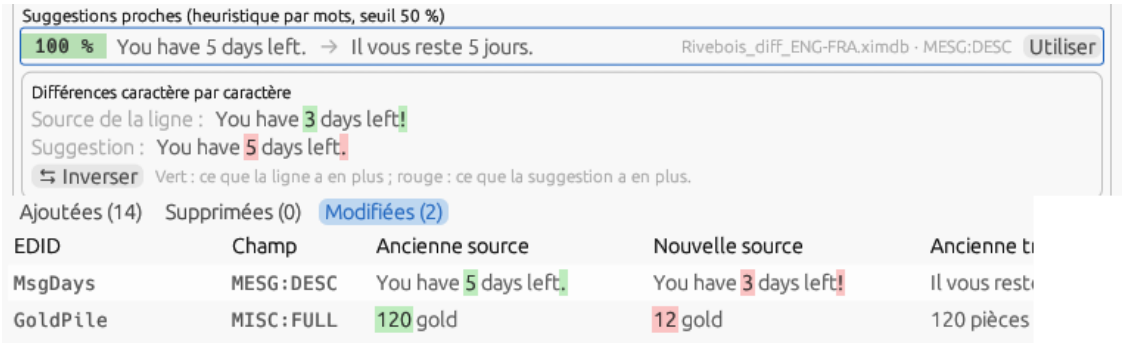


Figure 12.11 — The character-by-character differences: in the Vocabulary tab between the source of the line and the suggestion, and in the Mod update window between the old and the new source (French interface).

12.12 Compare two dictionaries

Dictionaries ▸ *Compare two dictionaries…* puts side by side two translations of the same mod — two DSD files, a ximdb and an SST, an open document and the French plugin. *A* and *B*: an open document chosen in the list, or a file (ximdb, DSD, pivot, SST, EET, XML, translated plugin — a plugin only has its texts: texts are then compared, not translations). *Pairing*: *Record and text* (record, field, FormID without the load index or EDID, rank — and the source when both sides are translations; this is the default, and it becomes text only when a side has no signatures) or *Text only*; *Ignore case and edge spaces* (checked) applies to the sources and to the verdict. A source present several times on one side counts once, with all its distinct translations.

The result. *Compare* gives the summary — lines of A and of B, common sources, identical, divergent, only in A, only in B — and four tabs, each a table (record, EDID, source, translation A, translation B). A double click on a line of an open document selects it in its grid. *Export the report* writes the CSV whose path is next to the button (UTF-8 with BOM, semicolon; `category;record;field;edid;source;translation_a;translation_b;file_a;file_b`, categories same, diff, only_a, only_b). *Merge into a dictionary* writes a XIMOD dictionary (format 4) of all sources: identical ones as they are, divergent ones with the translation of the chosen side (*Divergent: take A / B*) and the ambiguous mark "|amb:2" (§ 11.4) to be settled in the grid, those of one side with theirs; all Pending, except those A and B both give Validated. The file proposed is `mods/<game>/<name of A>_<pair>.ximdb`: under the plugin's name, it becomes the mod dictionary and its divergent lines open marked ≠.

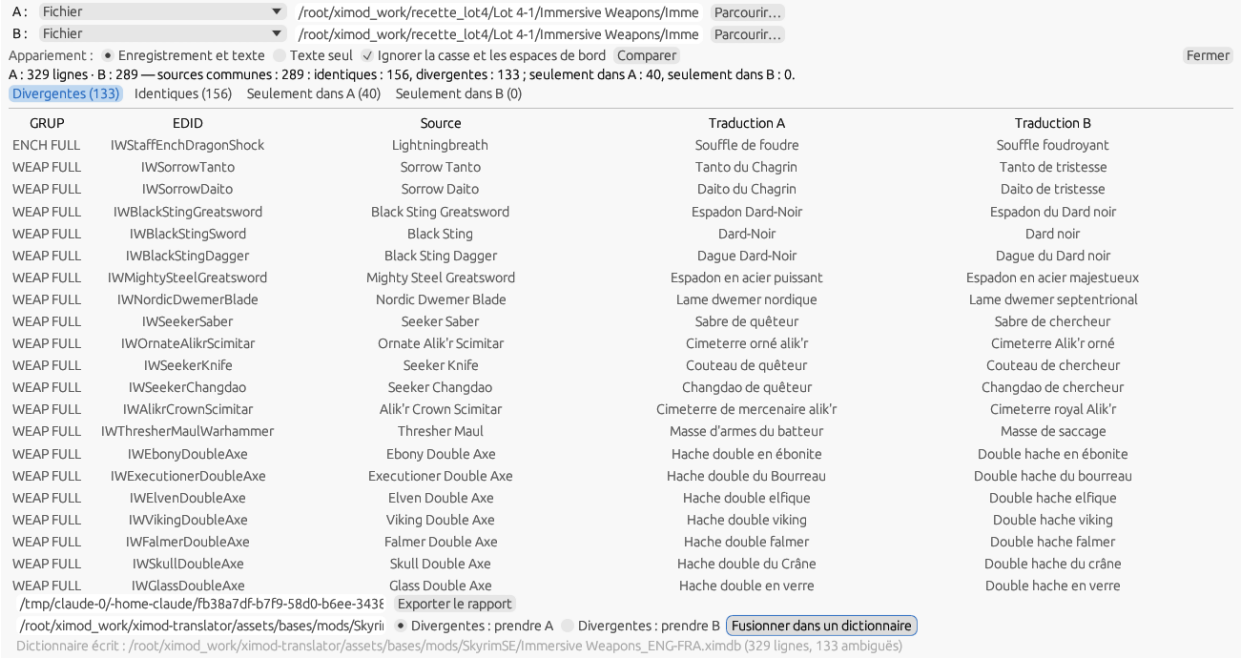


Figure 12.12 — Comparing the two French DSD translations of Immersive Weapons: the summary, the divergent lines, the report and the merge.

13. The dictionary viewer

Menu *Dictionaries* ▸ *View a dictionary…*. It opens in a separate window whose size and position are remembered.

- Open a `.ximdb` with *Open…* or by dragging it onto the window; *Close* unloads the file.
- A *Filter* field narrows the display; a counter shows the number of lines.
- Two equal columns, *Source* and *Translation*, with long texts wrapped.
- A **striping** (alternating rows, as in a spreadsheet) whose color is set with *Options* ▸ *Dictionary viewer stripe color…*.

14. The spell checker

14.1 Installing a dictionary

The spell checker relies on Hunspell and on a pair of `.aff` / `.dic` files for the *Target* language (e.g. `fr_FR.aff` / `fr_FR.dic`), to be placed in the folder shown by the window.

14.2 Opening and using the spell checker

Menu *Options* ▸ *Spell checker…*. For each error, you can apply a suggestion, *Ignore* the word for the session, or add it to the personal dictionary (*+ Dictionary*). *Recheck* runs the analysis again; *Reload* reloads the dictionary.

14.3 The personal dictionary

Added words (proper names, game terms…) are kept and are no longer flagged as errors in later checks.

15. The explorers (Tools menu)

The Tools menu also opens Find and replace (§ 8.6), Quality checks (§ 8.7) and Dialogues (§ 8.8). The three explorers open in separate windows, positioned near the main window with a slight offset; their size and position are remembered, and all three can be open at the same time.

- *ESP/ESM/ESL Explorer*: browses the structure of a plugin (GRUP, records, subrecords).
- *BSA/BA2 Explorer*: lists the contents of a Bethesda archive. You can show only the *Translatable only* entries, *Extract…* a file, *Open in translator*, or *Open associated plugin* (resolves its STRINGS from the archive).
- *SWF Explorer*: explores Flash interface resources (SWF).

> **Note —** An archive can also be opened directly by dragging it onto the main window (`.bsa` / `.ba2`).

15.1 Tools ▸ Data ▸ Translatable fields per game…

What is extracted as a translatable line is described, game by game, in a shipped data file (`assets/data/fields/<Game>.json`) and no longer in the program. The editor shows the effective list for the chosen game — one line per `RECORD:FIELD` signature (or per EDID or text pattern), with the mode (*Yes* = extracted and editable, *Locked* = extracted but with the Ignored status by default, *Never shown in game* = extracted with the Ignored status, in red, hidden by default and not editable — plugin header, editor-only names (§ 9), *No* = never extracted), the non-empty constraint, the maximum length, whether line breaks are allowed (Yes, No, or "—" as long as this is not known: the edit window then lets Enter through), an optional context rule ("EDID starts with s" for game settings, for example) and a comment. Grayed lines come from the parent game (Skyrim SE inherits from Skyrim, Fallout 76 from Fallout 4, Oblivion Remastered from Oblivion); editing a line makes it an entry specific to the game.

You edit the selected line in the bottom panel; validation is continuous (signature in the AAAA:BBBB format, uniqueness, well-formed rule) and an invalid file is not saved. *Save* writes only your differences from the shipped version, to `data/fields/<Game>.json` in the configuration folder — the shipped file is never modified — and the changes apply the next time a file is loaded. *Restore shipped version* deletes your file. An unreadable user file is reported at startup and the shipped version is used.

Engine tags. The `tags` key of the file, at the head of the field list, lists the tag names compared by the Aliases and numbers tab (§ 8.5); a name ending in `*` covers any tag that begins that way (`Alias*` covers `<Alias=…>` and `<Alias.Name=…>`). An empty list compares all `<…>` tags; the list of a child game replaces the parent's. It is edited in the file; the editor does not show it yet.

Shipped values. The tables specify line breaks for the fields whose behavior is certain: forbidden in names (FULL, Morrowind FNAM, short names, button and objective labels, activation verbs), allowed in descriptions (DESC, book text, effect descriptions); the others remain "—". Length limits are set only when they are those of the engine and not of an editor: Morrowind names, at 31 characters (fixed game buffer). The limits of the Creation Kit editor (149 characters for a dialogue response, 80 for a topic) do not

constrain the game: they appear in the rule comment, for information. Adjust these values in the editor if your experience differs.

List column. For games with STRINGS, the *List* column says, read-only, which file (STRINGS, DLSTRINGS, ILSTRINGS) the field goes to in a localized plugin, from the table of localizable fields (§ 19.4); a dash for a field the engine does not localize.

15.2 Tools ▸ Data ▸ Text formats…

Text file formats (§ 7.7) are described in `data/text_formats.json` in the configuration folder, initialized from `assets/data/text_formats.json`. Each format has an identifier, a name, **name patterns** ("Interface/Translations/*_*.txt", "translate_*.txt", "*.ini"; a pattern containing "/" is compared with the end of the path, the others with the file name), an **encoding** (auto, utf-8, utf-16le, utf-16be, windows-1252; auto reads the byte order mark, then tries UTF-8, then Windows-1252 — a mark, when present, always wins), **line endings** (keep, crlf, lf), the **line expression** (Rust syntax) with the named groups `key` and `value`, an **escaping** (none, json, c: "\"" and the line breaks of the value), the **Skip empty values** option and **Language in the name** (suffix: `<base>_<language>`; none). Shipped: MCM / translate, INI, flat JSON (one ""key": "value"" pair per line), CSV key;value.

The editor lists the shipped formats (padlock, read-only — *Duplicate* makes an editable copy) and yours; *New* creates a blank format, *Delete* removes the selected format. *Try on a file…* applies the expression to a file and reports the number of lines recognized out of the total, with the first three pairs. *Save* rejects an invalid expression (with the error message) or one without the key and value groups, and writes only your formats. When a file is written, the value is replaced in place in the recognized line — everything around it ("key = ", quotes, comma) is kept — which makes a write template unnecessary. The window remembers its geometry (`[Window] FormatsWin*`).

Structured formats. The *Type* of a format is *By line* (the expression above), *Structured JSON*, *Structured XML* or *CSV with columns* (`kind`: line, json, xml, csv; the file is in format 2, a file of the first version is read and written again without loss). A structured format replaces the expression with a **table of fields** — Texts, Identifier, Record, Field, Exclude, Translation —, each with its resolution and value: *path* (JSON: `$..text`, `.key`, `[n]`, `[*]`, `..` recursive; XML: `/a/b`, `//b`, `@attr`, `text()`, `[@k='v']`; several paths separated by "|"), *property* (of the node or of the nearest enclosing object that has it, "else automatic"), *column* (number or name, CSV), *fixed value*, *automatic* (the path of the node), *property name*, *regex* (Exclude: a matching value is left out) and, for the translation, *copy of the file* or a sibling property / a column of the file itself. The *Extraction* of the Texts is a regex whose (`?P<text>…`) group is the only part translated. CSV: separator (empty: guessed) and header row. Also shipped: MCM Helper (config.json), SKSE JSON (array id / text), XML (texts and attributes), CSV with columns (id, source, translation).

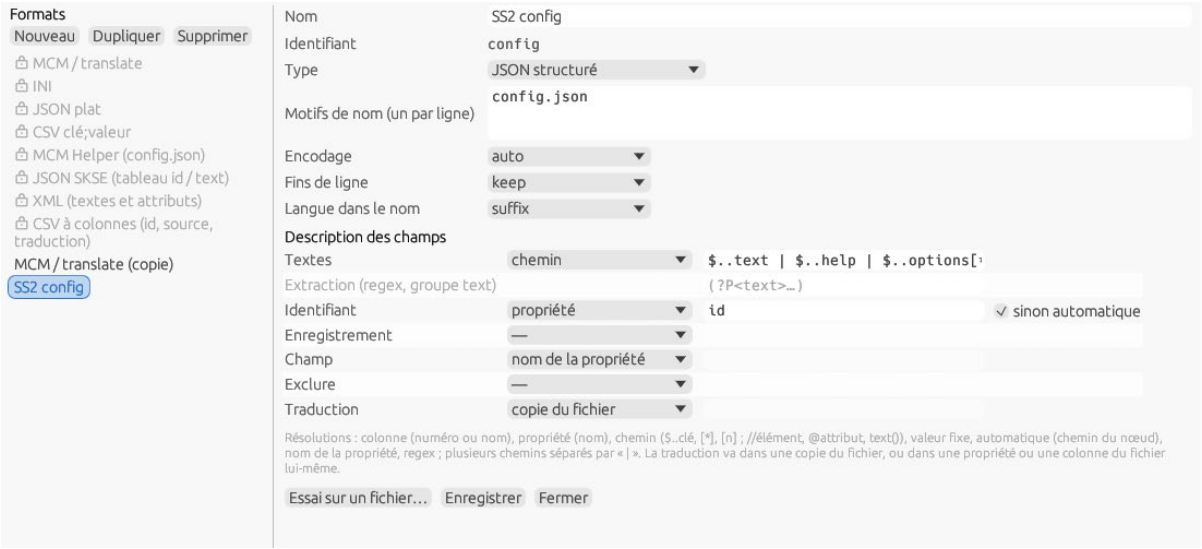


Figure 15.2 — Text formats: a structured JSON format created with New format… from Import with a format for the config.json of Sim Settlements 2 — the type, the pattern, the table of fields.

Try on a file… on a structured format counts the lines recognized and the values left out and shows the first line (“Lines recognised: 550; values left out: 1089; first: “About” (text, content[2])”). *Save* checks the paths, the regex (with its text group when it extracts), the presence of the Texts field, the agreement of the type with the patterns and, after a trial, that the named columns exist; the format is taken into account without a restart.

15.3 Tools ▸ Translate a FOMOD…

FOMOD installers — the ones XIMOD Architect builds — carry texts that the player reads in Mod Organizer 2 or Vortex: the name and description of the mod (`fomod/info.xml`), the module name, the names of the steps, groups and options, and the option descriptions (`fomod/ModuleConfig.xml`). This window translates them; it is the same as the one in XIMOD Architect, built on the library shared by both applications, and it works on the same file: a translation started in one reopens in the other.

Opening. *Open a mod folder…* accepts the mod folder or its `fomod` subfolder; *Open a mod archive…* accepts a zip or a 7z, of which only the `fomod` folder is extracted (into the configuration folder) — the “full mod” package is then not possible. Dropping a folder or an archive onto the main window also opens this window, as does `--fomod <folder>` on the command line. The source and target languages are chosen before loading and are fixed afterwards; the translator name is remembered (`[General] Translator`). The work is saved in `fomod/translations/<mod>.<language>.ximod-translation` (Ctrl+S): reopening the same FOMOD in the same language finds it again, and *Update from folder* merges a new version of the mod with the translation (new, changed, moved, removed texts).

The table. One line per text: number, status (○ to translate, ✓ translated, ◐ from memory, ~ to review, ✕ obsolete, 🔒 locked), context (step › group › option), source, translation editable in place for names, and ⚠ when a check has something to report. The *All / Untranslated / To review / With problems / Locked* filters, *Unique texts* (one line per distinct text; what you type propagates to identical ones), the field type and the search narrow the list; ↑ ↓ PgUp PgDn Home End move the line when you are not typing, *Next*

untranslated (Ctrl+Enter) jumps to the next one to do. The bottom panel shows the chosen line: source and translation side by side (descriptions are translated there, over several lines), *Clear*, *Copy source*, *Translate online* (Ctrl+G: the configured online service puts the translation in the target, to be reviewed; without a service, Google Translate in the browser, to be copied back — § 8.10), *Do not translate* (lock), the memory suggestion, the counter, the detected problems (missing tag, empty text, line break in a name, abnormal length, inconsistent duplicate…) and a note.

Helpers. *Fill from memory* proposes the translations already saved for the same texts (the memory learns at each save, per language pair); *Propagate to identical* copies the line's translation to all identical texts; *Export CSV… / Import CSV…* for a spreadsheet or an external translator; *Glossary* enforces the translation of a term, or keeping it as is, and flags deviations; *Copy the source list* and *Paste translations…* do the round trip with the AI of your choice as a numbered list ("translate this list into French, keeping the numbers").

Exporting. *Export the translated files* writes the translated XML files without losing anything from the originals (comments, encoding, layout, unknown elements) according to the chosen mode: into a `fomod_<lang>` folder next to it, in place, or as a zip / 7z package ready for Nexus (with a readme, the full mod if the whole folder is available, and a configurable name). *Keep the original order* keeps the explicit order of the steps. A blocking problem prevents the export and selects the *With problems* filter. Closing the window with unsaved changes shows a Save / Discard / Cancel row in the window; the ✕ button in the title bar is the only way to close it (not Esc, so that no input is lost).

15.4 Tools ▸ Package a translation…

Once the translation is exported (§ 11.2), this window builds the folder and the `.zip` archive that Mod Organizer 2 and Vortex install, according to the game family (§ 19): fill in the game and the language (prefilled from the toolbar), the mod name as published, the version, your translator name, the website and the output folder, then add the files. The preview shows the package tree and the notes to read before publishing; a file with no known place for this game is flagged and left out.

File	Place in the package
Plugin (.esp / .esm / .esl)	At the root (Data folder). An override patch must stay below the original mod in the load order.
STRINGS (Skyrim, Skyrim SE, Fallout 4, Fallout 76, Starfield)	Strings/; exact file names, language included. Starfield: loose files must be enabled in StarfieldCustom.ini.
DSD (.json, Skyrim SE)	SKSE/Plugins/DynamicStringDistributor/<target plugin>/; requires SKSE and Dynamic String Distributor.
BSA / BA2	At the root, named after the plugin that loads it.
Morrowind	At the root of Data Files; the translated plugin replaces the original.
Oblivion Remastered (PAK / UCAS / UTOC)	OblivionRemastered/Content/Paks/~mods/; the three files go together. A plugin goes under OblivionRemastered/Content/Dev/ObvData/Data/.

File	Place in the package
Manifest (manifest.json)	fomod/manifest.json: the source plugin and its CRC-32, the mod dictionary, the stack, the outputs (§ 18.4); written by ximod-cli manifest.

The package is named “<mod> - <LANGUAGE> - <version>”; *Create the folder and the archive* writes both to the output folder (an existing folder with the same name is recreated) and, if the box is checked, a `fomod/info.xml` with the name, the translator, the version, the website and the description (by default “<language> translation of <mod>”). *Open the folder* shows the result.

The same packaging exists on the command line (`ximod-cli package`, § 18.2); a `manifest.json` among the files — the one `ximod-cli manifest` produces — goes into the package as `fomod/manifest.json`: it says for which version of the mod the translation was made (§ 18.4).

15.5 Tools ▸ Translate the interface…

This independent window lets you translate or complete a language of the interface (§ 2.6) without editing files by hand or knowing the Fluent format, then send it to the project. At the top: the **flag** opens the countries window, and the country offers its languages in *Language to translate*; only a language with an ISO 639-3 code in `Languages.json` can be translated, and a language without a folder is created when saving. *Displayed language* chooses the language shown alongside, any existing language — changing it keeps what you typed. Then come the country endonym (“Bundesrepublik Deutschland”, “Schweiz” depending on the flag), the language endonym (“Deutsch”), the author and the *Font*: *Browse…* picks it inside `assets/fonts` (a font stored elsewhere is refused: copy it into that folder first), *Google Fonts…* opens the site to download one; the font is used at once for the endonym fields, the Translation column and the preview.

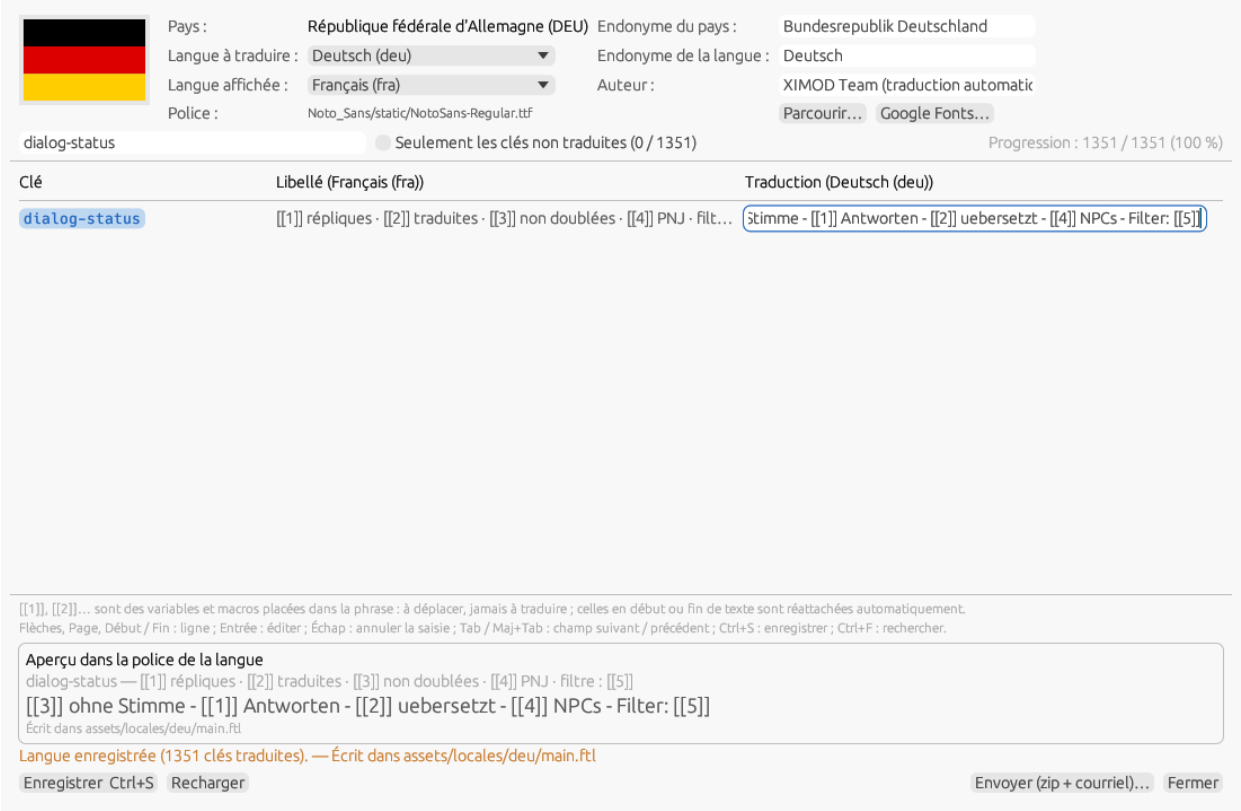


Figure 15.5 — Translate the interface: German, with French alongside; the dialog-status key has just been edited and saved.

The table has three columns: *Key*, *Label* in the displayed language (the tooltip gives the full text) and *Translation*, the only editable one. The search field filters on the key, the label and the translation; *Only untranslated keys* (with their number) keeps the empty translations only — a row you fill in stays shown until the filter changes again; the progress ("1,028 / 1,240 (83 %)") is on the right. Fluent variables (`{ $count }`) and macros (`$MODNAME$`) are never typed: at the start or end of the text they are hidden and re-attached on save with their separators; in the middle of the text they become markers `[[1]]`, `[[2]]`… that you move within your sentence. A lost, unknown or duplicated marker turns the row red (the tooltip says which): the key will then take the label on save. Untranslatable keys (`app-title` and the `# @untranslatable` list of the reference file) and keys made of variables only are hidden; plural or multi-line messages are shown as they are, in italics, and cannot be edited here. The preview, at the bottom, shows the whole current row in the font of the language.

Keyboard. Arrows, Page Up / Down, Home / End change the row; Enter edits the row's translation, Esc cancels the current edit; Tab / Shift+Tab go to the next / previous row (to the untranslated rows when the filter is checked); Ctrl+S saves; Ctrl+F goes to the search.

Save writes `assets/locales/<code>/main.ftl` in the order of the reference file (English), with its blank lines and comments — those of the language when its file follows the reference —, headed by `# @language`, `# @font`, `# @langname`, `# @author`; a key left empty takes the label, so the file is always complete. The language endonym and the font go into `Languages.json`, the country endonym into `Countries.json`, only if you changed them; the country endonym is also recorded in `countryEndonyms.tsv` next to the file. When `assets/` cannot be written (system installation), the save

goes to `<configuration folder>/locales/<code>/main.ftl`, which the application reads before the one in `assets/`; the “Written to …” line of the preview always says where. The language appears at once in Settings ▸ Interface language and, if it is the current one, the interface reloads. *Reload* reads the files again, discarding unsaved changes after confirmation; closing the window with changes also asks for confirmation.

Send (zip + e-mail)… builds `ximod-translator_<code>.zip` in the configuration folder — `locales/<code>/main.ftl` and `countryEndonyms.tsv`, never the ISO data files —, opens that folder and a pre-filled e-mail (project recipient, subject “XIMOD Translator — traduction <language>”, author and path of the archive): attach the archive and send; nothing leaves without you. The window remembers its geometry (`[Window] TrWin*`).

15.6 Tools ▸ Data ▸ Replacement matrices…

A **matrix** is a named sequence of replacement rules, applied in order in one go from Find and replace (§ 8.6): French typography (narrow no-break space before “!”, “?”, “;”, no-break space before “:”, ellipsis), « » quotes, typographic apostrophes… Matrices are filed by target language, one JSON file per language: `assets/data/matrices/<code>.json` for the shipped versions (`fra`, `deu`, `spa`, `eng`) and `all.json` for the matrices common to all languages (extra spaces). A copy of the same name in the configuration's `data/matrices/` folder replaces the shipped version; that is the one the window writes.

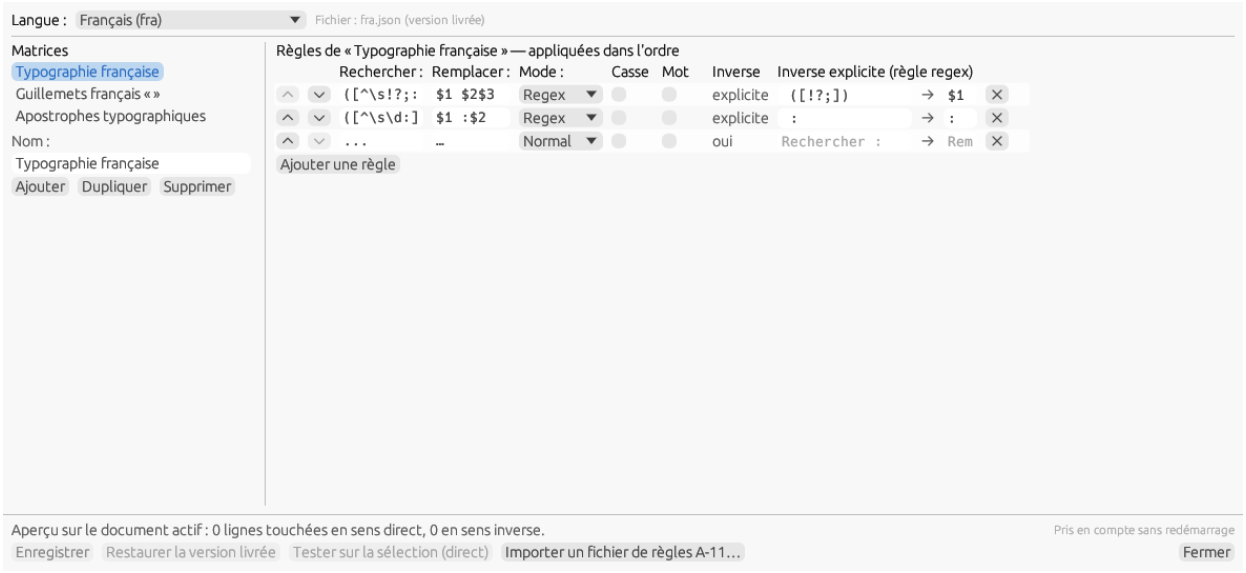


Figure 15.6 — The French matrices: the list on the left, the rules of Typographie française on the right with their inverse, the preview on the active document.

At the top, *Language* picks the file — *All languages (common)* first, then the shipped or already copied languages, and the active document's target language — and the *File* line says which one is read: “user copy”, “shipped version” or “no file for this language”. On the left, the list of matrices, the *Name* of the chosen matrix and *Add* / *Duplicate* / *Delete*. On the right, its rules, in the order they apply: ▲ ▼ to move them, *Find*, *Replace*, the mode (Normal, Extended, Regex), *Case*, *Word*, then the *Inverse* — “yes” for a Normal or Extended rule, which inverts by swapping its two texts (unless its replacement is empty),

“explicit” when the rule carries its own inverse, “none” otherwise; a regex rule gets its inverse in the two *Explicit inverse (regex rule)* fields. ✕ deletes the rule, *Add a rule* appends one.

At the bottom, the preview counts the lines of the active document the matrix would touch in each direction; an invalid expression shows in red with its rule number. *Test on the selection (direct)* applies the matrix to the lines selected in the grid, as an undoable operation. *Save* validates the whole file (name present and unique within the language, non-empty pattern, compilable expressions) — an invalid file is never written — then writes it to the user copy; it also checks that each matrix, applied twice to the document's translations, changes nothing more, and warns otherwise. Saved matrices are *Applied without restarting* in Find and replace. *Restore the shipped version* deletes the language's user copy. *Import an A-11 rules file…* reads a former `{ "format": 1, "rules": [ … ] }` file and turns it into a matrix of the same name, to be saved to keep it. Closing with unsaved changes asks to save them or *Discard* them. The window remembers its geometry (`[Window] MatricesX/Y/W/H`, 1180 × 540 by default). A user copy that has become unreadable (edited by hand) is ignored in favour of the shipped version, and the window says so.

15.7 Tools ▸ Data ▸ Glossary, patterns and derivations…

An equipment mod names its items by composition — “Elven Boots”, “Iron Sword”, “Tribunal Heavy Boots White”: hundreds of names made of a dozen materials and a dozen types. The **glossary** translates them term by term. There is one per language pair, shared by the games: `assets/data/assist/glossary_<SRC>-<DST>.json` (shipped: `glossary_ENG-FRA.json`, 290 terms in eight groups — materials, colours, races and peoples, titles and trades, places and factions, armours, weapons, modifiers —, and `glossary_ENG-ZHO.json`, 40 terms taken from the Chinese translation of Immersive Weapon Integration); a copy of the same name in `data/assist/` wins, and it is the one the window writes. A glossary per game, `glossary_<Game>_<SRC>-<DST>.json` (for instance `glossary_Oblivion_ENG-FRA.json`), optional, is added to the pair's for the documents of that game and its terms come first.

Term. Each term has its *Original* (one or more words), its *Translated* (empty: the word disappears, like “Light”), its *Class* — noun, bound (“d'apparat”, “à pointes”), people (adjective of a people or a place, “nordique”, “elfique”, which French places next to the noun), adjective, colour, material (“en acier”), origin (“du Tribunal”, “de chevalier”) or — (no class, left in place) —, *Whole word* (checked: “Iron” does not take “Ironhand”; unchecked, the term may cover a piece of a word), the *Position* (anywhere, start, end: a start “Iron” does not take “Sword of Iron”), the *Case* (checked: the term matches and translates as is, for proper names and acronyms; unchecked, case is ignored and the translation takes a capital when it opens a text that has one), the *Gender* of a noun (m., f., m. pl., f. pl.) and, for an adjective or a colour, its four *Forms*. A term of several words wins over its words (“Heavy Boots” → “Solerets”, rather than “Heavy” + “Boots”); at equal length, the first in the files.

Composition. French puts the noun first and orders the other terms by nature. The pair's `patterns_<SRC>-<DST>.json` file gives the order of the classes in the target, the separator and a maximum length (`"composition": { "order": ["noun", "bound", "people", "adj", "color", "origin", "material"], "joiner": " ", "maxLen": 60 }` for ENG-FRA, the origin before the material as in the French translation of Immersive Armors; empty order and empty separator for ENG-

ZHO, which keeps the source order). The terms are ranked by class, in the source order within a class; the noun gives the gender and number that adjectives and colours take; two consecutive materials are merged (“en os de dragon et ébonite”, key `merge`); the “,” and “-” separators of a variant (“Fur Hood - Black”) disappear. Without a term of class noun, the text is not composed: the terms stay in the source order. Examples: “Antique Honed Silver Sword” → “Épée antique affûtée en argent”; “Nord Steel Warhammer” → “诺德钢制战锤”.

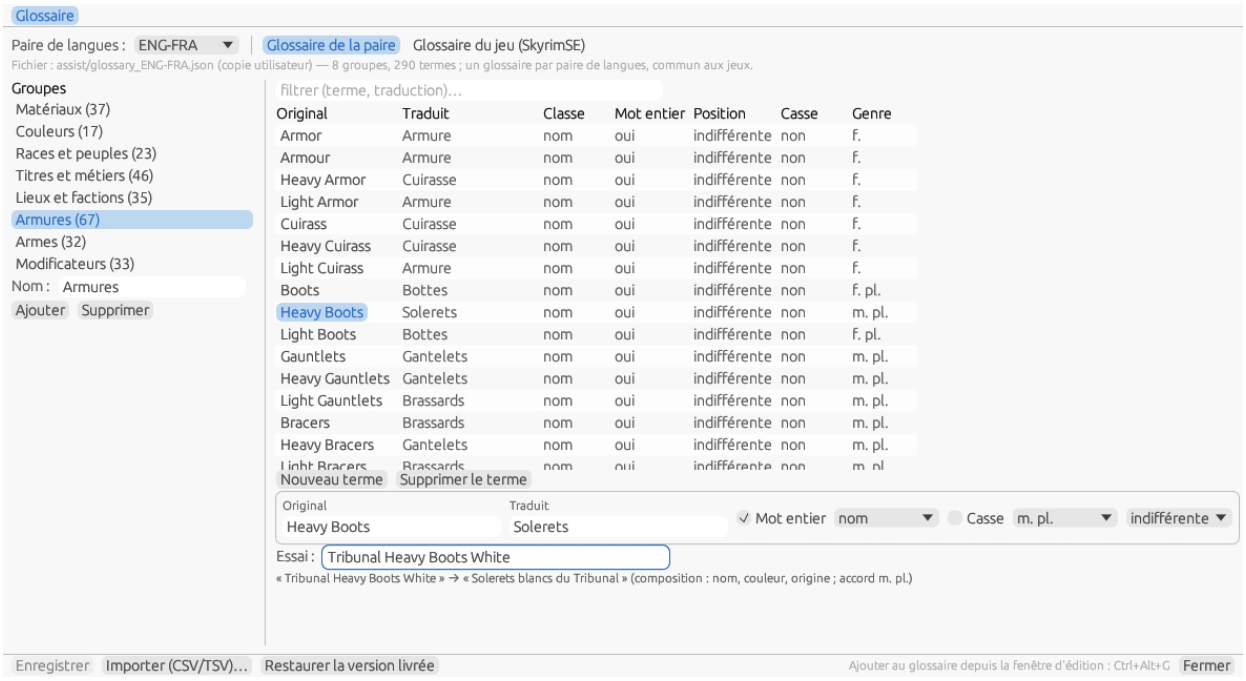


Figure 15.7 — Glossary, patterns and derivations, Glossary tab: the groups, the terms of the group Armures, the current term and the try of “Tribunal Heavy Boots White”.

The window. Independent, it opens on the pair of the active document; *Language pair* picks another, and *Glossary of the pair* / *Glossary of the game (SkyrimSE)* picks the file edited; the *File* line says which one is read (user copy, shipped version, new file) with its counts. On the left, the *Groups* with their number of terms, the *Name* of the chosen group, *Add* and *Remove*; on the right, the filter (term or translation), the table of the group's terms — Original, Translated (the forms separated by “/”), Class, Whole word, Position, Case, Gender —, *New term* / *Remove the term*, the current term and its forms, and the *Try* field: a text typed is translated by the glossary as edited, with the path followed (“composition: noun, colour, origin; agreement m. pl.”), or the words not covered. An error of the group — empty original, duplicate in the group, forms without the m. form — shows in red and *Save* refuses it. *Import (CSV/TSV)…* reads a two-column file “original; translated” (tab, semicolon or comma separator, optional header) into the group named after the file, new or existing; empty or already present originals are refused and named. *Restore the shipped version* deletes the user copy. Everything is taken into account without a restart; the window remembers its geometry (`[Window] AssistX/Y/W/H`).

Patterns with variables

The glossary translates terms; languages do not all build phrases in the same order: English says “Iron Sword of Burning”, French “Épée en fer de feu”. A **pattern** “%A% of %B%” → “%A% de %B%” captures

fragments (each variable `%NAME%` takes at least one word; the rest of the source is literal, compared without case; when there are several cuts, the leftmost first literal wins), has each fragment translated and writes them back in the order of the target. The patterns are in the pair's `patterns_<SRC>-<DST>.json` file, after the composition (pattern 1, always tried first), numbered from 2 and tried in order: the first that matches and whose variables all translate wins. Each pattern has its maximum length (60 characters by default): it is not tried on a longer text. Shipped for ENG-FRA: “%A% of %B%”, “%A% of the %B%”, “Scroll of %A%” (“Parchemin - %A%”), “Potion of %A%”, the possessive “%A%'s %B%” (“%B% de le %A%”) and parentheses; ENG-ZHO has the composition only.

Resolution. A variable is translated, in order: by a glossary term covering the whole capture; by the dictionaries of the document's stack (text only, then normalized form, § 7.3); by another pattern if the pattern is *recursive* (three levels at most, never itself); last by the composition of the glossary's terms. `vars` restricts the resolvers of a variable (`glossary`, `glossary:adjective` — a term that has forms —, `bases`, `patterns`, `pattern:3`, and `copy`, which copies a proper name whose every word is capitalised). An unresolved variable makes the pattern fail: the line is left alone. Examples: “Scroll of Elven Sword” → “Parchemin - Épée elfique” (pattern 4, %A% composed); “Mace of Molag Bal” → “Masse de Molag Bal”.

Agreement, articles, elision. The head variable (the last one of the target, or `head`) gives the pattern its gender; a variable resolved by an adjective with forms takes the matching form; `%A:cap%` forces a capital, `%A:f%` a form. At the junctions the pattern creates — never inside a translated term — the rules of the file's language apply: the article “le” takes the variable's gender (“la”, “les”, and disappears before a proper name without gender), the elision before a vowel or a mute h (“de Aevar” → “d'Aevar”, the `aspirated` list names the aspirated h), then the contractions (“de le” → “du”, “de les” → “des”, “à le” → “au”, and “de” before an origin that already has one). So “Ice Blade of the Monarch” → “Lame glacée du Monarque”, “Shield of the Night” → “Bouclier de la Nuit”, “Duke's Sword” → “Épée du Duc”, “Sithis's Blade” → “Lame de Sithis”.

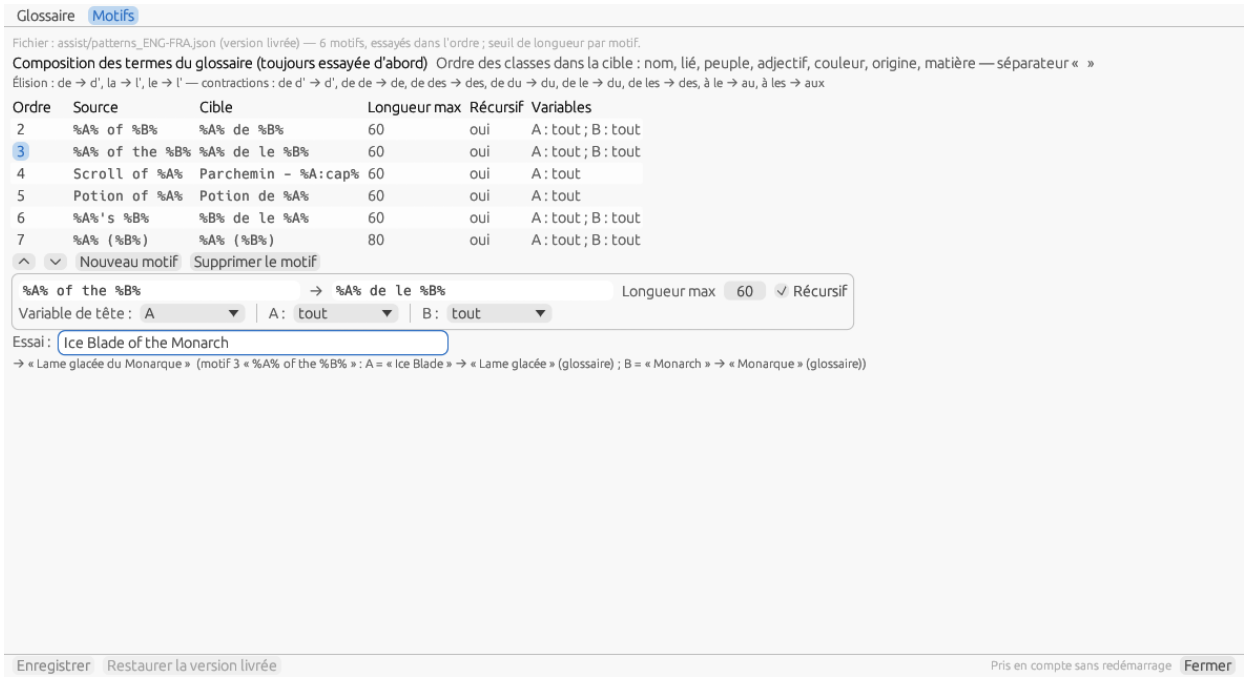


Figure 15.7 b — The Patterns tab: the composition and the rules of the language, the patterns in their order, pattern 3 being edited and the try of "Ice Blade of the Monarch" with the path followed.

The Patterns tab. The *File* line says which file is read; below, the composition (order of the classes, separator) and the file's elision and contraction rules. The table lists the patterns in their trial order — *Order*, *Source*, *Target*, *Max length*, *Recursive*, *Variables* ("A: all; B: glossary") —; ▲ ▼ reorder them, *New pattern* and *Remove the pattern*. The current pattern is edited below: source, target, maximum length, recursive, head variable and the resolver of each variable. The *Try* field translates a typed text by the glossary and the patterns as edited, with the full path: "pattern 3 "%A% of the %B%": A = "Ice Blade" → "Lame glacée" (glossary); B = "Monarch" → "Monarque" (glossary)". *Save* validates the file — each variable of the source in the target and back, length between 1 and 500, no patterns calling each other — and *Restore the shipped version* deletes the user copy; taken into account without a restart. In Apply the assistants (§ 11.5), the *Patterns with variables* box puts them to work; a line translated by a pattern has the provenance "rule: pattern 2".

The derivations of objects

In a magic or alchemy mod, many objects are only the name of a parent in a template: the scroll "Scroll of Fireball", the tome "Spell Tome: Fireball", the potion "Weak Potion of Fortify One-handed". A **derivation rule** says where to look for that parent: %A% is resolved by the **translated name** of a parent record — the spell SPEL:FULL, the effect MGEF:FULL —, looked up without case among the lines of the document, then of the other open files of the same pair, then of the dictionaries of the stack (by signature, otherwise text only: the compiled game bases have no signatures). The rules are per game: `assets/data/assist/derive_<Game>.json` (shipped: Skyrim, SkyrimSE, Fallout4, Oblivion, Starfield); the user copy of the same name in `data/assist/` wins. A rule: `{ "id": "scroll", "target": "SCRL:FULL", "src": "Scroll of %A%", "parent": ["SPEL:FULL", "MGEF:FULL"], "dst": { "fra": "Parchemin - %A%" }, "strip": { "fra": ["Invocation - "] } }`; the patterns follow the syntax of the patterns with variables. The target pattern depends on the target language: `dst` has one entry per language, and a missing language leaves the rule inactive for that target — the tab says so ("Rule "scroll": no target pattern for Deutsch — inactive.").

Transformations and variables. The parent's translation first goes through `strip`, which removes a prefix or suffix listed for the language ("Invocation - Liche" → "Liche"); `%A:low%` lowers its first letter, except a proper name (a glossary term whose case is kept), `%A:cap%` raises it; the language's elision and contractions apply at the junctions ("Potion d'éthérisation"). The other variables are resolved like those of the patterns: the degree `%G%` of a potion by `glossary:adjective`, agreed with the gender the rule declares for the noun it writes (`"gender": { "fra": "f" }`: "Potion de renfort d'armes à une main - Diluée", "Poison de … - Dilué"). A tome (`"ref": "teaches"`) first takes the spell it teaches (the book's DATA field, by FormID), the name serving only otherwise: the 190 tomes of Apocalypse teach their spell.

Status, provenance, following. A derived line is *Pending*, with the provenance "rule: derivation scroll (from WB_C050_ConjureLich_Spell)" (`rule:derive:<id>:<parent EDID>`); the Provenance ▸ rule filter finds it. Several parents of the same name translated differently (two open files) make an ambiguous line (§ 11.4). When the parent's translation changes — typing, dictionary, review —, the lines derived from it that are still Pending are recomputed in the same operation, which one Ctrl+Z undoes, and the status bar

says so: “Derived lines recomputed after the translation of “Conjure Lich”: 2”. A derived line set to Validated, or changed by hand, no longer follows.

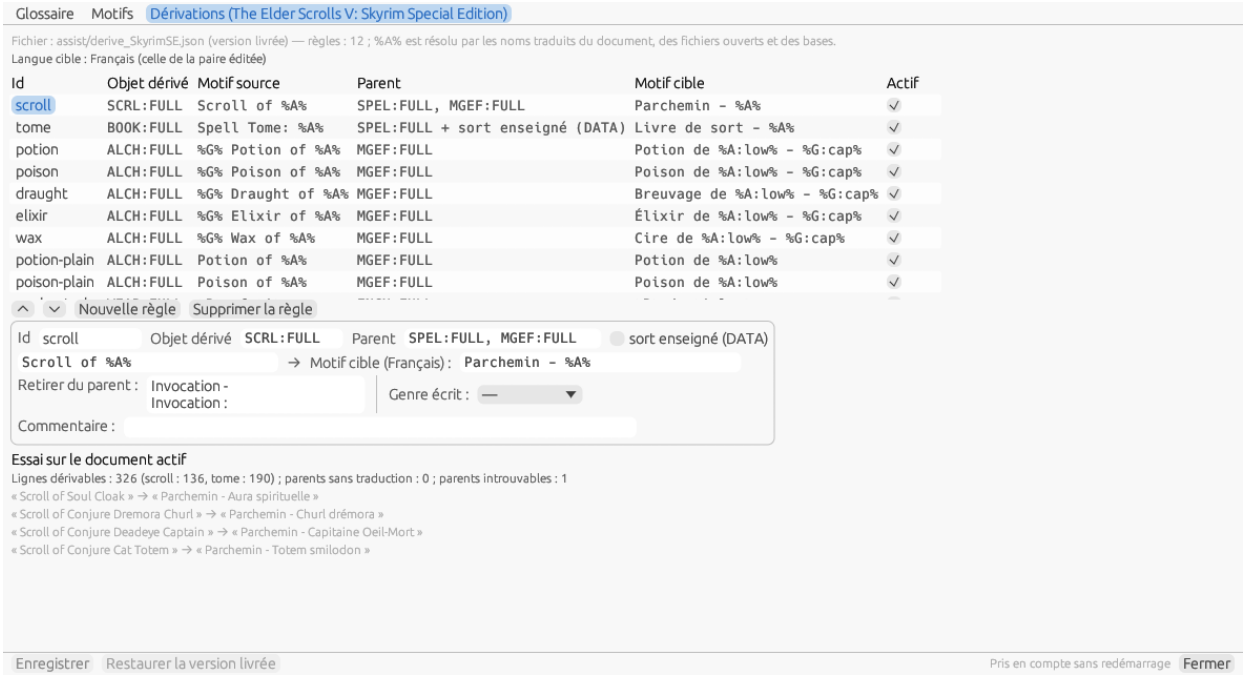


Figure 15.7 c — The Derivations tab for Skyrim Special Edition: the game's rules (enchanted and rank inactive), the scroll rule being edited (parents spell then effect, “Invocation - ” and “Invocation : ” stripped) and the try on Apocalypse, before applying: 326 derivable lines.

The Derivations tab. It bears the name of the game edited. The *File* line says which file is read and the target language, that of the window's pair. The table lists the rules — *Id*, *Derived object* (REC:FLD), *Source pattern*, *Parent* (with “spell taught (DATA)” for a tome), the language's *Target pattern* (“(none: inactive)” otherwise), *Active* — ; ▲ ▼ reorder them (the first one that applies to a line wins), *New rule* and *Remove the rule*. The current rule is edited below: identifier, derived object, parents (separated by commas), spell taught, source and target patterns, *Strip from the parent* (one prefix or suffix per line), *Gender written*, resolver of each other variable, comment. *Try on the active document* applies the rules as edited to the document's objects as if they had no translation: “Derivable lines: 326 (scroll: 136, tome: 190); parents without a translation: 0; parents not found: 1”, and the first lines obtained. *Save* validates the file — AAAA:BBBB signatures, unique identifier, non-empty source pattern, every variable of the target pattern present in the source pattern — and *Restore the shipped version* deletes the user copy; taken into account without a restart. In Apply the assistants (§ 11.5), the *Derivations* checkbox puts them to work.

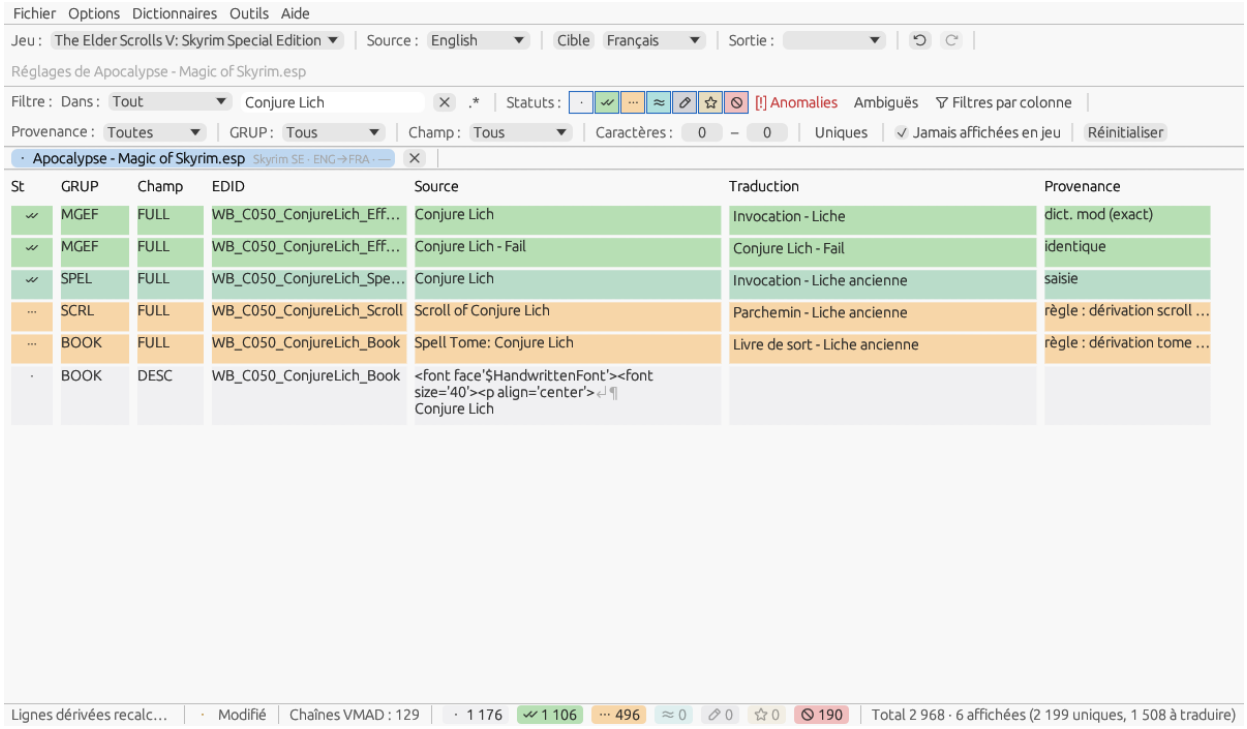


Figure 15.7 d — Following the parent: the translation of the spell “Conjure Lich” changed to “Invocation - Liche ancienne”, the derived scroll and tome follow, Pending; one Ctrl+Z undoes the three lines.

16. Settings

The *Settings* window (menu *Options* ▸ *Settings…*) has four tabs: General, Recent files, Online and Advanced.

16.1 General tab

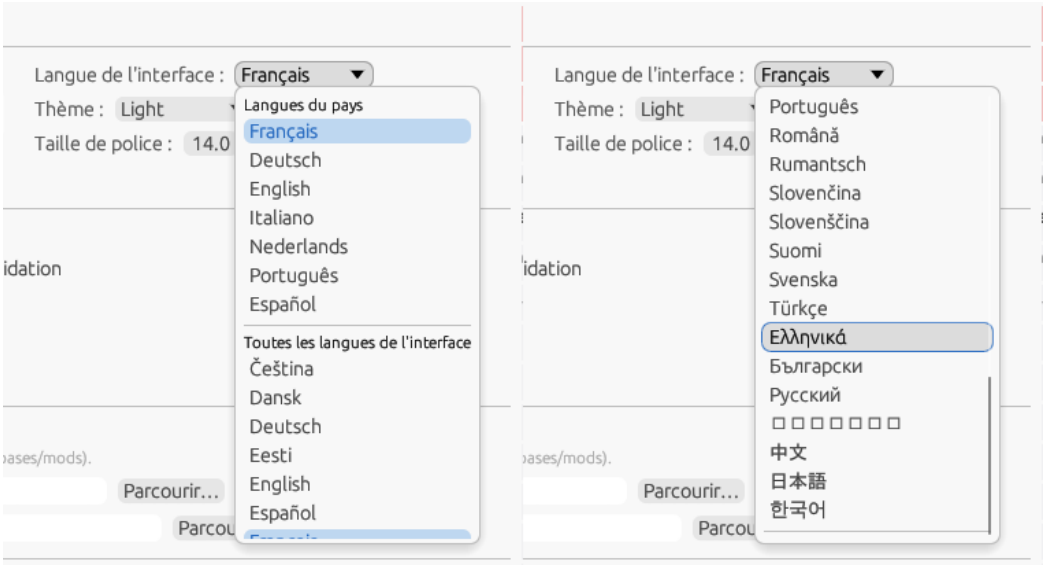


Figure 16.1 — The interface language: the languages of the chosen country (France), then all languages, each in its font (screenshots of this manual are in French).

- *Theme*: light, dark or system.
- *Interface language*: the list first offers the *Languages of the country* chosen with the flag that have an interface, then *All interface languages*, each under its own name and in its own font, in alphabetical order (§ 2.6); applied immediately. The code is stored in `[General] Locale`.
- *Font size*: previewed live while you adjust it.
- *Propagate a translation to the open files with the same source*: checked at first (§ 7.4).
- *Max recent files*: number of entries kept in the Recent menu.
- *Window width / Window height*: dimensions of the main window.
- **Translation database folders** — *Game bases* and *Mod translations*; leave empty for the default locations (`assets/bases/jeux` and `assets/bases/mods`).
- **External tools** — root folders of *ESP-ESM Translator* and *xTranslator*. An empty path disables the corresponding conversions in the Dictionaries menu.

As in XIMOD Architect, Save and Cancel stay visible at the bottom of the window whatever the height of the tab; the control that has keyboard focus is outlined; the theme and the font size are previewed while you adjust them and reverted on cancel (Esc); the language applies only on save. *Transient notifications* turns the messages at the top right on or off (§ 4.5); *Propose the propagation to the lines of the same source after a validation* opens the propagation dialog after F5 or Ctrl+↓ (§ 8.4).

16.2 Recent files tab

Lists the recently opened files; you can clear the list.

Removing an entry or clearing everything is saved immediately, without waiting for Save; *Clear* asks for a second click (“Click again to clear”) before acting.

16.3 Advanced tab

- **Backups** — *Automatically save the work in progress* (enabled), interval in minutes (5, from 1 to 60), *Timestamped copies kept per mod dictionary* (10, from 0 to 100) and *Rescue folder* (empty = `autosave/` in the configuration folder), with a button to open it, and *Write the mod dictionary at each save and export* (checked; § 7.6), *Fill my personal dictionary at each save* (unchecked) and *Ctrl+M keeps the field* (unchecked; § 12.10) and *Keep the old data of the mod dictionaries* (0 = forever; § 7.6). See § 7.5 and § 12.4.
- **Word heuristic** — *Word heuristic threshold*: proportion of differing words allowed between a line and a close source (20%, from 5 to 50; § 8.5 and § 11.3).
- **Normalized matching** — *Normalized matching at opening (punctuation, numbers, case)*: the “normalized form” level of the stack at opening (checked by default; § 7.3).
- **Mod dictionary** — *Check at opening that the mod's dictionary matches the plugin (CRC-32)* (checked; § 7.6).
- **Outputs** — *Keep the original before overwriting an existing file* (checked) and *Copies kept per file* (5, from 1 to 50; § 11.2).

- **Configuration location** — *Per user (recommended)* or *Portable* (§ 2.5); the current location is shown with an *Open folder* button. Changing it copies the configuration, the personal dictionaries, the data and the backups to the destination and takes effect at the next startup; switching to portable is refused if the executable's folder is not writable.
- **Undo** — number of undoable operations kept per file (200, from 10 to 1,000).
- **History of changes** — *Keep the history of changes* (checked) and *Entries kept per file* (20,000, from 1,000 to 200,000; § 8.12).

16.4 Online tab

The online translation services (§ 8.10). At the top, the services with the state of their key — DeepL, Azure, one service per OpenAI-compatible profile ("OpenAI — Ollama local"), Google —; the *Default service* list sets the one that Ctrl+G calls. On the right, the selected service: *API key* (hidden, *Show* / *Hide*), *Endpoint* (DeepL: api-free.deepl.com for a free key, api.deepl.com otherwise; Azure: the endpoint and the *Region*; for an OpenAI profile, the whole profile: name, endpoint, model, prompt, answer format, sampling and its own guards, § 8.10), *Formality* (DeepL), and *Test…*. The *Guards* group holds the quotas, the session threshold and the three protections; the *Network* group, the proxy and the timeout; *Clear the cache* erases the stored responses. Keys are saved with *Save*, in the system keyring or, failing that, in `secrets.json` — the line under the key says which.

16.5 Keyboard navigation

Dialog boxes can be driven from the keyboard: Tab to move around, arrows for drop-down lists, Enter to confirm, Esc to close.

Shortcut	Function
Ctrl+Z / Ctrl+Y	Undo / Redo (§ 6.6).
F5 / F6 / F7 / F8	Validated / Pending / Custom / Ignored status for the selection (§ 8.2).
Ctrl+↓ / Ctrl+↑	Validate, then next / previous line (§ 8.2).
Shift+click / Ctrl+click	Extend the selection / add or remove a line.
Right-click	Line menu (§ 8.2); Ctrl+C / Ctrl+V / Ctrl+F8 when not typing: copy, paste, translation = source.
↑ ↓ / PgUp PgDn / Home End	Move the selection by one line, by one page, to the start or to the end of the view (when not typing).
Ctrl+O / Ctrl+S / Ctrl+Shift+E / Ctrl+W / Ctrl+Q	Open / Save as / Export / Close document / Exit (§ 4.1).
Ctrl+, / F1	Settings / User manual.
Ctrl+F, Esc	Focus on the Filter field with its text selected; Esc in the field clears it (§ 4.3).
Ctrl+Shift+F	Shows or hides the column filter row (§ 10.1).

Shortcut	Function
Ctrl+Alt+F	Advanced search (§ 10.2).
Ctrl+Tab / Ctrl+Shift+Tab	Next / previous tab, “All mods” included (§ 7.4).
Ctrl+K	Concordance: the selected term, or the word under the cursor, looked for in the active dictionaries (§ 10.3); also in the edit window.
Ctrl+H, F3 / Shift+F3	Find and replace; next / previous occurrence (§ 8.6).
Ctrl+M	Add the pair of the selection, or of the edit window's line, to my personal dictionary (§ 12.10).
Ctrl+Alt+G	Edit window: add the selection of the source and translation to the glossary (§ 8.5, § 15.7).
F10	Dialogues: opens the dialogues window of the active document (§ 8.8).
Ctrl+Alt+Left / Ctrl+Alt+Right	Previous / next line of the conversation, in the Dialogues window and the Dialogue tab (§ 8.8).
F12	Quality checks: opens the window and runs the checks; in the window, Enter opens the selected line (§ 8.7).
Enter, double-click	Open the edit window on the line, or bring it to the front (§ 8.5).
Ctrl+Enter	Edit window: validate, then next line (§ 8.5).
Alt+Right / Alt+Left	Edit window: next / previous line without changing the status (§ 8.5).
Ctrl+F6	Apply to every line with the same source… — propagation with dialog, grid and edit window (§ 8.4).
F9 / Ctrl+F9	Edit window: instant translation of the source selection or of the whole source / in two steps, at the source cursor (§ 8.5).
Esc	Edit window: close while keeping the input; close a dialog.
Ctrl+S, Ctrl+Enter, Ctrl+G	Translate a FOMOD window: save, next untranslated, translate the line online (§ 15.3).

16.6 Options ▸ Properties…: the country / language database

The flag selector, the list of interface languages, the interface translation editor (§ 15.5) and the choice of fonts rely on four reference files shared with XIMOD Architect, in `assets/data/`: `Countries.json` (244 countries, their official languages and the name of the country in each), `Languages.json` (6,777 ISO 639-3 languages, their ISO 639-1 code, their font), `CountryLanguages.json` (every language spoken in each country) and `CountryNames.json` (the name of each country in each interface language). *Options ▸ Properties…* opens the independent window **Country / language database**, which lets you explore them without changing anything — the same as in Architect. It stays open when you change document, moves to a second screen and comes back to its place (`[Window] PropsX/Y/W/H`); Escape or *Close* closes it.

Countries tab. The filter keeps the countries whose shown name (in the interface language, otherwise English), English name, an official name or ISO 3166 code contains the text typed, ignoring case and

accents (“confederation” finds the Swiss Confederation, “svizra” too); “1 of 244” says how many remain. The chosen country shows its flag, its name, its English name and its code, then its *Official languages* — ISO 639-3 / 639-1 code, language name, name of the country in that language, font, and whether the interface exists in that language — and the other *Languages spoken*, from CountryLanguages.json. A click on a language opens the Languages tab on it.

Languages tab. The filter applies to the name and the codes; an exact code comes first (“fr” gives French first). The chosen language shows its endonym, its codes, its font (or “no associated font”), whether the interface exists in that language with its folder, then the list of countries where it is spoken — those where it is official first — with the flag, the name of the country in the interface language, its local name in the chosen language and “Official”. A click on a country opens the Countries tab on it.

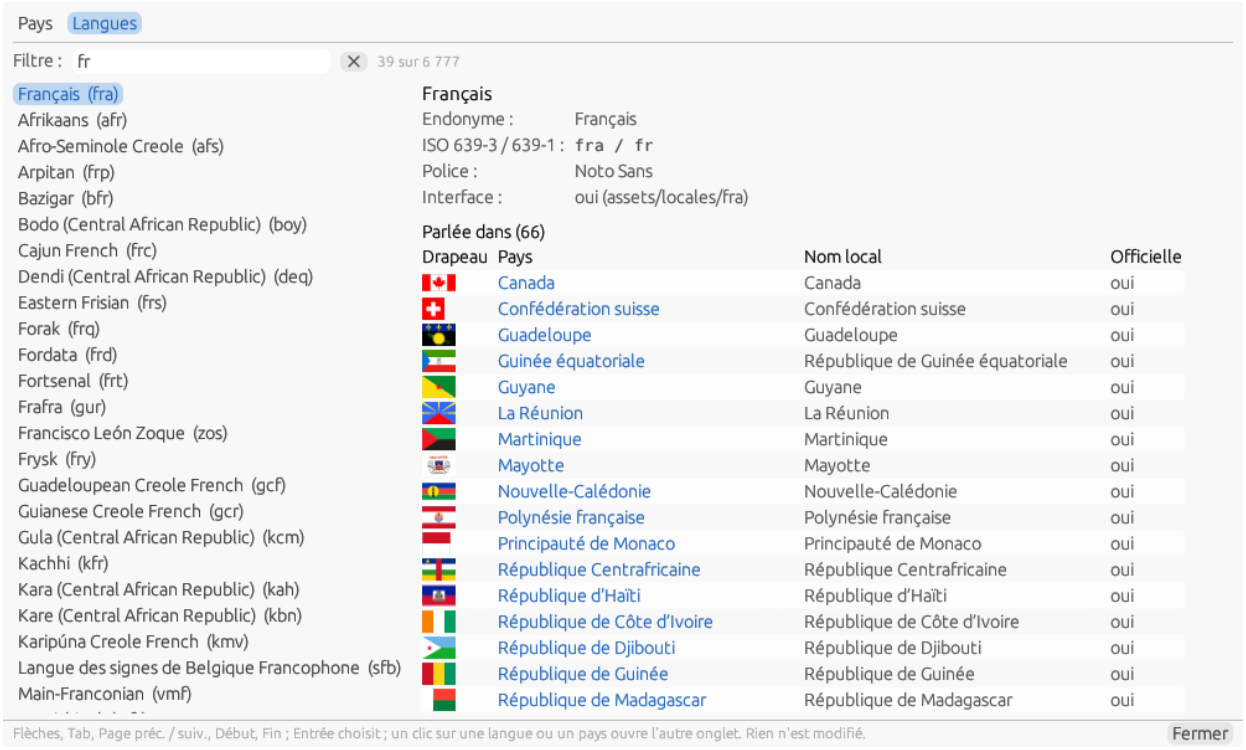


Figure 16.6 — Country / language database, Languages tab: French, its codes, its font, the interface, and the countries where it is spoken, official ones first.

From the keyboard. The cursor is in the filter at opening and at each change of tab; the arrows, Tab and Shift+Tab move the chosen row of the list, Page Up / Down by a page, Home and End go to the ends, Enter chooses the row. The lists only build the visible rows: the 6,777 languages scroll without waiting. The window never writes anything; if the translation editor changes a language during the session, it reads its tables again. A missing file blocks nothing: without CountryNames.json, the names are in English; without the flags folder (`assets/images/svg/`, shared with Architect), an empty frame replaces them.

17. The configuration file

All preferences are saved in `Config.ini`, in the configuration folder (§ 2.5), and restored at the next launch. The file is managed automatically and written atomically (never truncated); it is human-readable and repairs itself if a value is missing. Its main sections:

Section	Contents
`[General]`	Interface language, country, theme, font size, number of recent files, splash duration, Toasts (transient notifications), Translator (name signed in FOMOD translations); ConfigMode (portable / user), informational.
`[Translation]`	Last choices in the Source, Target, Output and Game (DefaultGame) lists, remembered for information only: at launch, the lists start empty; PersonalDictDefault (initial state of the personal dictionary box).
`[Editing]`	UndoDepth: undoable operations kept per file; ShowUnused; EditTab: last tab of the edit window; ShowSpecialChars: special characters visible; ProposePropagation: propagation dialog after validation (§ 8.4); ColumnFilters: column filter row shown (§ 10.1).
`[History]`	Enabled: history of changes kept (1 by default); MaxEntries: entries kept per file (20,000, from 1,000 to 200,000; § 8.12).
`[Columns]`	Optional grid columns (§ 8.1): ShowStatusButtons (status buttons column, § 8.2), ShowComment (Comment column), ShowReview (Review column, § 8.11), ShowProvenance (Provenance column, § 9.1); 0 by default.
`[Export]`	GeneratedComments: generated comments are written to EET and XML files (§ 11.1); UntranslatedCopy: untranslated lines of a text file taken from the source (§ 7.7).
`[Dialogues]`	FollowGrid: the Dialogues window follows the grid selection; Split: position of its splitter (§ 8.8).
`[Diagnostics]`	FrameLog: threshold in milliseconds above which a slow frame and its sections are written to logs/ximod.log (0 = nothing, the default); set it when support asks.
`[Quality]`	Checks: selected quality checks, separated by commas (§ 8.7).
`[Fuzzy]`	Threshold: word heuristic threshold, as a percentage of differing words (§ 8.5); Normalize: normalized matching at opening, 1 or 0 (§ 7.3).
`[Personal]`	AutoFill: personal dictionary filled at each save; KeepField: Ctrl+M keeps the field (§ 12.10).
`[Dictionaries]`	Dicts_<Game>_<SRC>-<DST>: the dictionary tree of each game and language pair — roles and files in order, “!” for an unchecked box, full path for an added file, “+” before an added folder (§ 12.8).
`[Backup]`	AutosaveEnabled, AutosaveMinutes, AutosaveDir, Copies; ModDictAuto: mod dictionary written at each save and export; OldDataMonths: old data kept (months, 0 = forever) (§ 7.6).

Section	Contents
`[Online]`	Default service, endpoints and service options (DeepLEndpoint, DeepLFormality, AzureRegion, AzureEndpoint, OpenAIEndpoint, OpenAIModel), quotas (MaxCharsPerRequest, MaxLinesPerBatch, MaxRequestsPerMinute, SessionWarnChars), protections (ProtectTags, ProtectNumbers, ProtectNewlines), Proxy, TimeoutSeconds (§ 8.10, § 16.4). API keys are never stored there: system keyring or secrets.json.
`[Window]`	Size and position of the main window and of all secondary windows (Settings, viewer, explorers, spell checker, session log: JournalX/Y/W/H, § 4.7…).
`[Paths]`	Folders of the databases (games/mods) and of the external tools (EET, xTranslator).
`[Games]`	Games detected or added manually.
Colors & recent files	Status colors, stripe color, list of recent files.

> **Note —** If the application behaves unexpectedly, closing it and then deleting `Config.ini` restores all default settings at the next launch.

18. The command line

Everything the window does to a file — open it, apply the stack, the assistants, translate online, check, export, package — can also be done without it, with `ximod-cli`, the second executable shipped in the same folder (`ximod-cli.exe` on Windows, a console program used in a command prompt or PowerShell; `XIMOD Translator.app/Contents/MacOS/ximod-cli` on macOS). It calls the same functions as the interface: a mod dictionary, a DSD output or a patch it writes are the ones the interface would write with the same settings. It reads the same `Config.ini` (§ 2.5; `--config <folder>` imposes another one, a portable folder included) for the base folders, the default languages, the formats, the rules and the services, and never writes it; it opens no window and touches neither the recent files nor the autosave. Uses: a translator rebuilding twenty mods after an update, a list maintainer checking his translations in a continuous integration.

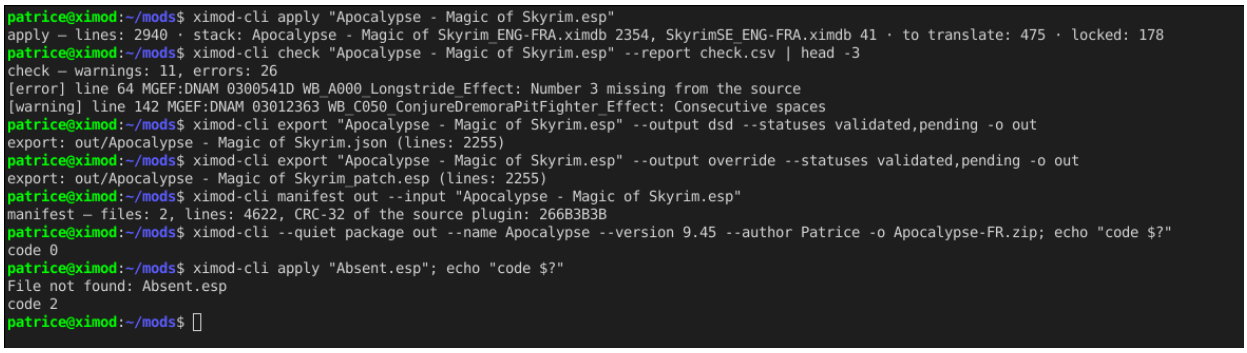


Figure 18.1 — A session: apply on Apocalypse, check, DSD and override export from the mod dictionary, manifest, package; then a command on a missing plugin, code 2.

18.1 Common options and return codes

Option	Effect
`--game <id>`, `--from <iso3>`, `--to <iso3>`	The game and the languages. Defaults: the game read in the plugin header, then those of Config.ini (DefaultGame, SourceLanguage, TargetLanguage).
`--json`	The machine output: one JSON object per command on the standard output (one per step for a batch), the messages on the error output.
`--quiet`, `--verbose`	Nothing on the standard output but the JSON; or the session log (§ 4.7) on the error output, as the run goes, one entry per line “time ⇥ level ⇥ document ⇥ message” — opening, one entry per dictionary of the stack, reports, warnings, errors — and, with --json, in the response object (key journal). No API key and no text sent online ever appears in it. The technical trace is set apart, through the RUST_LOG variable.
`--log <file>`	The same log appended to the end of that file (UTF-8), with or without --verbose; a file that cannot be written stops the command before anything is done (code 2).
`--lang <iso3>`	The language of the messages and of the help; default: that of the interface.
`--config <folder>`, `--no-color`, `--help`	The folder of Config.ini; output without colours (the output has none); the help of the program, or of a command: ximod-cli export --help.

There are three return codes, so that a script only has to choose between going on and stopping: **0** done; **1** done with reservations (check found errors, or warnings with `--fail-on warning`; export leaves untranslated lines with `--strict`; translate refuses beyond `--max-chars`); **2** not done (file missing or unreadable, unknown option or value, invalid format, service unreachable, write refused). The error messages are those of the interface.

18.2 The commands

Command	What it does
`info <file>`	The type, the game, the detected languages, the lines per record type, the state of the mod dictionary (recognised, other version, absent, § 7.6), the size and the CRC-32, the interface files found.
`import <file> --format <id> [-o …]`	A text, JSON or XML file read with a format (§ 7.8, § 15.2) and written as a pivot (.json, default <name>.ximod.json) or as a dictionary (.ximdb).

Command	What it does
`apply <file>`	Opens as the interface does (the mod dictionary recognised, the stack applied, the exclusion rules laid) and says what the stack filled. --dict and --base add a dictionary or a folder to the stack, --no-personal leaves the personal dictionary out; --mode and --overwrite replay Apply dictionaries… on the whole document (§ 11.3); --write-dict writes the mod dictionary (format 4) and the history next to it (§ 8.12).
`assist <file>`	Apply the assistants (§ 11.5) on the whole document: --glossary, --patterns, --derive (all three without an option), --overwrite; lines Pending, provenance of the rule; --write-dict.
`translate <file> --service <id> --max-chars <n>`	Translate online (§ 8.10): --scope untranslated (default) or document; Validated and Custom are never replaced; nothing is sent if the characters to send exceed --max-chars, required; lines received Pending; --write-dict.
`check <file>`	The quality checks ticked in the Settings (§ 8.7), one line per finding; errors: tags, numbers, characters the code page cannot write; warnings: the rest; --report writes the report as CSV.
`export <file> --output … -o <folder>`	strings, hybrid, override, full or dsd, from the mod dictionary (or --dict), with the status policy (§ 11.2; --statuses validated,pending,…, Validated always). Names: <plugin>_patch.esp, <plugin>.esp (full), <plugin>.json (DSD), <plugin>_<Language>.STRINGS; the output never replaces the source plugin.
`manifest <folder> [--input <plugin>]`	The manifest of the outputs of a folder (§ 18.4), the source plugin as input.
`compare <A> <B>, scan <folder>`	Compare two dictionaries (§ 12.12: --mode, --fold, --report, --merge with --prefer); analyse a folder (§ 7.9: --no-subdirs, --scripts, --report).
`convert <input> -o <output>`	The conversions of the Dictionaries menu (§ 12.3): to .ximdb from EET, SST, XML or pivot, from .ximdb to those formats.
`package <folder> -o <zip>`	Packaging (§ 15.4): --name, --version, --author, --website, --description, --plugin (the plugin a DSD targets), --no-info; the manifest of the folder goes into fomod/.
`batch <batch.json>`	Chains the commands of a batch file (§ 18.3).

A command is an opening: `check` or `assist` alone reopen the file and its stack, as the interface does; what a command changes is only kept by `--write-dict`. `export` reads the mod dictionary, that is the saved work; in a batch, it exports the document as the previous steps left it.

18.3 The batch file

A JSON file chains the commands on several files, portably between Windows, Linux and macOS:

```
{ "format": 1, "game": "SkyrimSE", "from": "eng", "to": "fra",
  "out": "out/", "stopOnError": false,
  "jobs": [ { "file": "mods/Apocalypse - Magic of Skyrim.esp",
      "steps": [ "apply", { "assist": { "derive": true } }, "check",
        { "export": { "output": "dsd",
                      "statuses": ["validated", "pending"] } },
        "manifest", { "package": { "version": "9.45" } } ] },
    { "files": "mods/patches/*.esp", "steps": [ "apply", "check" ] } ] }
```

- The keys at the head hold for every job; a job may give again `game`, `from`, `to`, `out`, `stopOnError`. `"files"` takes a pattern (* and ? in the name), each file found becomes a job. The paths are relative to the batch file.
- Each step is a command — info, apply, assist, translate, check, export, manifest, package — with its options: `true` for an option without a value, a list for --dict or --statuses; writeDict may also be written write-dict. The outputs of each job go into `<out>/<plugin name>/`, the package next to it.
- The whole file is validated before the first step (format, existing files, known steps and options, values): an invalid batch is refused without writing anything, code 2.
- Each step says its result (one line, or a JSON object with `--json`); a step not done (code 2) stops its job; `stopOnError` then stops the whole batch, otherwise the batch goes on and its code is the highest met.

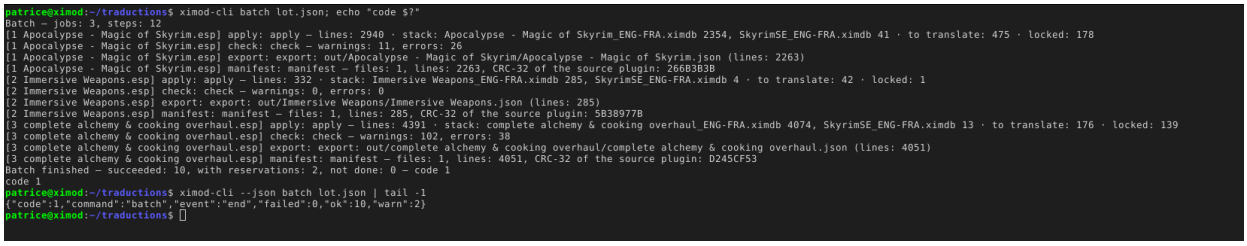


Figure 18.3 — A batch of three mods (apply, check, DSD export, manifest): one line per step, the summary, code 1 because check found errors; with --json, one object per step and an end object.

18.4 The manifest

`manifest.json` (format 1) says for which version of the mod the translation was made and what was produced: `app`, `date`, `game`, `from`, `to`; `inputs` (the source plugin: path, size, CRC-32 — the identity of § 7.6); `dictionary` (the mod dictionary, its Validated, Pending, Ignored lines); `stack` (each dictionary of the stack, its role, the lines it filled); `statuses` (the exported statuses); `outputs` (path, type, size, CRC-32, lines for the files the application reads); `checks` (errors, warnings). The command alone describes a folder; the manifest step of a batch describes the whole job. The package puts it in `fomod/manifest.json`; in continuous integration, comparing two manifests says what changed.

18.5 Scripts, continuous integration and keys

The `docs/cli/` folder shipped with the application contains: `rebuild.sh` and `rebuild.ps1` (rebuild a published translation: apply, check, DSD and override export, manifest, package — nothing is written if the plugin is missing, code 2); `batch-example.json`; `github-actions.yml` (rebuild the outputs at each update of the mod dictionary and fail if check finds errors).

```
XIMOD_CLI=./ximod-cli bash docs/cli/rebuild.sh "mods/Apocalypse - Magic of
Skyrim.esp" out/Apocalypse "Apocalypse" 9.45 Patrice
echo $?        # 0, 1 or 2
```

> **Note —** The keys of the online services come from the system keyring or from secrets.json, as for the interface (§ 16.4), or from the environment variable `XIMOD_KEY_<SERVICE>` (`XIMOD_KEY_DEEPL`, `XIMOD_KEY_OPENAI_LOCAL` for the openai:local profile) — never from an argument; no key is written on the output or in the log, even with --verbose.

19. Localization of Bethesda games

The way a game stores its texts determines how to translate it. There are four families:

Family	Games	Where the texts are
A — embedded strings	Morrowind, Oblivion, Fallout 3, New Vegas	Directly in the ESP/ESM fields (one language per file, Windows-1252 encoding).
B — Bethesda STRINGS	Skyrim LE, Skyrim SE/AE, Fallout 4	The “Localized” plugin contains identifiers; the text is in the .STRINGS/.DLSTRINGS/.ILSTRINGS triplets.
C — centralized STRINGS	Fallout 76, Starfield	ESM + a multilingual “… - Localization.ba2” archive.
D — Unreal LOC	Oblivion Remastered	LOC_* keys, Game.locres and Unreal StringTables in the PAK/UCAS/UTOC containers.

A mod from family B or C can nevertheless be **non-localized** and contain its English text in plain form in the ESP: XIMOD Translator detects the encoding automatically on opening. On the installation side, a translation is distributed according to the family: translated plugin, “loose” STRINGS triplet, localization archive, or Unreal/PAK files — making sure, in MO2 or Vortex, that the translation wins over the original.

The three sections that follow extend this table with concrete cases: where the application looks for the texts of a localized plugin, how to know the language of a plugin, and how XIMOD Translator can be used beyond translating a mod — up to giving a game a language that Bethesda never released.

19.1 Where the application finds the STRINGS

When a localized plugin is opened, XIMOD Translator first looks for the **loose files** `<plugin>_<language>.strings`, `.dlstrings` and `.ilstrings` in the `Strings` folder next to the plugin; if they are missing, it opens the **neighboring BSA or BA2 archives** whose name begins with the plugin's name (`MyMod.ba2`, `MyMod - Main.ba2`) and extracts the STRINGS from them. A mod downloaded as is, whose author packed the texts into its archive, therefore opens without any manual steps. The same path exists in reverse: *Tools ▸ BSA/BA2 Explorer* (or dropping an archive onto the window) lists the contents

of an archive, and *Open associated plugin* loads the neighboring plugin, resolving its texts from the archive (§ 15).

> **Note —** Loose files always win over the archive: this is what makes it possible to install a translation as "loose" STRINGS without touching the original BA2 — provided the game loads loose files (for Starfield, enable this in `StarfieldCustom.ini`).

19.2 Knowing the language of a plugin

For a **localized plugin**, the answer is in the files: each STRINGS file carries its language in its name (`_english`, `_french`…; `_en`, `_fr`… for Starfield), and the application lists them. This is how it checks that the chosen *Source* language really exists next to the plugin, or in its archive, and refuses to open the file otherwise (§ 6.3). A plugin can thus offer several source languages.

For a **non-localized plugin**, the format says nothing: a plugin header has no language field, and the plain text can be in any language — in practice, almost always English. The *Source* language is then **detected from the text** by the Lingua library, which recognizes 75 languages from their letter sequences and remains reliable on short texts: the list receives the plausible language or languages, the most likely first (§ 6.3). When in doubt — a mod made only of short names, often proper names — two clues help you decide: the mod page (its author states the language), and the grid itself, where the names of records taken from the base game (a renamed NPC, a rebalanced weapon) can be compared at a glance with the text of the game dictionary.

19.3 Adding a language that Bethesda does not offer to a game

XIMOD Translator translates mods, but nothing limits it to mods: the game's ESM and its DLCs are plugins like any other, and the output modes make it possible to give a game a language that the publisher never released. The method depends on the game family.

Games with STRINGS (Skyrim, Skyrim SE/AE, Fallout 4, Fallout 76, Starfield)

The engine does not have a closed list of languages: it reads a token in the ini file (`sLanguage=ENGLISH` in `Skyrim.ini`, and its equivalent in `Fallout4.ini` or `StarfieldCustom.ini`) and loads, for each localized plugin, the file `<plugin>_<token>.strings`. Nothing prevents you from writing `sLanguage=DUTCH` and supplying `Skyrim_dutch.strings`, `Update_dutch.strings`, `Dawnguard_dutch.strings`…: communities have produced complete translations this way in languages that were never released (Brazilian Portuguese, Turkish, Hungarian, Korean…). With XIMOD Translator, this means opening each ESM of the game with *Source* set to English and *Target* set to the new language, and exporting in *STRINGS* mode (§ 11).

- The set of STRINGS must be **complete**: the ESM, all its DLCs and every installed localized mod. The engine has no fallback to English: a plugin without STRINGS in the active language displays empty texts.
- The game's **fonts** must contain the glyphs of the language: Latin diacritics and Cyrillic are included; Greek, Vietnamese or Thai require replacement fonts, and right-to-left scripts are not supported by the engine.

- The platform may rewrite `sLanguage` at launch (Steam aligns the ini with the language chosen in its settings). This is why most community translations choose instead to **overwrite the English STRINGS** while keeping `sLanguage=ENGLISH`: untranslated mods then remain readable in English, which is the best compromise in practice. With XIMOD Translator, you simply name the exported files with the English token.

Games without STRINGS (Morrowind, Oblivion, Fallout 3, New Vegas)

The texts are in the ESM itself, and a language version is a different ESM (the French `Oblivion.esm` is not the English `Oblivion.esm`). Adding a language means translating the ESM — *Full plugin* mode — or shipping an override ESP that rewrites all the records that carry text — *Patch override* mode. Both work; the patch has the drawback of conflicting with any mod that touches the same records (the last one loaded wins and brings back English), hence the value of loading it early and of translating the mods too. Fonts are replaced through the `.fnt` files in the `Fonts` folder.

Skyrim SE with DSD

The Dynamic String Distributor (SKSE) replaces texts in memory from JSON files, without touching either the STRINGS or the plugins, and therefore without load-order conflicts: for this game alone, it is the cleanest way to add a language to the ESM as well as to mods. The *DSD (SKSE)* output mode writes these files directly.

> **Note —** These indications describe the known behavior of the engines; each added language deserves an in-game test before distribution, especially for fonts.

19.4 Localizing a mod

Many Skyrim SE, Fallout 4 and Starfield mods keep their texts in the plugin. To distribute a translation as STRINGS — the form Bethesda prescribes for multilingual mods, and that of ESL patches —, you need a **localized** plugin: its text fields replaced by identifiers, the texts stored in STRINGS. On such a plugin, the *Output* list shows *STRINGS (localize the plugin)*; the export (figure 18.4) writes:

- the **localized plugin**, where you save it (its original place is proposed, the original being kept in `backups/`, § 11.2): Localized flag set, each localizable field gets an identifier, from 1 to n in file order — one per field, even when two fields have the same text, like the Creation Kit —, an empty field gets 0; the header (author, description) stays as text; a compressed record that changes is rewritten uncompressed, the others are copied byte for byte;
- the **STRINGS of the target language** in `Strings/` next to it — `<plugin>_French.STRINGS`, `.DLSTRINGS`, `.ILSTRINGS` following the game's style —, with the translation of the exported lines (Validated by default) and the original text of the others;
- the **STRINGS of the source language** (box checked by default, `[Export] LocalizeSourceStrings`): the original texts, so the mod can also be played in its own language.

Localizable fields. In a localized plugin, the engine reads as an identifier every field it holds to be a localizable string, and looks it up in a specific list (STRINGS for names, DLSTRINGS for descriptions and books, ILSTRINGS for dialogue lines). The conversion therefore follows a **table of localizable fields** per game. The application ships one (`assets/data/lstrings/<Game>.json`, deduced from the known

assignment rules); *Dictionaries* ▸ *Compile game bases* replaces it with a table derived from the files of the installed game — each field of the official master whose value is an identifier of its STRINGS, with the list where it was found — written to `data/lstrings/` in the configuration folder. The export dialog says which one applies; the translatable fields editor (§ 15.1) shows it in its *List* column, read-only. A text field of the plugin that the table does not know stays as text: the dialog counts it ("Text fields the table does not know"), the log names them; the engine may read it as an identifier, to be checked in game.

The plugin produced reopens as a localized plugin: the lines are found again with their translations (mod dictionary), and the ordinary STRINGS mode applies from then on. Delocalizing it (Export ▸ Full plugin) gives back the original plugin, record for record. The mode is not offered for an already localized plugin, a text document or a game without STRINGS; a text longer than 65,535 bytes is refused.

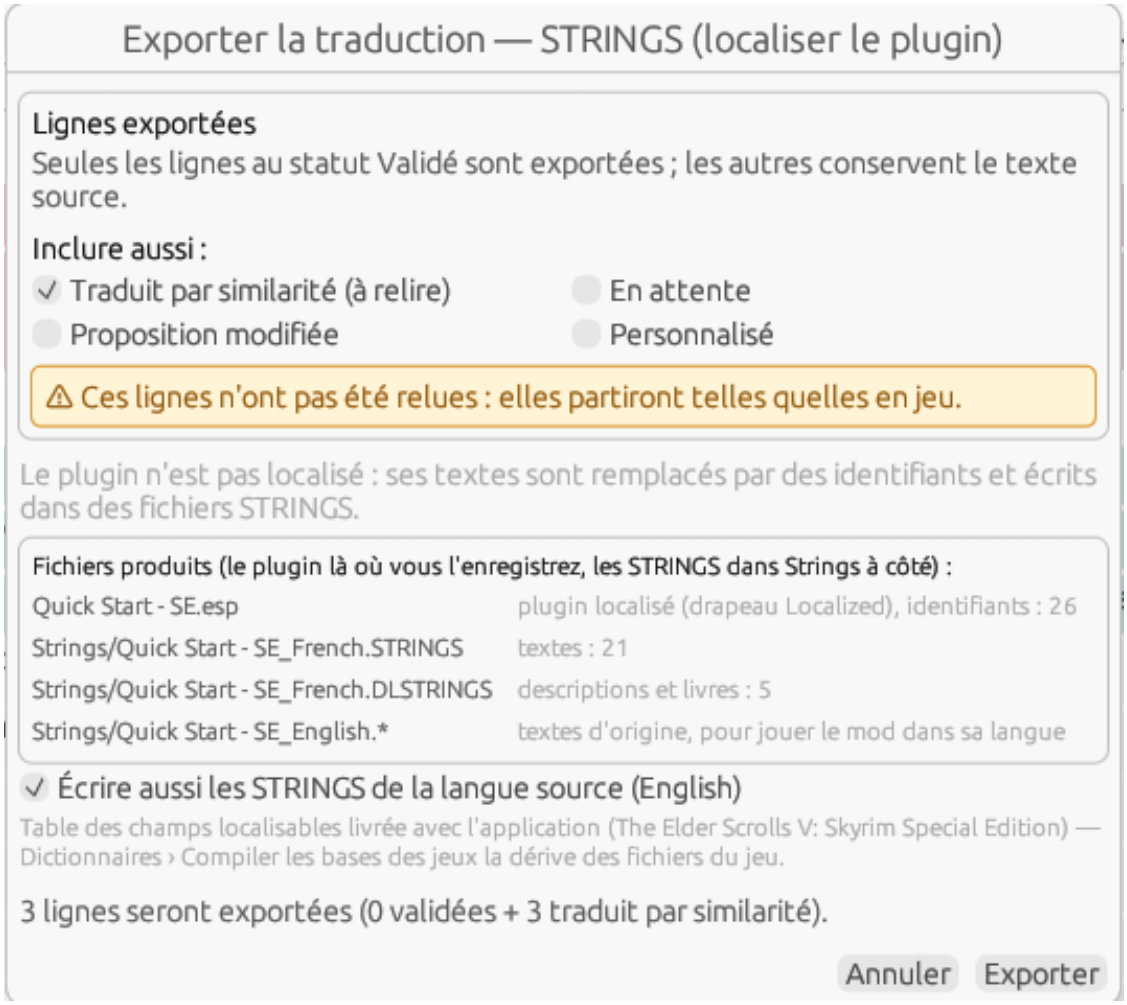


Figure 19.4 — Exporting the translation in STRINGS (localize the plugin) mode: the files produced, the source-language STRINGS option, the table of localizable fields.

19.5 Encodings and code pages

The 64-bit games (Skyrim SE and AE, Fallout 4, Fallout 76, Starfield) write their text in UTF-8: the encoding is never a choice there, and *File* ▸ *File encoding…* is greyed out. The older games (Morrowind, Oblivion, Fallout 3, New Vegas, Skyrim LE) write each character on one byte in a Windows **code page** that depends

on the language: Western European (Windows-1252) for English, French, German or Spanish, Central European (1250) for Polish, Czech or Hungarian, Cyrillic (1251) for Russian, Greek (1253), Turkish (1254), Hebrew (1255), Arabic (1256), Baltic (1257), Vietnamese (1258), Thai (874), and multi-byte pages for Japanese (Shift_JIS), Simplified Chinese (GBK), Korean (EUC-KR) and Traditional Chinese (Big5). The table of languages and their page is in `assets/data/codepages.json`; a language that is not there goes to 1252.

When reading. For a plugin of an older game, the application first reads the file as before: UTF-8 when the bytes are, 1252 otherwise. If some texts went through the 1252 fallback, it then decides the page: that of the document's source language when it is known (a file reopened with its settings), otherwise it tries the candidate pages on a sample and asks the language detector (§ 6.3) which one gives a confident language written in that page — a Russian plugin is then re-read in 1251 and Russian is proposed as the source language —, otherwise 1252. The status bar says so (“Read as Cyrillic (Windows-1251) (detected)”) and the tab carries the page (“Oblivion · RUS→FRA · — · 1251”). The STRINGS of Skyrim LE follow the page of their language, the interface files and text files (§ 7.7) that of the source language when their format says “auto”.

When writing. Everything the export writes for the document — full plugin, override patch, STRINGS, text files — is in the page of the **target language**; the byte counter of the edit window (§ 8.5) counts in that page. A translation that contains a character the page cannot represent (an “œ” in Cyrillic, an “Ł” in 1252) is an anomaly of the *Unencodable character* check (§ 8.7, the character and its position), marked on the line; at export, the dialog counts those lines, refuses to write and offers *Replace with ? and export*, which puts a question mark in place of each offending character.

Forcing. *File ▸ File encoding…* (figure 18.5) shows the reading page and its reason, the writing page, and lets you choose either among the fifteen; the preview shows the first lines with non-ASCII characters decoded with the chosen reading page, which makes the right one recognisable at a glance; *Reload* reloads the file with that page (refused while the document has unsaved changes: save or discard first), the translations being reapplied by the mod dictionary as at a reopening. The choice is remembered in the header of the mod dictionary (`enc_in`, `enc_out`, like the plugin's identity, § 7.6) and in the pivot JSON: it comes back when the file is reopened. A file re-read then rewritten without modification stays identical byte for byte, whatever the page: a byte the page does not define is kept as is.

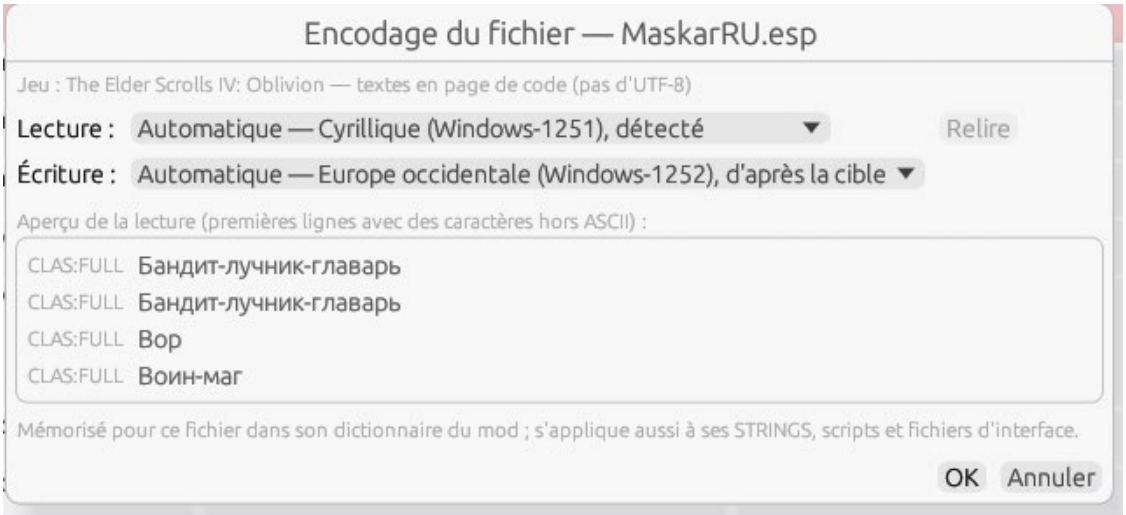


Figure 19.5 — File ▸ File encoding… on a Russian Oblivion plugin: detected reading page, writing page from the target, preview in Cyrillic.

> **Note —** Arabic and Hebrew are displayed without shaping nor right-to-left order until the bidirectional display is delivered (E-08); the shipped fonts cover Cyrillic, Greek, Vietnamese, Thai, Japanese, Chinese and Korean.

20. Frequently asked questions

The Dictionaries menu is grayed out, and so is Open…

No game is selected. First choose a *Game* in the toolbar (§ 7.1).

The Source list is empty or does not offer my language.

It offers only the languages that the file provides: the STRINGS present for a localized plugin, otherwise the language detected in the text (§ 6.3). For a localized plugin, add the STRINGS of the desired language next to the plugin.

Nothing is translated automatically on opening.

Check that *Game* points to the right game (to load its dictionary) and that the *Target* language is correct. After opening, the status bar says which dictionaries were found and how many lines they filled; if the mod dictionary is named but fills only a handful of lines, it is because it contains only those: it is rewritten at each save and export (§ 7.6) — save or export once the translation is well advanced, or use *Dictionaries ▸ Save the mod dictionary*.

Changing the game reloaded my file.

This is the expected behavior: the active document is read again with the rules of the new game, and its translations are kept by matching (§ 6.2). The other documents are not affected. If you answered *Cancel* to the confirmation, nothing changed.

A “□” box appears in place of a character.

This is an old encoding problem (Latin-1 control characters), now fixed automatically on reading. Regenerate the dictionary concerned if an old file still showed it.

The spell checker says “No dictionary for the target language”.

Put a `.aff` / `.dic` pair in the folder shown by the spell checker window.

21. Glossary

Term	Definition
BSA / BA2	Bethesda archives grouping resources (textures, STRINGS, interface…).
DSD	Dynamic String Distributor: SKSE framework (Skyrim SE/AE) that injects translations at launch without modifying the mod.
EDID	Editor ID: readable name of a record in the game's editor.
EET	ESP-ESM Translator: reference tool, and its .eet database format.
FOMOD	Mod installer format (handled by XIMOD Architect).
FormID	Unique numeric identifier of a record in a plugin.
locres	Localization file of the Unreal engine (Oblivion Remastered).
Plugin	.esp / .esm / .esl mod file.
SKSE	Skyrim Script Extender: extension that loads script mods.
SST	xTranslator translation database format.
STRINGS	External text tables of recent games (.STRINGS/.DLSTRINGS/.ILSTRINGS).
ximdb	Compact dictionary format specific to XIMOD.

Appendix — Summary of formats

Type	Read	Written
Translation databases	.sst, .eet, .dsd, .json, .xml	.sst, .eet, .dsd, .json, .csv, .xml
Plugins	.esp, .esm, .esl (extraction)	.esp/.esm (full plugin, override patch)
String tables	.strings, .dlstrings, .ilstrings	.strings, .dlstrings, .ilstrings
Dictionaries	.ximdb	.ximdb
Archives	.bsa, .ba2 (browsing/extraction)	—
Command line (§ 18)	Batch file .json (format 1); manifest.json (format 1)	manifest.json (format 1); .csv reports; JSON output (--json)